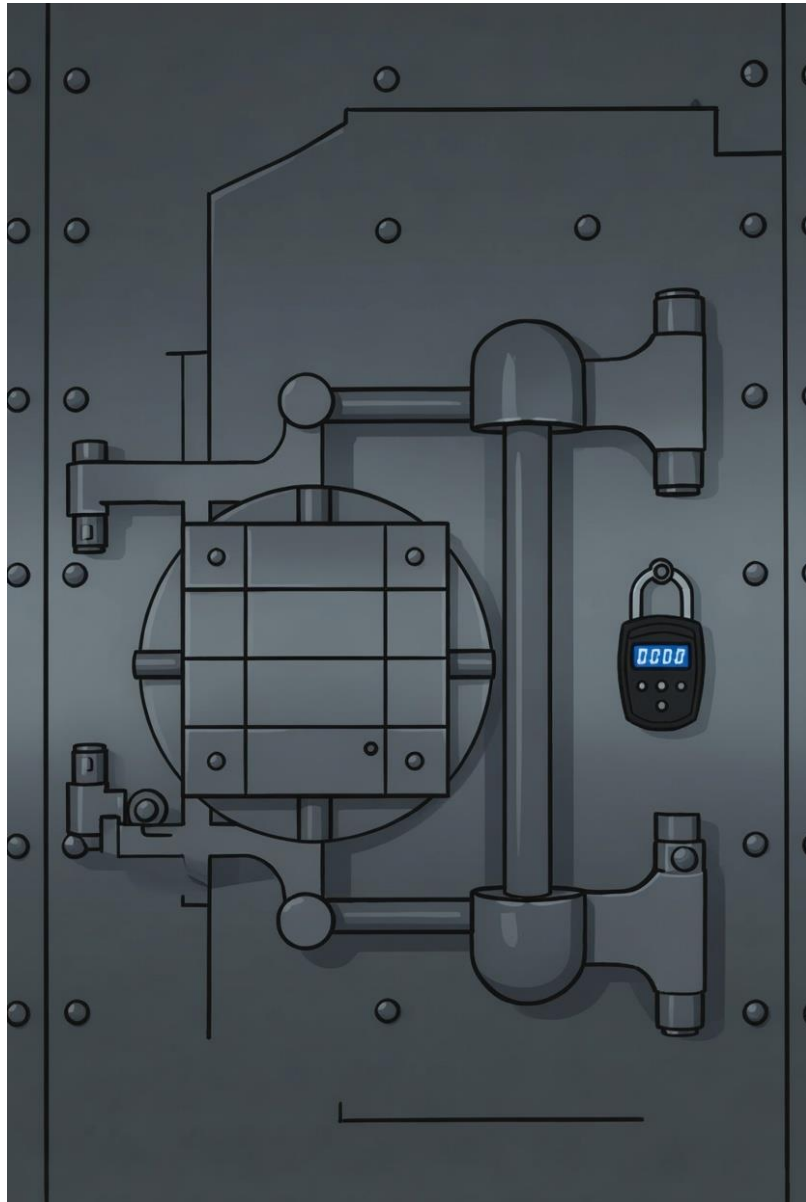




Password Manager
Secure Local Password Storage and Management System

Author: Peters Temidayo

Table of Contents

1. Introduction	2
2. Analysis	4
2.1 identifying the Problem and Computational Methods Needed	4
2.2 Identifying and Describing Stakeholders and Requirements	4
2.3 Research on the Problem and Existing Solutions	6
Key Research Points	6
Existing solutions analysed:	6
2.4 Description and Justification of Essential Features	7
Essential features chosen and justified from research:.....	7
2.5 Identification and Justification of Limitations	11
2.6 Hardware and Software Requirements	12
2.7 Measurable Success Criteria	12
3. Design	15
3.1 Systematic Decomposition.....	16
3.2 Structure of the Solution	16
3.3 Chosen Algorithms and Justification.....	17
PBKDF2 Key Derivation	17
AES-256 Encryption	17
Exponential Backoff for Lockout	18
3.4 Usability Features Added.....	18
3.5 Key Variables, Data Structures, Classes and Validation	20
3.6 Test Data and Plan	26
Iterative Test Data	27
Screenshots - Testing the Program.....	28
Master Password Creation and validation	28
Master Password Rejection.....	28
Master Password Incorrect Login.....	29
Exponential Lock out Check	30
Adding Passwords.....	31
Viewing Password	32

Deleting Passwords.....	33
Copying Passwords.....	34
ClipBoard Check and timed password reveal	35
Before.....	36
After	36
Post Development Test Data	38
Testing Screenshots	38
Empty Fields.....	38
Common Password Rejection	38
Encryption Verification	39
Master Password Change	40
Before.....	40
After	40
Lockout Persistence.....	41
After	41
4. Development.....	41
4.1 Iterative Development Process	42
4.2 Evidence of Prototyping and Milestones.....	44
Main Version History	45
Version 1 (Beta)	45
Version 2 (Added master password change option + Encryption and Hashing) ..	45
Version 3 (Delete Entry Option + Expanded View Entry Option + Dark Theme) ...	46
4.3 Code Modules	47
4.4 Code Comments	49
4.5 Variable and Structure Names	50
4.6 Validation.....	50
4.7 Review	51
5. Testing: Developmental.....	52
5.1 Evidence of Testing During Iterative Development.....	52
EndOfStreamException Reading master.dat	52
CryptographicOperations.FixedTimeEquals Not Available	52
Password Policy Not Enforced Initially	53

Blank Entry Validation Missing.....	53
Clipboard Timer Duration	53
5.2 Failed Tests and Remedial Actions	53
6. Testing: Evaluative	54
6.1 Post Development Testing	54
6.2 Usability Testing.....	55
7. Evaluation	56
7.1 Evaluation Against Success Criteria	56
7.2 Evaluation of Usability Features	57
7.3 Limitations and Maintenance Considerations	58
7.4 Potential Improvements	58
References and Resources.....	59
Appendix	61
Program.cs	61
LoginForm.cs	62
LoginForm.Designer.cs	68
MainForm.cs.....	71
MainForm.Designer.cs.....	73
ViewPasswordForm.cs	73
ViewPasswordForm.Designer.cs.....	73
ChangePasswordForm.cs	73
ChangePasswordForm.Designer.cs	73
ConfirmPasswordForm.cs	73
ConfirmPasswordForm.Designer.cs.....	73
PasswordUtils.cs	73
LockoutManager.cs	73
SecurityPolicy.cs	73

1. Introduction

This programming project was designed to enable users effectively manage passwords with the use of a secure, local-only password manager, which was developed in C# using Windows Forms and .NET 8.0. The project was designed to eliminate security risks that emanates from using the same password across multiple sites. The solution allows users to securely save passwords on their computers in an encrypted data file, removing the risk of system breach.

The application allows users to:

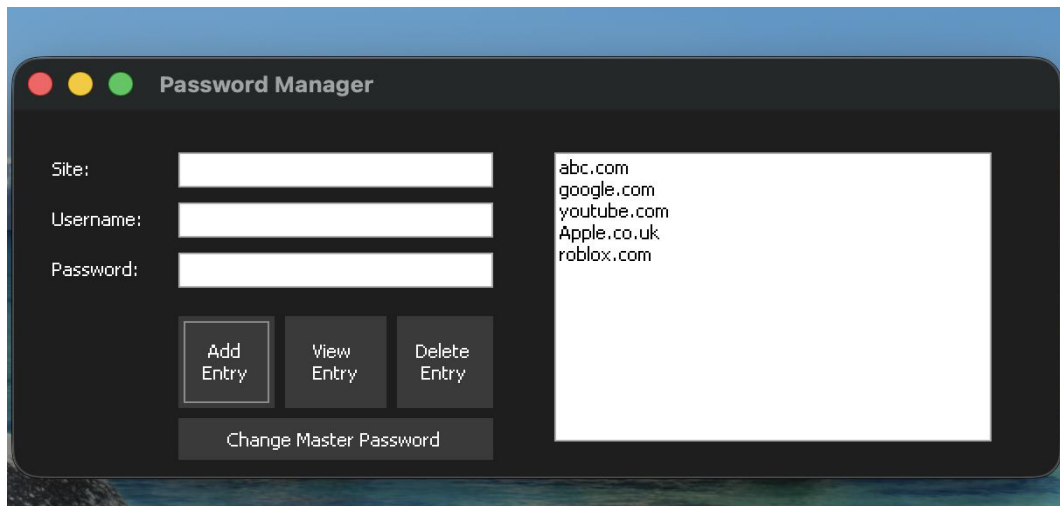
- Create and verify a strong master password
- Add, view, and delete encrypted password entries for websites/services.
- Copy passwords to clipboard with automatic clearing for security.
- Temporarily reveal passwords with auto-hide timer.
- Change the master password securely.
- Protect against brute-force attacks via exponential logout.

The motivation for creating this program is based on my observation around challenges faced by many users while using their computers everyday e.g. password reuse, plain text storage in notes or browsers which often results in personal data being exposed online. With my solution, I aim to address these problems by implementing industry standard cryptography techniques, avoiding cloud risks and simplifying end users experience.

The program developed required independent research beyond the A level syllabus which covers basic encryption/hashing concepts but does not extend to PBKDF2 implementation or in depth AES Encryption. The research deep dives on OWASP (Open Web Application Security Project), NIST (National Institute of Standards and Technology) guidelines, as well as .NET cryptography APIs.

The program was built in stages over several months, with repeated testing and refining to meet success criteria.

The program opens to a login page where the user can input their login credentials. Once validated, it takes them to the main page where they can store passwords for multiple sites as show in the image below.



Above is the core user interface for entry management. An intuitive, modular layout was chosen for simplicity.

2. Analysis

2.1 identifying the Problem and Computational Methods Needed

Recent online studies indicate that most system users face the challenge of managing multiple passwords for different online account. On average 100+ passwords per person¹, and this challenge increases the likelihood of many users adopting weak, reused, or insecurely storing passwords on various sites. This leads to risks including credential stuffing attacks, phishing, identity theft, and local device compromise.

To address the problem, I adopted computational methods for three practical reasons:

- a. First, the cryptographic operations involved for encrypting passwords securely are not feasible to perform manually. PBKDF2 with 100,000 iterations of SHA 256 would take approximately 15 minutes to complete by hand, compared to milliseconds on a modern processor.
- b. Second, the average user manages over 100 passwords, exceeds human working memory capacity (typically 3-5 passwords at best)².

¹ Kamile Viezelyte: Juggling security <https://nordpass.com/blog/how-many-passwords-does-average-person-have/>

² Nelson Cowan: The Magical Mystery 4: **How is Working Memory Capacity Limited, and Why?** <https://pmc.ncbi.nlm.nih.gov/articles/PMC2864034/>

- c. Third, manual encryption of passwords is highly prone to error. A single error in the AES ciphertext renders the entire plaintext unrecoverable. These constraints make computational methods essential, not merely convenient³.

Without computation, secure management at large scale is impossible.

2.2 Identifying and Describing Stakeholders and Requirements

Identified Stakeholders:

- **Members of the public:** All users of computer systems who manage diverse online accounts (for personal or professional work) such as learning platforms, bank apps, social media accounts, gaming apps etc. require quick access to their system without memorising complex passwords. Part of my objective for this project was to develop a viable solution that could function outside the classroom setting, to help privacy conscious users avoid potential identity theft or loss of asset.
- **Private Institutions:** Private institutions can suffer loss of sensitive commercial information, as a result of a cyber-attacks, if employees do not use secure methods for storing their passwords. Consumers and customers would also lose confidence in such institutions who do not encourage users who interact with their system to use strong passwords.
- **Public Institutions:** Similar to private institutions, the impact of employees not using a secure password has far reaching consequences for regulatory and other state agencies; particularly because public institutions manage large data sets containing sensitive information, which can impact national security.
- **Educational bodies and professional bodies:** This solution can be further enhanced and referenced as a learning material to train students and stakeholders on effective password management.

User interaction with the proposed solution:

- Set one strong master password on first run.
- Add entries (site, username, password).
- Authenticate to view and copy passwords securely.

³ Guy Bavly: The Challenges of Seamless Encryption <https://community.ibm.com/community/user/blogs/guy-bavly/2023/02/09/the-challenges-of-seamless-encryption#:~:text=Manual%20processes%20are%20time%2Dconsuming.vulnerable:%20with%20no%20active%20effort.>

- Change master password if compromised or forgotten.

Why it works:

- Local storage ensures no third-party access.
- Usability features (clipboard timer, dark theme) reduce friction while maintaining security.
- Lockout protects against guessing if device left unlocked.

For students, this solution is appropriate because cloud-based alternatives require internet connectivity that may be unavailable in school environments, and high subscription costs are not affordable for this user group. The local approach for this solution is the best approach, protecting against sophisticated targeted attacks.

For privacy-conscious users who prioritise data sovereignty, the offline nature of the solution eliminates vendor lock-in and cloud breach risks, as demonstrated by the LastPass 2022 incident. This makes the solution more appropriate than SaaS alternatives for users.

2.3 Research on the Problem and Existing Solutions

Key Research Points

My research covered a detailed review of three key points an effective password manager should entail:

1. SHA-256 alone is too fast and does not include a salt. This means attackers can brute force hashes quickly using GPUs or other specialised hardware.
2. Most Industry level Solutions have a salt attached to hashed passwords. If there is no built-in salt it makes hashed passwords vulnerable to rainbow tables.
3. Modern guidance from online sources strongly recommends using slow, tuneable functions such as PBKDF2, scrypt, or Argon2 instead of plain hashes for passwords.

PBKDF2 is commonly used as a KDF (Key Derivation Function) in modern password managers. PBKDF2 makes brute force attacking much slower by iterating a hash multiple times, increasing the computational cost for attackers.

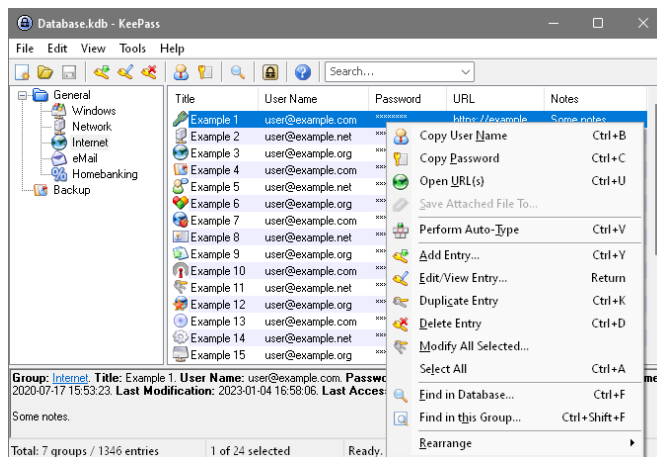
AES 256 is the industry standard when it comes to symmetric encryption algorithms. Many password managers use it to encrypt vault data after deriving the key with PBKDF2.

My research led me to reject simple hashing (e.g. SHA 256 alone) as it is not up to industry standard; this informed the adoption of PBKDF2 and AES 256.

Existing solutions analysed:

As part of the research, I examined some existing commercial, open-source password managers and authoritative security systems, and below are some of my observations:

1. **KeePass:** Runs completely locally, uses AES 256 and PBKDF2. Does not use the cloud, hence it is not susceptible to cloud based attacks. Identifiable strengths: Strong security model and open source. key features include Salting, high iteration key derivation and encryption for each entry. KeePass seemed like an efficient solution, hence it became the model template I referenced when programming my own password manager.



2. **Bitwarden:** Uses AES 256 and PBKDF2. It's a **self-hosted local vault**. Deemed not viable because the default cloud sync system introduces **more risk points** compared to local-only storage i.e. server breaches and relying on third party infrastructure to function properly.
3. **1Password / LastPass:** Proprietary software unlike previous solutions covered, is a cloud-based system, has a history of breaches (LastPass 2022). Deemed not viable because dependency on vendor could cause issues and the subscription model limits usage.

While Argon2id offers superior GPU resistance, PBKDF2-SHA256 was selected for three reasons. It is NIST FIPS 140-2 approved, ensuring regulatory compliance. It has native .NET support, eliminating external dependencies that could introduce vulnerabilities. With 100,000 iterations, it provides sufficient slowdown (approximately 100ms on target hardware) against brute force attacks while maintaining acceptable usability.

AES 256 was preferred over ChaCha20-Poly1305 because .NET's AES class provides hardware acceleration on x86 AES-NI instructions, offering significant performance improvements on modern CPUs that are essential for responsive UI operation.

2.4 Description and Justification of Essential Features

Essential features chosen and justified from research:

- **Master password hashing with PBKDF2 SHA256:** The master password is processed using PBKDF2 with SHA 256 as the underlying hash function. PBKDF2 applies salting and thousands of iterations to derive a cryptographic key from the user's password. This significantly slows offline brute-force attacks and prevents rainbow table attacks, as each salt makes precomputed tables ineffective.

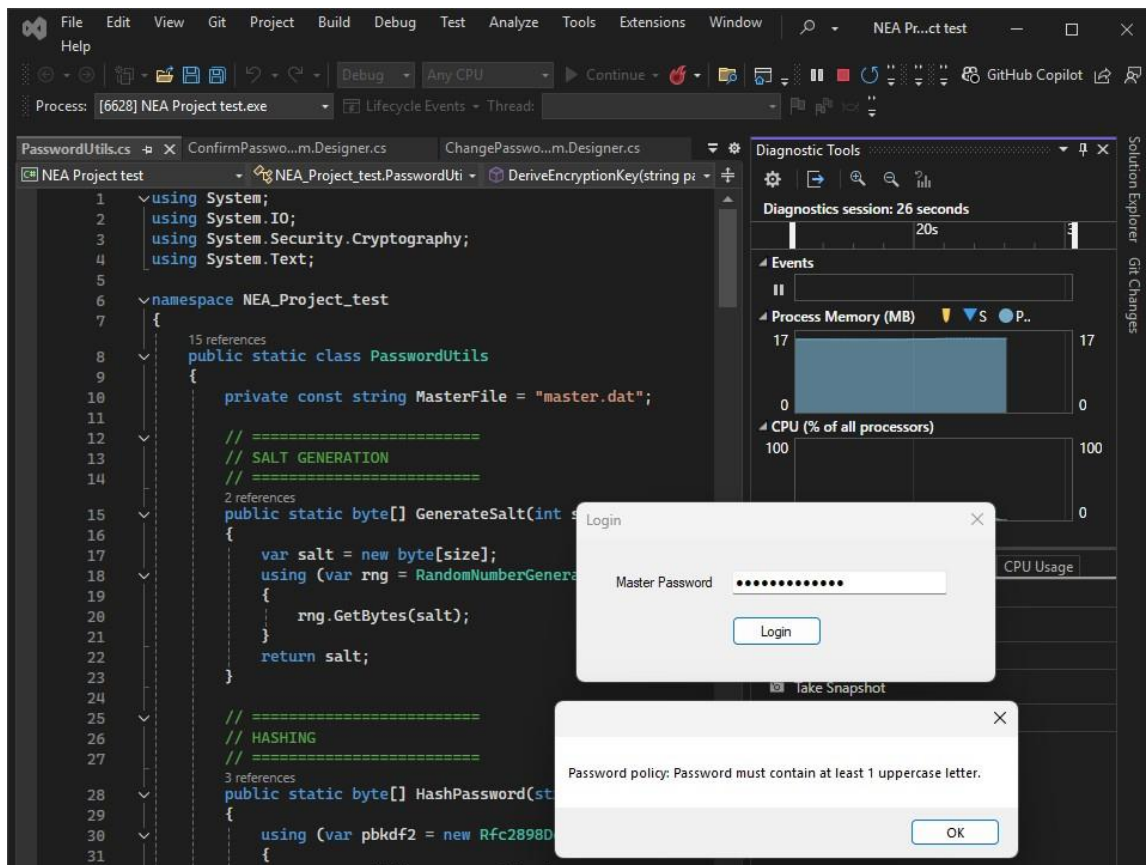
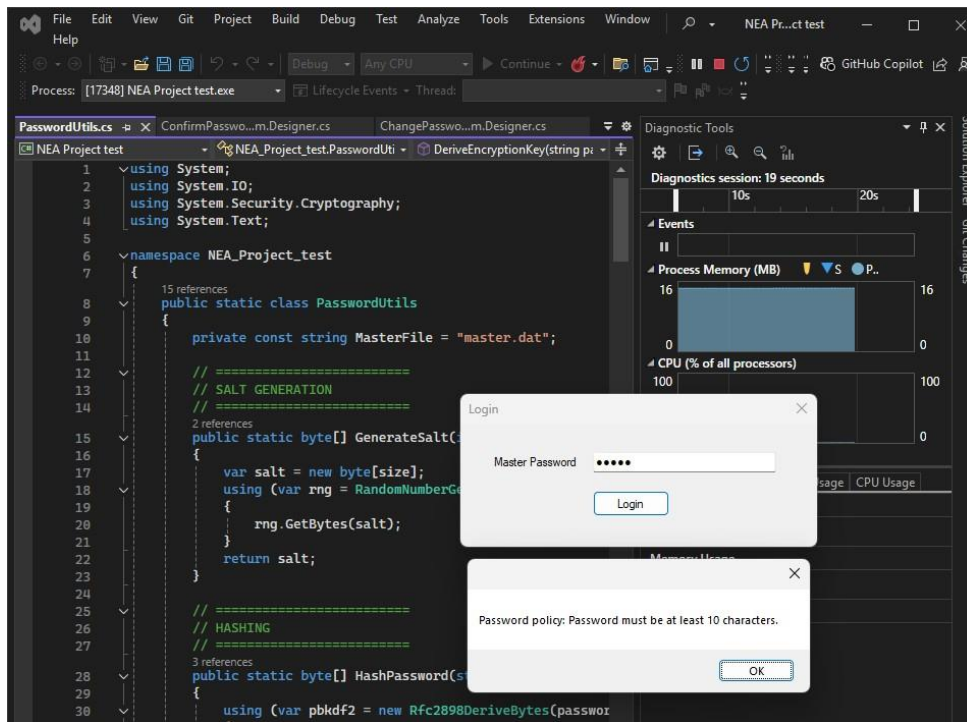
Rather than storing the password directly, the derived key is used for vault encryption, ensuring the master password itself is never stored in plaintext.

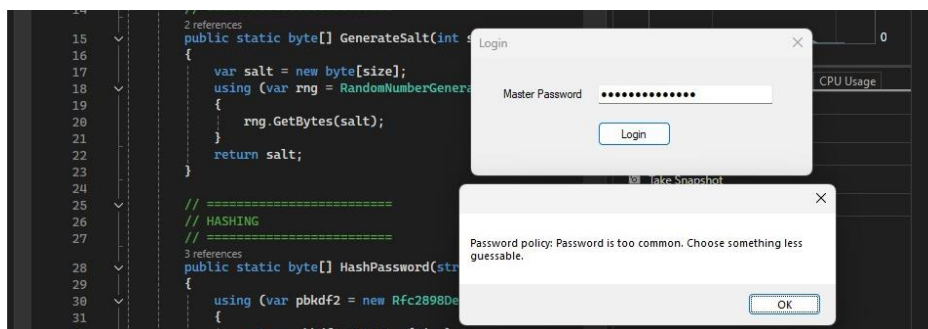
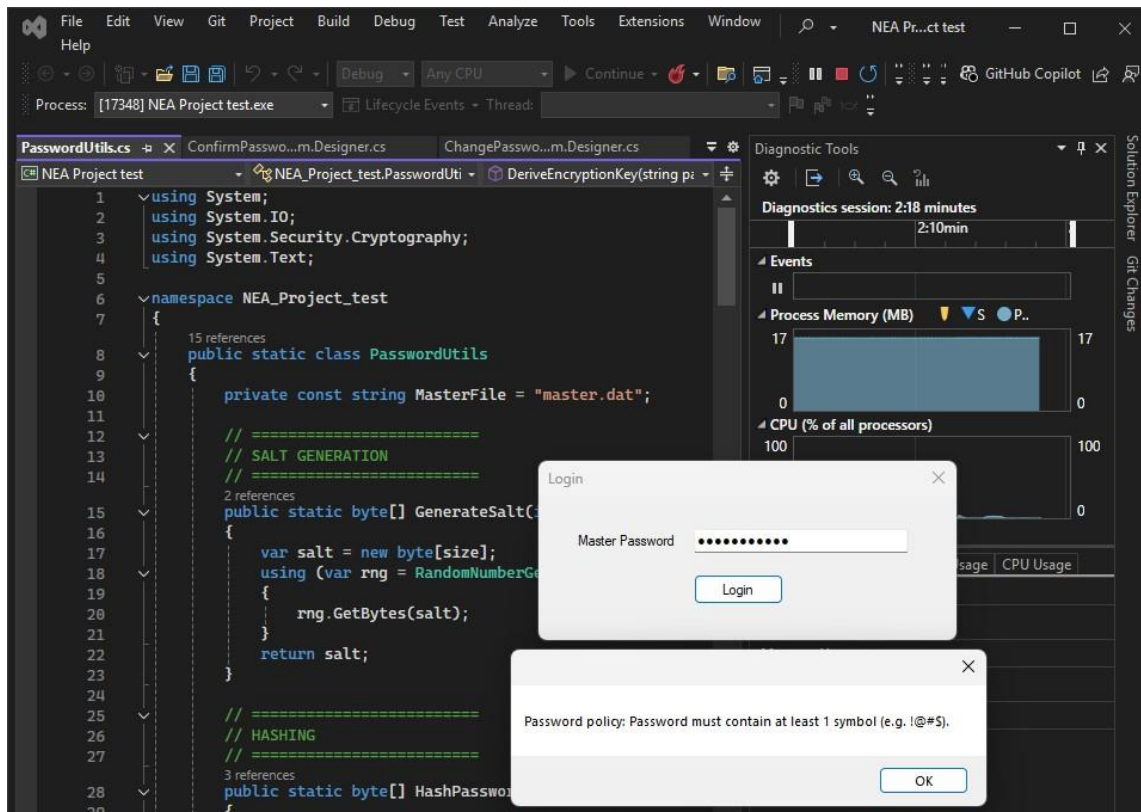
- **AES 256 Encryption:** Each stored credential is encrypted using AES 256. Encrypting entries individually ensures that even if part of the vault were compromised, other entries remain protected. Using unique initialisation vectors (IVs) ensures identical passwords do not produce identical ciphertext, preventing pattern analysis.

AES 256 was selected due to its widespread industry adoption and recognition as a secure symmetric encryption standard.

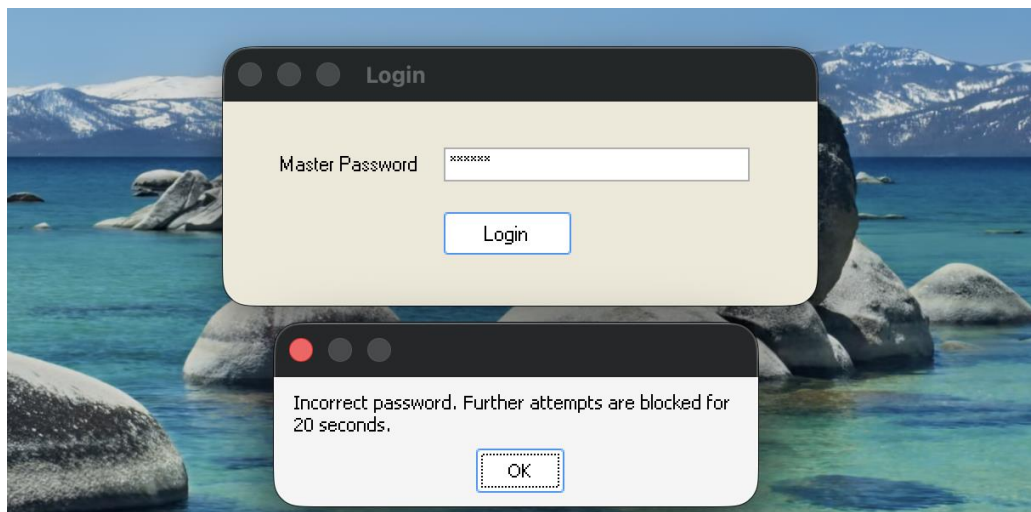
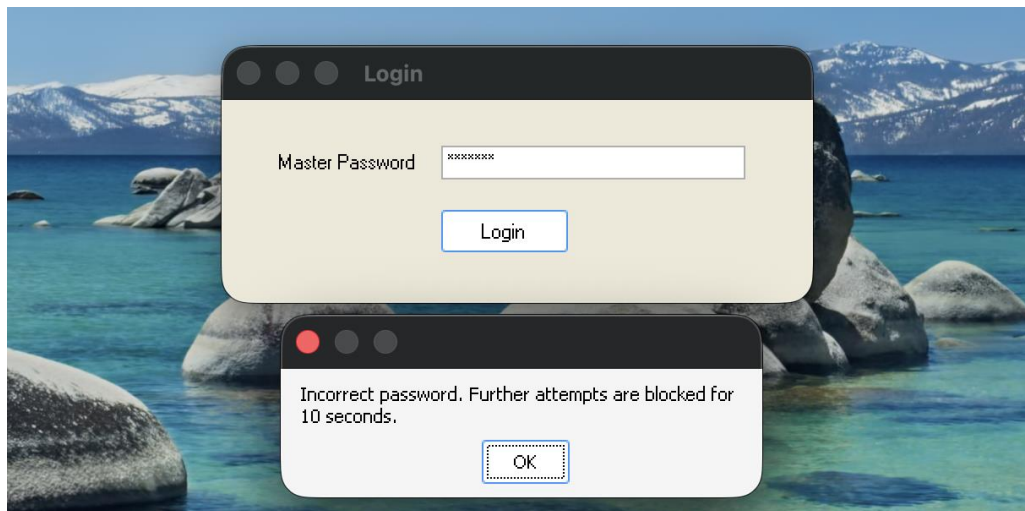
Below are some examples of the enforced password policy:

- **Strong master password policy:** A password policy was implemented requiring a minimum length of 10 characters and a mixture of character types. This increases password entropy and reduces the likelihood of successful guessing attacks.
- This decision was informed by NIST guidance, which emphasises creating passwords that resist brute-force and dictionary attacks. Additionally, restricting common passwords helps prevent users from selecting weak credentials.





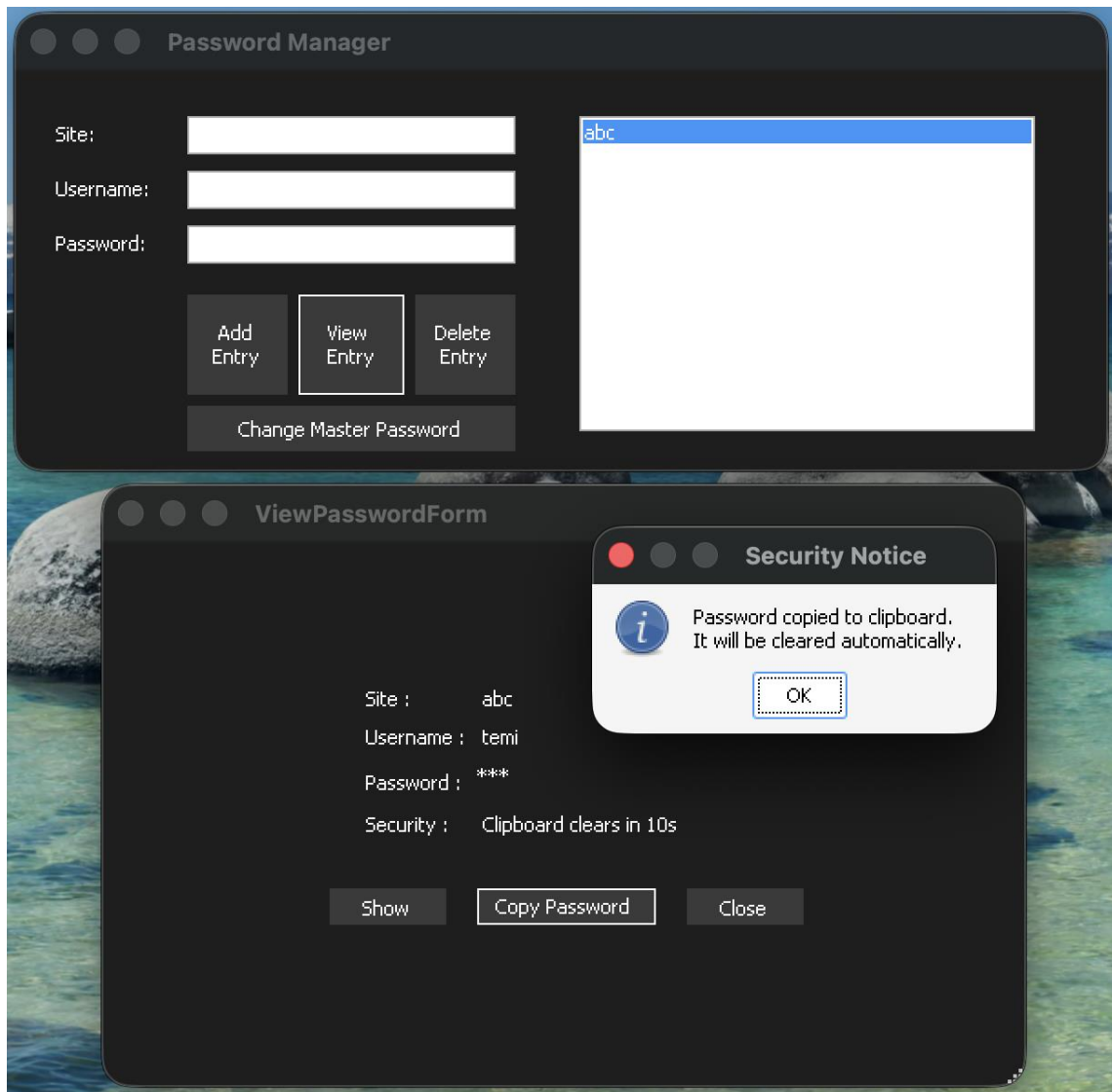
1. **Exponential backoff lockout** (5s → 10s → ... → 300s): Additionally, to deter brute-force attackers from running programs with rainbow tables, I added a timeout lock that increases exponentially without permanent lock.



2. **Clipboard auto clear after 10s:** I also had the program clear your clipboard after 1 seconds. In addition, I added a **timed password reveal** to balance visibility and security

Some malwares specifically:

- Monitors your clipboard
- Reads copied data
- Sends it to attackers



2.5 Identification and Justification of Limitations

- Local files vulnerable to physical theft: This risk was accepted because the solution targets personal devices with existing physical security measures (PINs, biometrics). The risk is mitigated by the strong master password requirement using PBKDF2.
- No built-in password generator: Omitted to maintain focus on core storage security. Implementation would require integration with cryptographically secure random number generators and character set management, exceeding project scope.
- Master password change does not re-encrypt existing entries: This represents a deliberate trade-off. Full re-encryption would require decrypting all entries

simultaneously, creating a window where all passwords exist in plaintext in memory. The chosen approach limits exposure to single-entry decryption on demand.

- Single-user only: Multi-user profiles would require database schema redesign with user tables and foreign key relationships, exceeding project scope. This is mitigated by the target use case. Personal devices rarely require multi-user access.

The Project focused more on secure storage, as other extensions to the program would exceed the project scope.

2.6 Hardware and Software Requirements

Quite minimal: Windows 10/11 with .NET 8 runtime installed. 4GB RAM, any modern CPU. No internet needed.

2.7 Measurable Success Criteria

Num	Description	Test Procedure	Justification	Expected Outcome
	Master password authentication using PBKDF2-SHA256 hashing	Enter a valid master password on first run, then re-enter on the login screen. Verify login succeeds. Enter an incorrect password three times and confirm access is denied. Inspect master.dat to confirm that no plaintext is stored.	PBKDF2-SHA256 with 100,000 iterations is an industry standard key derivation function that significantly slows brute-force attacks. The master key is kept separate from the session key to support rotation without requiring reauthentication.	Correct password grants access. Incorrect password is rejected with an error message. master.dat contains only

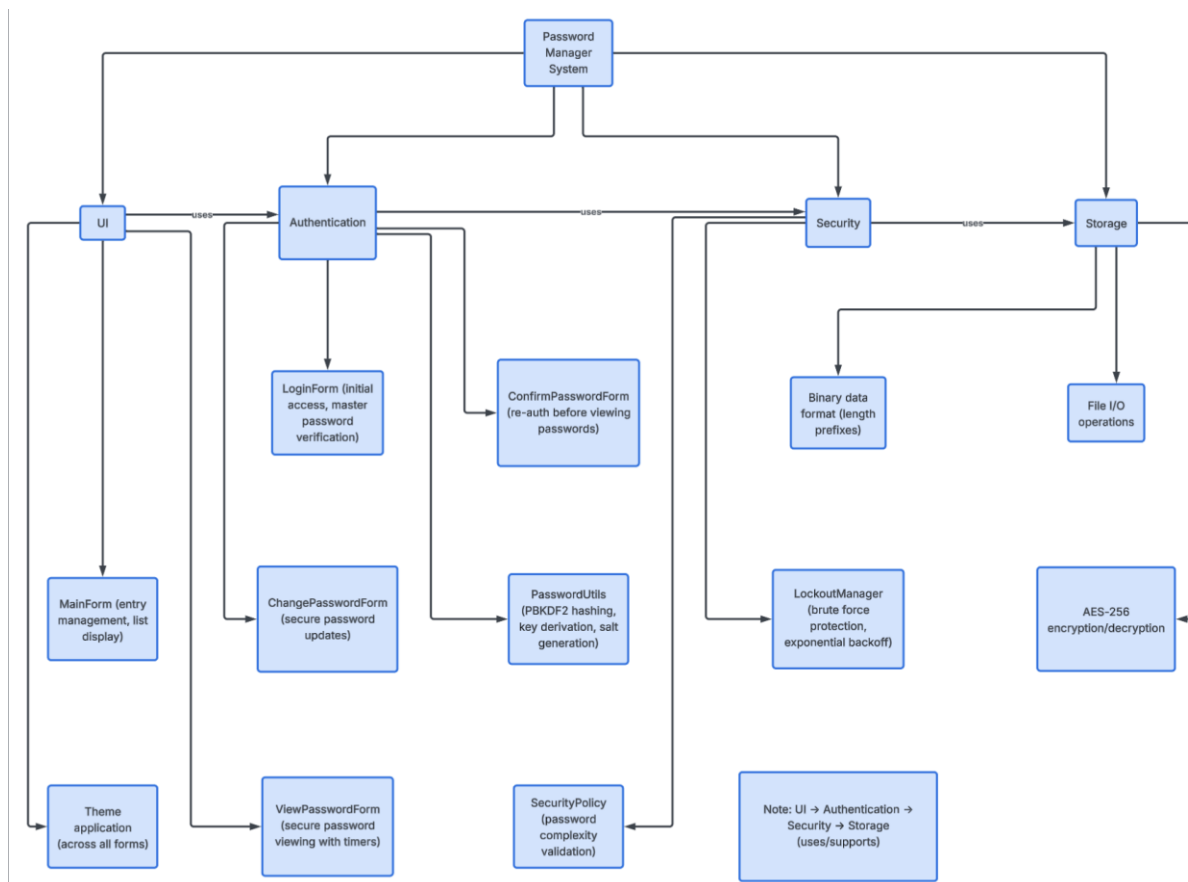
				a salt and hash.
2	Password entries are encrypted with AES 256 CBC and can only be decrypted with the correct derived key	Add a password entry and open passwords.dat. Confirm all content is ciphertext. Attempt to decrypt the file using a different key and verify decryption fails. Decrypt with the correct master key and verify the entry is recovered.	AES 256-CBC is a NIST-approved symmetric encryption standard that ensures confidentiality of stored credentials. Each entry is encrypted individually via EncryptString() and DecryptBytes() in PasswordUtils.cs, so a single compromised entry does not expose others.	passwords.dat contains only ciphertext with no readable strings. Decryption with the wrong key fails or returns garbage. Correct key recovers the original entry accurately.
3	Session key rotates every 30 seconds while the master key remains persistent	Launch the application with debug logging enabled. Observe the console for 'Session key rotated' messages. Compare the 8-character key fingerprints printed at each rotation to confirm they differ. Verify the UI remains responsive and no errors are thrown during rotation.	Periodic session key rotation limits the window of exposure if a key is compromised in memory (OWASP Session Management guidelines). Separating the short-lived session key from the persistent master key reduces the risk surface without impacting usability.	A new 'Session key rotated' message with a distinct fingerprint appears every 30 seconds. The UI continues functioning normally throughout rotation and no exceptions are raised.
4	Master password complexity rules are enforced at creation	Attempt to set a weak master password (e.g., 'pass' — fewer than 10 characters, no symbols). Then set a valid password (e.g., 'StrongP@ss1'). Use ValidatePasswordComplexity() in SecurityPolicy.cs to confirm validation logic triggers correctly for each case.	Enforcing password complexity prevents trivially guessable master passwords (OWASP Authentication Cheat Sheet; NIST SP 800-63B). The validator checks minimum length, uppercase and lowercase characters, digits, symbols, and common patterns.	Weak password is rejected with a specific reason (e.g., 'Must contain at least 1 symbol'). Valid password is accepted without error.
5	Exponential backoff lockout activates after repeated failed login attempts	Enter an incorrect password five or more times in succession. After each failure, measure the delay before the next attempt is permitted using GetRemainingLockoutSeconds() in LockoutManager.cs. Log in successfully and confirm the lockout counter resets.	Exponential backoff deters automated brute-force attacks by increasing wait time after each failure (OWASP Brute Force Prevention). The formula applies a base delay of 5 seconds, doubling per attempt up to a cap of 300 seconds. No permanent lockout is imposed to avoid denial-of-service risk.	Delays increase as expected (5s, 10s, 20s, etc.). A successful login resets the counter. The user is never permanently locked out.

6	Clipboard is automatically cleared after a password is copied	Copy a password via the <code>btnCopyPassword_Click()</code> handler in <code>ViewPasswordForm.cs</code> . Wait 15 seconds without manually clearing the clipboard. Check the clipboard contents after the timer elapses. Minimise the application mid-timer and verify clearing still occurs.	Clipboard persistence is a common attack vector for credential theft (OWASP Transient Secrets guidance). Auto-clearing via <code>Clipboard.Clear()</code> on timer tick ensures the password does not remain accessible after the user has finished with it.	Password is available in the clipboard immediately after copying. After 15 seconds, the clipboard is cleared — confirmed by a paste attempt returning empty — regardless of whether the app is in focus.
7	Password reveal is temporary and auto-hides after a timeout	Toggle password visibility using <code>btnToggleVisibility_Click()</code> in <code>ViewPasswordForm.cs</code> . Confirm the password is shown in plaintext. Wait 10 seconds without interacting and verify the field reverts to asterisks automatically.	Temporary reveal balances usability with security, consistent with best practices observed in reference implementations such as KeePass. The <code>revealTimer</code> in code enforces the timeout to prevent indefinite exposure on unattended screens.	Password is briefly visible in plaintext after toggling. After 10 seconds it automatically reverts to masked asterisks with no manual input required.
8	Master password can be changed securely without affecting stored entries	Change the master password via <code>ChangePasswordForm.cs</code> . Attempt to log in using the old password and confirm it is rejected. Log in with the new password and verify all existing entries are still accessible and correctly decrypted.	Supporting password changes without data loss is required for recovery if credentials are compromised (NIST SP 800-63B). The code generates a new salt and hash via <code>SaveMasterHash()</code> and re-derives the encryption key, so entries remain accessible.	Old password is rejected after the change. New password grants access. All previously stored entries decrypt correctly.
9	Application handles erroneous and malformed inputs without crashing	Submit the 'Add Entry' form with all fields empty. Deliberately corrupt <code>passwords.dat</code> (e.g., truncate or overwrite bytes) and attempt to load the application. Observe error handling in both cases.	Robust input validation and error handling are essential for real-world reliability (OWASP Input Validation Cheat Sheet). Null and whitespace checks (<code>string.IsNullOrEmpty()</code>) and try-catch blocks in utility classes prevent unhandled exceptions from exposing internal state.	Empty form submission shows a clear error message (e.g., 'All fields are required') without crashing. Corrupted file load is caught gracefully with an appropriate error message and no

				unhandled exception.
10	UI is consistently themed in dark mode and usable within a reasonable time	Time the full workflow of adding a new entry and viewing it. Target is under 30 seconds. Inspect all forms for consistent background and foreground colours applied via ApplyTheme() and check for any mismatched elements.	Consistent theming improves usability and reduces visual strain for the target stakeholders (research: dark themes lower eye fatigue during extended use). ApplyTheme() applies BackColor and ForeColor uniformly across all forms.	Full add-and-view workflow completes in under 30 seconds. All forms display consistent dark-mode colours with no visible mismatches or unstyled elements.
11	Sensitive data is cleared from memory and UI when a form is closed	Open ViewPasswordForm.cs with a decrypted entry, then close the form. Inspect text fields and active timers in a debug session to confirm SecureCleanup() has been called and no plaintext remains in memory or visible UI state.	Minimising the lifetime of sensitive data in memory reduces the risk of extraction via memory dumps or debugging tools (OWASP Memory Management guidance). SecureCleanup() clears text fields and cancels active timers on form close.	All text fields are empty and timers are stopped after the form closes. No plaintext passwords remain visible or accessible in UI or debug-inspected memory.

3. Design

3.1 Systematic Decomposition



I decomposed the system into four modules based on function - (i) Authentication handles getting users in and out. (ii) Security manages protection mechanisms. (iii) Storage deals with encryption and files. (iv) User Interface (UI) handles what the user sees and interacts with. This split means I can change how encryption works without touching the interface and update the appearance without the risk of changing the security code. Each module has one clear job.

3.2 Structure of the Solution

The system uses four layers that pass data in one direction.

Presentation Layer: The forms (LoginForm, MainForm, ViewPasswordForm, ChangePasswordForm, ConfirmPasswordForm). These handle buttons, text boxes, and labels. They do not perform any cryptography. They call other complex classes for that.

Business Logic Layer: PasswordUtils, LockoutManager, SecurityPolicy. These contain the actual algorithms and rules. I designed them to be stateless so I can test them without running the full UI.

Data Access Layer: File reading and writing in PasswordUtils. All AES encryption happens here before data touches the disk. I used a binary format with length prefixes so I can change the format later without breaking old files.

Security Layer: Validation, encryption, and cleanup code that runs across all layers. These are static classes so I do not need to create objects every time.

Data Flow: User types something → Form checks it is not empty → Security checks it follows rules → Cryptography runs → Data gets saved. The flow is one-way to prevent loops that could cause race conditions."

3.3 Chosen Algorithms and Justification

I selected two core cryptographic algorithms based on my research: PBKDF2 for key derivation and AES 256 for encryption. Both are widely used in industry and have native .NET support.

PBKDF2 Key Derivation

The master password is never stored directly. Instead, I derive a key using PBKDF2 SHA256 with 100,000 iterations and a random 16-byte salt.

```
// HASHING
public static byte[] HashPassword(string password, byte[] salt, int iterations = 100000, int size = 32)
{
    using (var pbkdf2 = new Rfc2898DeriveBytes(password, salt, iterations, HashAlgorithmName.SHA256))
    {
        return pbkdf2.GetBytes(size);
    }
}
```

The method uses Rfc2898DeriveBytes with SHA256, 100,000 iterations, and the provided salt to generate a 32-byte hash.

AES 256 Encryption

Each password entry is encrypted individually using AES 256 in CBC mode with a random initialization vector.

I settled with putting the 16-byte IV on the front of each encrypted entry. This keeps everything in one file. The IV is random every time, so encrypting the same password twice looks completely different, which stops pattern spotting. I use "using" statements so memory gets cleaned up automatically, even if something breaks.

```

PasswordUtils.cs

// ENCRYPT STRING
public static byte[] EncryptString(string plainText, byte[] key)
{
    if (plainText == null)
        throw new ArgumentNullException(nameof(plainText));
    if (key == null)
        throw new ArgumentNullException(nameof(key));

    using (var aes = Aes.Create())
    {
        aes.Key = key;
        aes.GenerateIV();

        using (var ms = new MemoryStream())
        {
            // Write IV first
            ms.Write(aes.IV, 0, aes.IV.Length);

            using (var encryptor = aes.CreateEncryptor())
            using (var cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write))
            using (var sw = new StreamWriter(cs, Encoding.UTF8))
            {
                sw.Write(plainText);
            }

            return ms.ToArray();
        }
    }
}

```

The method generates a random IV, writes it to the output stream first, then encrypts the plaintext using AES 256.

Exponential Backoff for Lockout

Failed login attempts trigger increasing delays to deter brute force attacks.

The delay doubles with each attempt, starting at 5 seconds and capping at 300 seconds (5 minutes). This prevents automated guessing while avoiding permanent denial of service.

```

// Returns remaining seconds to wait (0 if allowed now)
public static int GetRemainingLockoutSeconds()
{
    var info = Load();
    if (info.FailedCount <= 0) return 0;

    // exponential backoff: delay = BaseDelay * 2^(FailedCount-1)
    double delay = BaseDelaySeconds * Math.Pow(2, info.FailedCount - 1);
    if (delay > MaxDelaySeconds) delay = MaxDelaySeconds;

    DateTime windowStart = info.LastFailed;
    double elapsed = (DateTime.UtcNow - windowStart).TotalSeconds;
    int remaining = (int)Math.Ceiling(delay - elapsed);
    return remaining > 0 ? remaining : 0;
}
}

```

The method calculates delay as $5 \times 2^{(\text{failedCount}-1)}$, capped at 300 seconds, then returns remaining time based on when the last failure occurred.

3.4 Usability Features Added

A system that is difficult to use will be abandoned or worked around, which creates more risk than it prevents. With that in mind, a lot of usability features were added.

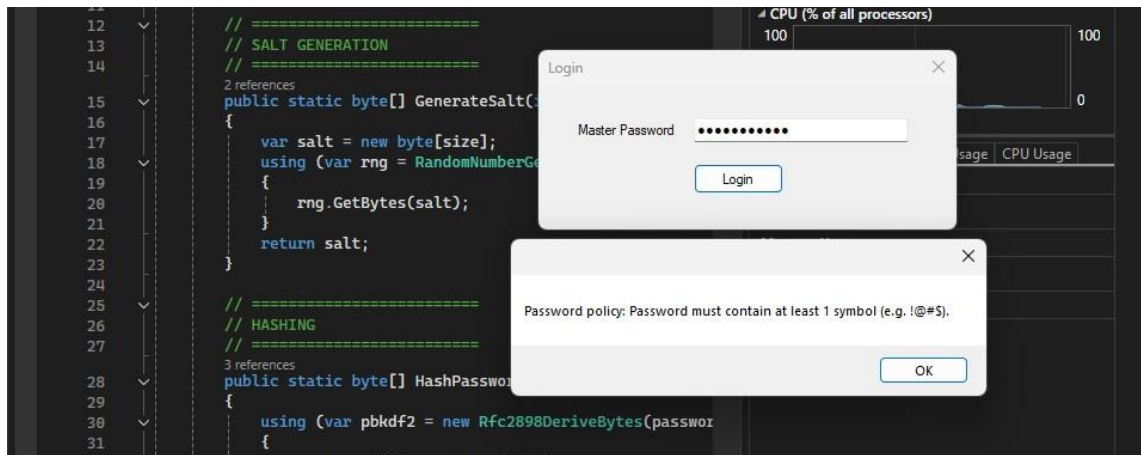
The dark theme was applied across all forms. This was chosen because password managers are used on screens in varying lighting conditions, and a dark background reduces eye strain during extended use. It also makes the application feel distinct from your standard Windows forms application, which helps users immediately recognise they are in a secure space. The theme was applied using a shared `ApplyTheme()` method called on each form load, ensuring consistency across all forms without duplicating code.

The clipboard auto clear gives the user a 15 second window after copying a password before it is automatically removed from the clipboard. A notice is shown immediately after copying so the user knows the timer has started. This balances the usability need to copy passwords quickly against the security risk of clipboard monitoring malware, which reads whatever is currently copied and exfiltrates it (to exfiltrate is to steal sensitive data from a computer). The 15 second window was chosen as sufficient time for a user to switch to another application and paste, without leaving the password exposed indefinitely.

The timed password reveal allows a stored password to be temporarily shown on screen for 10 seconds before automatically hiding behind asterisks again. This avoids the frustration of having no way to view a password when needed while preventing it from being left visible on screen.

The lockout feedback messages tell the user exactly how long they need to wait after a failed login attempt. Rather than just showing a generic error, the application displays the remaining lockout time left. This reduces confusion for legitimate users who mistype their master password and makes the security mechanism transparent without undermining it.

The password policy gives specific and helpful error messages rather than generic ones. When a password fails the complexity check, the message states what rule was violated, for example "Password must contain at least 1 symbol". This was chosen over a generic rejection message because users are more likely to correct their input correctly when told specifically what is wrong, rather than guessing.



3.5 Key Variables, Data Structures, Classes and Validation

The Password manager is split into four core classes, each with their own clearly defined responsibility. I chose to program this way to make the program easier to test, maintain and extend without changes in one area breaking another.

PasswordUtils is the main class handling all cryptographic operations. It contains the methods for salt generation, PBKDF2 hashing, AES 256 encryption and decryption, and the file read/write functions for both `master.dat` (where the master password is stored) and `passwords.dat` (where encrypted passwords are stored). It was implemented as a static class because its methods are stateless operations that take inputs and return outputs without needing to maintain any instance data. Essentially none of the methods are dynamic. The key variables within it include `Master.dat` and `Passwords.dat`, constants which define the file paths, and the iteration count of 100,000 which is used as a named constant rather than a magic number to make the security parameter explicit and easy to update.

```

namespace NEA_Project_test
{
    0 references
    public static class PasswordUtils
    {
        3 references
        private const string MasterFile = "master.dat";

        // =====
        // SALT GENERATION
        // =====
        0 references
        public static byte[] GenerateSalt(int size = 16)
        {
            var salt = new byte[size];
            using (var rng = RandomNumberGenerator.Create())
            {
                rng.GetBytes(salt);
            }
            return salt;
        }

        // =====
        // HASHING
        // =====
        1 reference
        public static byte[] HashPassword(string password, byte[] salt, int iterations = 100000, int size = 32)
        {
            using (var pbkdf2 = new Rfc2898DeriveBytes(password, salt, iterations, HashAlgorithmName.SHA256))
            {
                return pbkdf2.GetBytes(size);
            }
        }
    }
}

```



```

public static class PasswordUtils
// =====
// DERIVE ENCRYPTION KEY
// =====
0 references
public static byte[] DeriveEncryptionKey(string password, byte[] salt, int size = 32)
{
    using (var pbkdf2 = new Rfc2898DeriveBytes(password, salt, 100000, HashAlgorithmName.SHA256))
    {
        return pbkdf2.GetBytes(size);
    }
}

// =====
// ENCRYPT STRING
// =====
0 references
public static byte[] EncryptString(string plainText, byte[] key)
{
    if (plainText == null)
        throw new ArgumentNullException(nameof(plainText));
    if (key == null)
        throw new ArgumentNullException(nameof(key));
    using (var aes = Aes.Create())
    {
        aes.Key = key;
        aes.GenerateIV();

        using (var ms = new MemoryStream())
        {
            ms.Write(aes.IV, 0, aes.IV.Length);

            using (var encryptor = aes.CreateEncryptor())
            using (var cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write))
            using (var sw = new StreamWriter(cs, Encoding.UTF8))
            {
                sw.Write(plainText);
            }

            return ms.ToArray();
        }
    }
}

```

```

// =====
// DECRYPT BYTES
// =====
0 references
public static string DecryptBytes(byte[] cipherData, byte[] key)
{
    if (cipherData == null)
        throw new ArgumentNullException(nameof(cipherData));
    if (key == null)
        throw new ArgumentNullException(nameof(key));

    using (var aes = Aes.Create())
    {
        aes.Key = key;

        int ivSize = aes.BlockSize / 8;
        if (cipherData.Length < ivSize)
            throw new ArgumentException("Cipher data too short or invalid.");

        byte[] iv = new byte[ivSize];
        Array.Copy(cipherData, iv, iv.Length);
        aes.IV = iv;

        using (var ms = new MemoryStream(cipherData, iv.Length, cipherData.Length - iv.Length))
        using (var decryptor = aes.CreateDecryptor())
        using (var cs = new CryptoStream(ms, decryptor, CryptoStreamMode.Read))
        using (var sr = new StreamReader(cs, Encoding.UTF8))
        {
            return sr.ReadToEnd();
        }
    }
}

```

LockoutManager handles brute force protection. It tracks the number of failed login attempts and the timestamp of the last failed login. It then uses these to calculate the remaining lockout duration using the exponential backoff formula. The key variables are FailedCount, an integer tracking consecutive failures, and LastFailed, a Date and Time variable that stores when the most recent failure happened. These are tied to a small lockout file, so the lockout isn't reset by restarting the application.

```

namespace NEA_Project_test
{
    public class LockoutInfo
    {
        9 references
        public int FailedCount { get; set; }
        7 references
        public DateTime LastFailed { get; set; }
    }

    0 references
    public static class LockoutManager
    {
        3 references
        private const string LockoutFile = "lockout.dat";

        // Backoff parameters
        1 reference
        private const int BaseDelaySeconds = 5; // delay for first increment
        2 references
        private const int MaxDelaySeconds = 300; // maximum wait (5 minutes)

        2 references
        public static LockoutInfo Load()
        {
            if (!File.Exists(LockoutFile)) return new LockoutInfo { FailedCount = 0, LastFailed = DateTime.MinValue };

            try
            {
                using (var fs = new FileStream(LockoutFile, FileMode.Open, FileAccess.Read))
                using (var br = new BinaryReader(fs))
                {
                    int failedCount = br.ReadInt32();
                    long ticks = br.ReadInt64();
                    return new LockoutInfo { FailedCount = failedCount, LastFailed = new DateTime(ticks) };
                }
            }
            catch
            {
                // If file corrupted, reset
                return new LockoutInfo { FailedCount = 0, LastFailed = DateTime.MinValue };
            }
        }
    }
}

```

```

namespace NEA_Project_test
{
    public static class LockoutManager
    {
        public static void Save(LockoutInfo info)
        {
            using (var fs = new FileStream(LockoutFile, FileMode.Create, FileAccess.Write))
            using (var bw = new BinaryWriter(fs))
            {
                bw.Write(info.FailedCount);
                bw.Write(info.LastFailed.Ticks);
            }
        }

        0 references
        public static void RegisterFailedAttempt()
        {
            var info = Load();
            info.FailedCount++;
            info.LastFailed = DateTime.UtcNow;
            Save(info);
        }

        0 references
        public static void Reset()
        {
            var info = new LockoutInfo { FailedCount = 0, LastFailed = DateTime.MinValue };
            Save(info);
        }

        // Returns remaining seconds to wait (0 if allowed now)
        0 references
        public static int GetRemainingLockoutSeconds()
        {
            var info = Load();
            if (info.FailedCount <= 0) return 0;

            // exponential backoff: delay = BaseDelay * 2^(FailedCount-1)
            double delay = BaseDelaySeconds * Math.Pow(2, info.FailedCount - 1);
            if (delay > MaxDelaySeconds) delay = MaxDelaySeconds;

            DateTime windowStart = info.LastFailed;
            double elapsed = (DateTime.UtcNow - windowStart).TotalSeconds;
            int remaining = (int)Math.Ceiling(delay - elapsed);
            return remaining > 0 ? remaining : 0;
        }
    }
}

```

SecurityPolicy contains the password complexity logic. It checks the master password against a set of rules including minimum length of 10 characters, whether uppercase and lowercase letters are used, at least one digit, at least one symbol, and rejection of common passwords (ideally it should take reference from a file containing the top say 100 most common passwords but for now it only checks against the top 3 most commonly used). It is kept separate from **PasswordUtils** deliberately so that validation rules can be updated independent of any cryptography.

```
namespace NEA_Project_test
{
    0 references
    public static class SecurityPolicy
    {
        // Parameters you can tweak
        2 references
        public const int MinLength = 10;
        2 references
        public const int MinUpper = 1;
        2 references
        public const int MinLower = 1;
        2 references
        public const int MinDigits = 1;
        2 references
        public const int MinSymbols = 1;
        0 references
    }
}
```

```

public static bool ValidatePasswordComplexity(string password, out string reason)
{
    reason = null;
    if (string.IsNullOrEmpty(password))
    {
        reason = "Password cannot be empty.";
        return false;
    }

    if (password.Length < MinLength)
    {
        reason = $"Password must be at least {MinLength} characters.";
        return false;
    }

    int uppers = 0, lowers = 0, digits = 0, symbols = 0;
    foreach (char c in password)
    {
        if (char.IsUpper(c)) uppers++;
        else if (char.IsLower(c)) lowers++;
        else if (char.IsDigit(c)) digits++;
        else symbols++;
    }

    if (uppers < MinUpper) { reason = $"Password must contain at least {MinUpper} uppercase letter."; return false; }
    if (lowers < MinLower) { reason = $"Password must contain at least {MinLower} lowercase letter."; return false; }
    if (digits < MinDigits) { reason = $"Password must contain at least {MinDigits} digit."; return false; }
    if (symbols < MinSymbols) { reason = $"Password must contain at least {MinSymbols} symbol (e.g. !@#$)."; return false; }

    // Optionally reject very common patterns (simple example)
    string lower = password.ToLowerInvariant();
    if (lower.Contains("password") || lower.Contains("12345") || lower.Contains("qwerty"))
    {
        reason = "Password is too common. Choose something less guessable.";
        return false;
    }

    return true;
}

```

The forms themselves, **LoginForm**, **MainForm**, **ViewPasswordForm**, **ChangePasswordForm** and **ConfirmPasswordForm**, act purely as the presentation layer, what the user sees. They hold no sensitive data beyond what is immediately needed for the current operation. The session key, which is the AES key derived from the master password after login, is stored as a byte array in memory only for the duration of the session and is cleared by the **SecureCleanup()** function when the application closes or the user logs out.

The program stores password entries in a binary file called passwords.dat. Rather than using a readable format like JSON or CSV, each entry is saved as raw bytes. Before each piece of data, the program writes a number indicating how long that data is, so when reading it back it knows exactly how many bytes to expect for each field. This approach was chosen for three reasons. It handles encrypted data naturally since ciphertext is already in binary form. It avoids issues where special characters in a password could accidentally break the file structure, which is a common problem with comma or delimiter based formats. And because the file is not human readable, anyone who opens it directly cannot see the structure or field names, adding a small but useful layer of obscurity on top of the encryption.

3.6 Test Data and Plan

Iterative Test Data

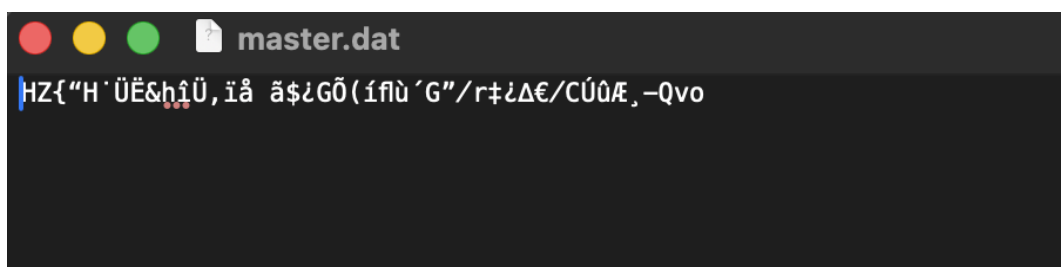
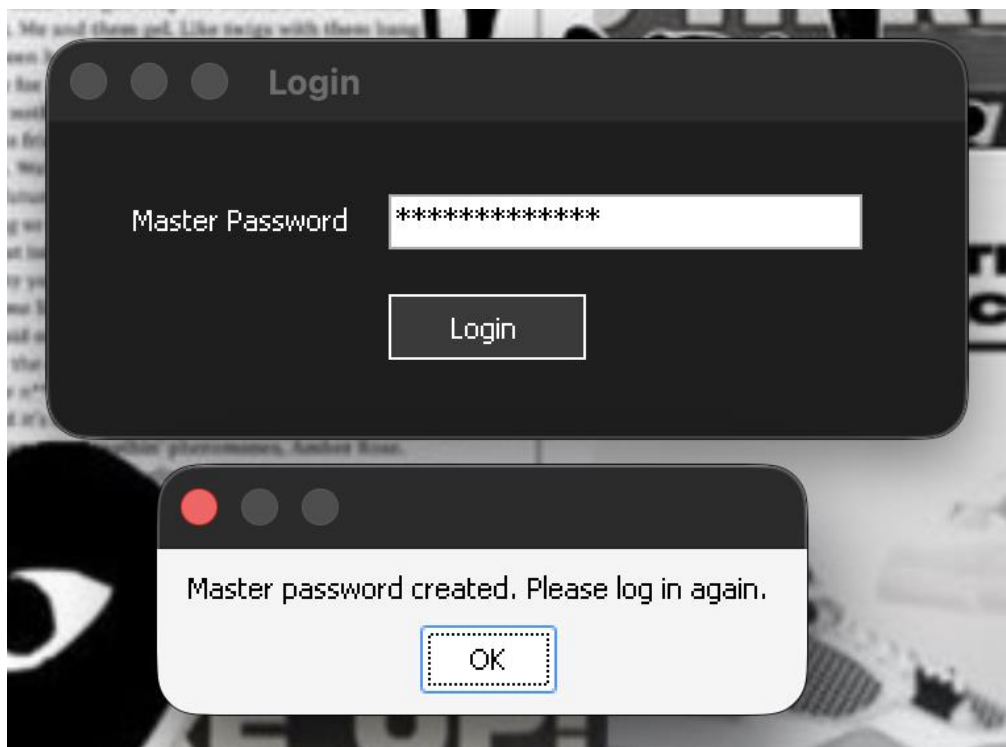
Test	Input	Expected Outcome
Master password creation	Enter valid password meeting all complexity rules on first run	Password accepted, hash stored in master.dat, login screen shown
Master password rejection	Enter password under 10 characters with no symbols	Rejection message stating specific rule violated
Login with correct password	Enter correct master password on login screen	Access granted, main vault screen shown
Login with incorrect password	Enter wrong master password	Access denied, attempt counted, lockout timer begins
Lockout after failed attempts	Enter wrong password repeatedly	Delays increase exponentially, 5s then 10s then 20s up to 300s
Add password entry	Enter site, username and password then click Add Entry	Entry encrypted and saved to passwords.dat, appears in vault list
View password entry	Select entry and click View Entry	Entry decrypted and displayed in ViewPasswordForm
Delete password entry	Select entry and click Delete Entry	Entry removed from vault list and passwords.dat
Copy password to clipboard	Click Copy Password in ViewPasswordForm	Password copied, security notice shown, clipboard cleared after 10 seconds
Timed password reveal	Click Show in ViewPasswordForm	Password visible for 10 seconds then hidden behind asterisks

The iterative tests were chosen to cover the core functionality at each stage of development rather than waiting until the end to find problems. Testing master password creation and rejection early meant the authentication system was confirmed to be working before any vault features were built on top of it. The lockout tests were included specifically because the exponential backoff logic involves a mathematical formula and time-based calculations that are easy to get wrong, so manual verification after each attempt was necessary to confirm the delays were increasing correctly. The add, view and delete entry tests were kept simple at this stage because the goal was confirming the basic vault operations worked end to end, not stress testing edge cases. Those were saved for the post development phase.

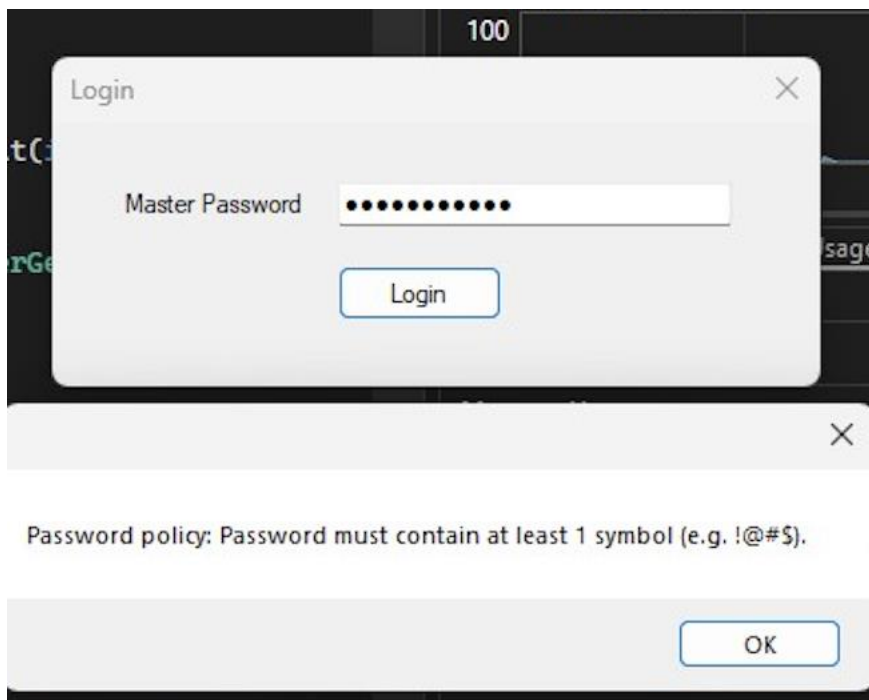
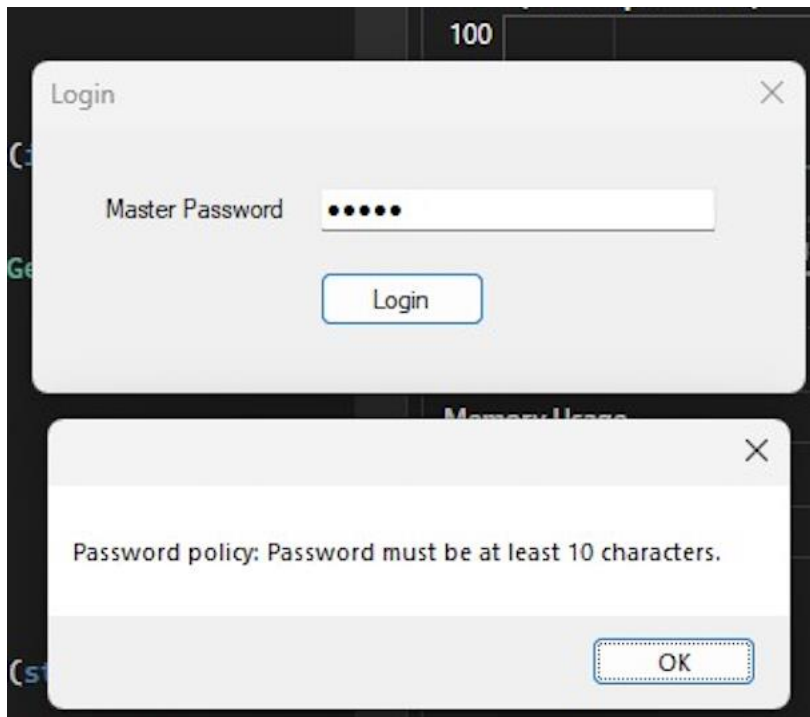
The post development tests were chosen to target the areas most likely to fail in real world use and the areas where failure would have the most serious consequences. Empty field validation and common password rejection are both things a real user could trigger accidentally or deliberately, so confirming the application handles them well was important for both usability and security. The AES encryption verification test was included because correct encryption is the most critical requirement in the whole application, and without directly inspecting the stored file there is no way to confirm the data is actually being encrypted rather than just appearing to be. The lockout persistence test after restart was specifically chosen because the lockout would be completely ineffective against an attacker who simply closes and reopens the application if it were not implemented correctly, so this was a necessary check to confirm the protection actually works as intended.

Screenshots - Testing the Program

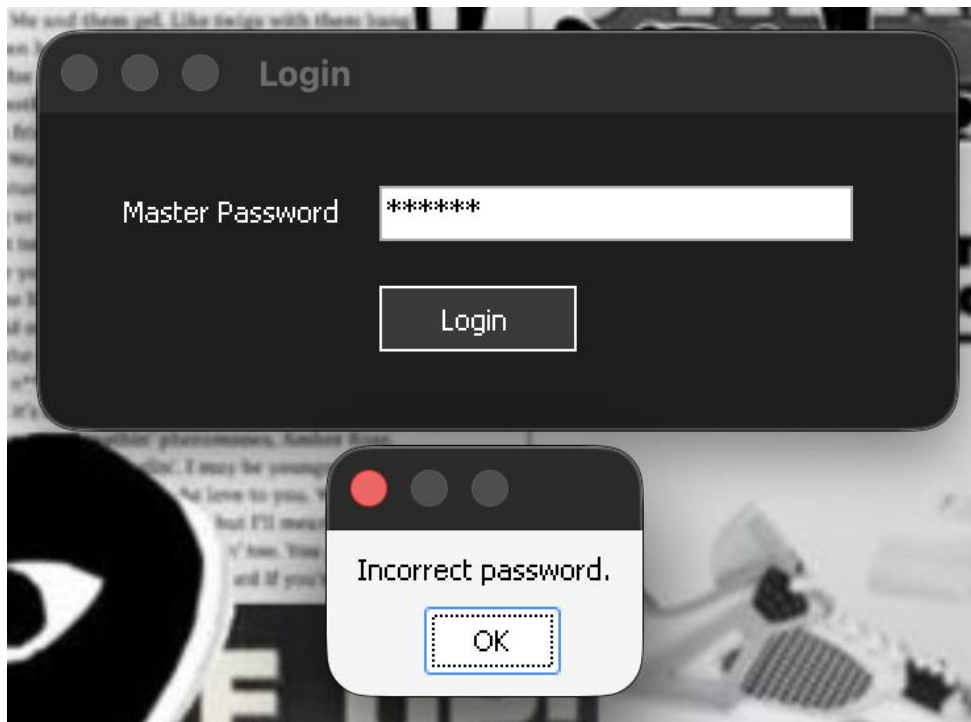
Master Password Creation and validation



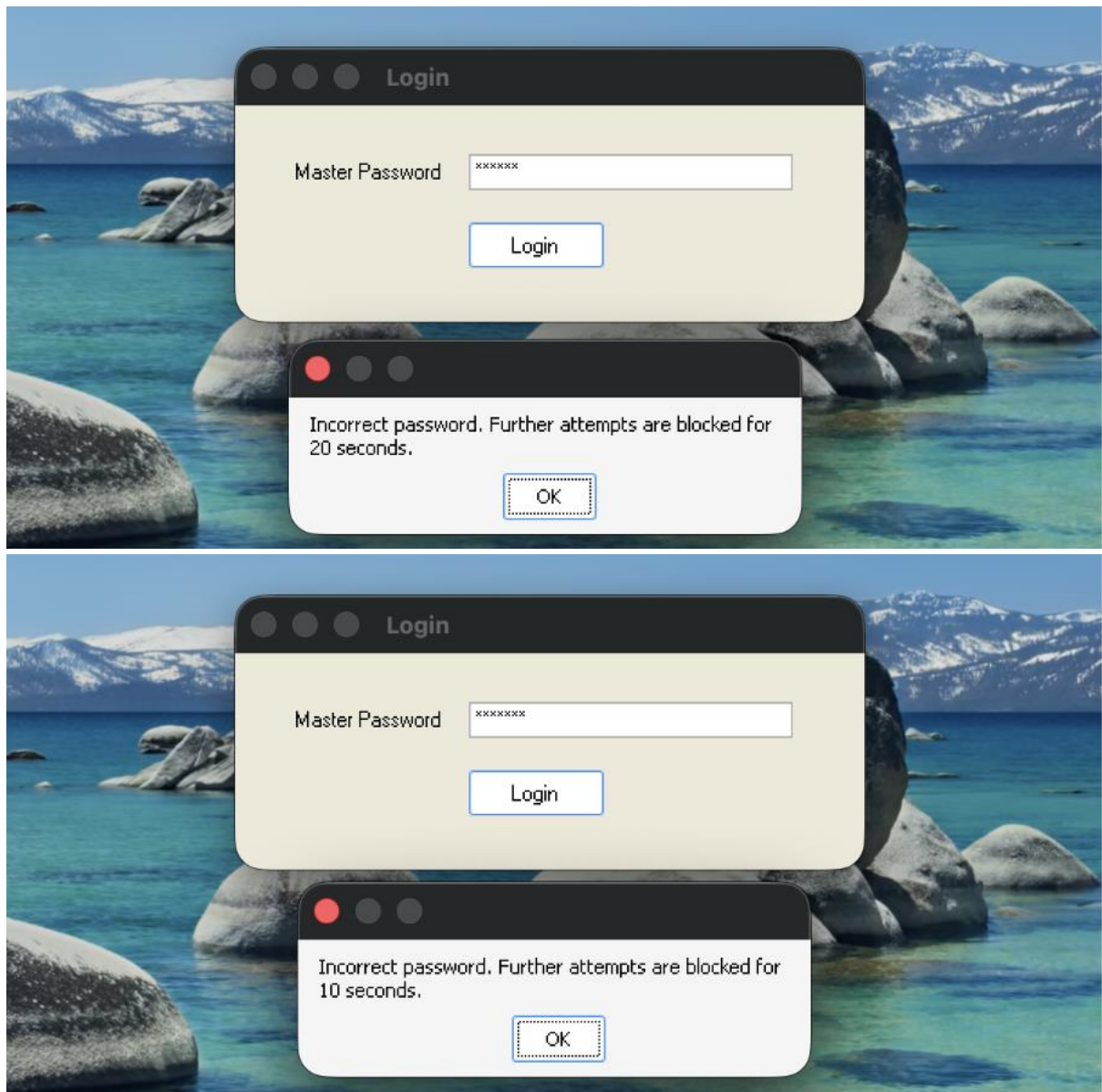
Master Password Rejection



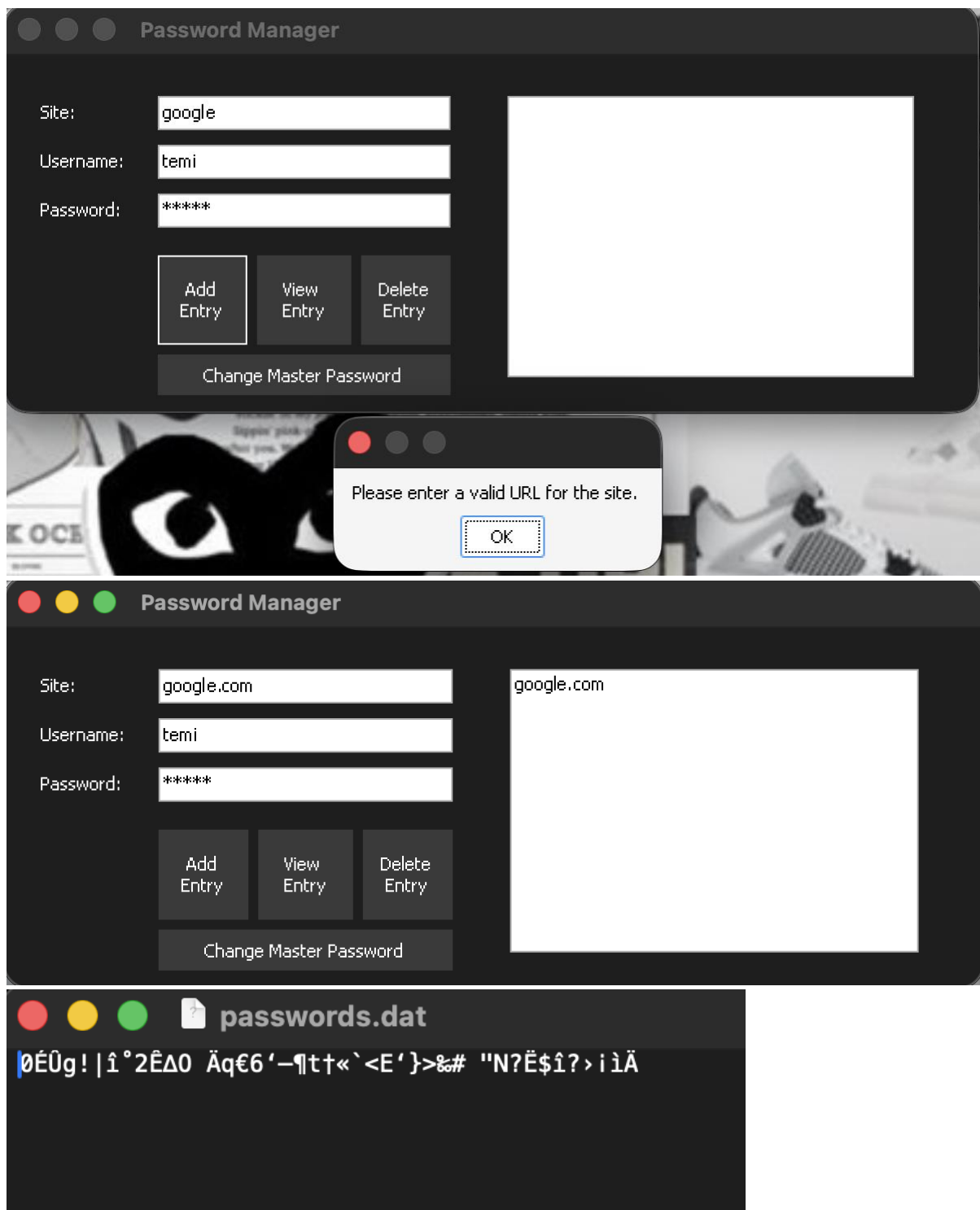
Master Password Incorrect Login



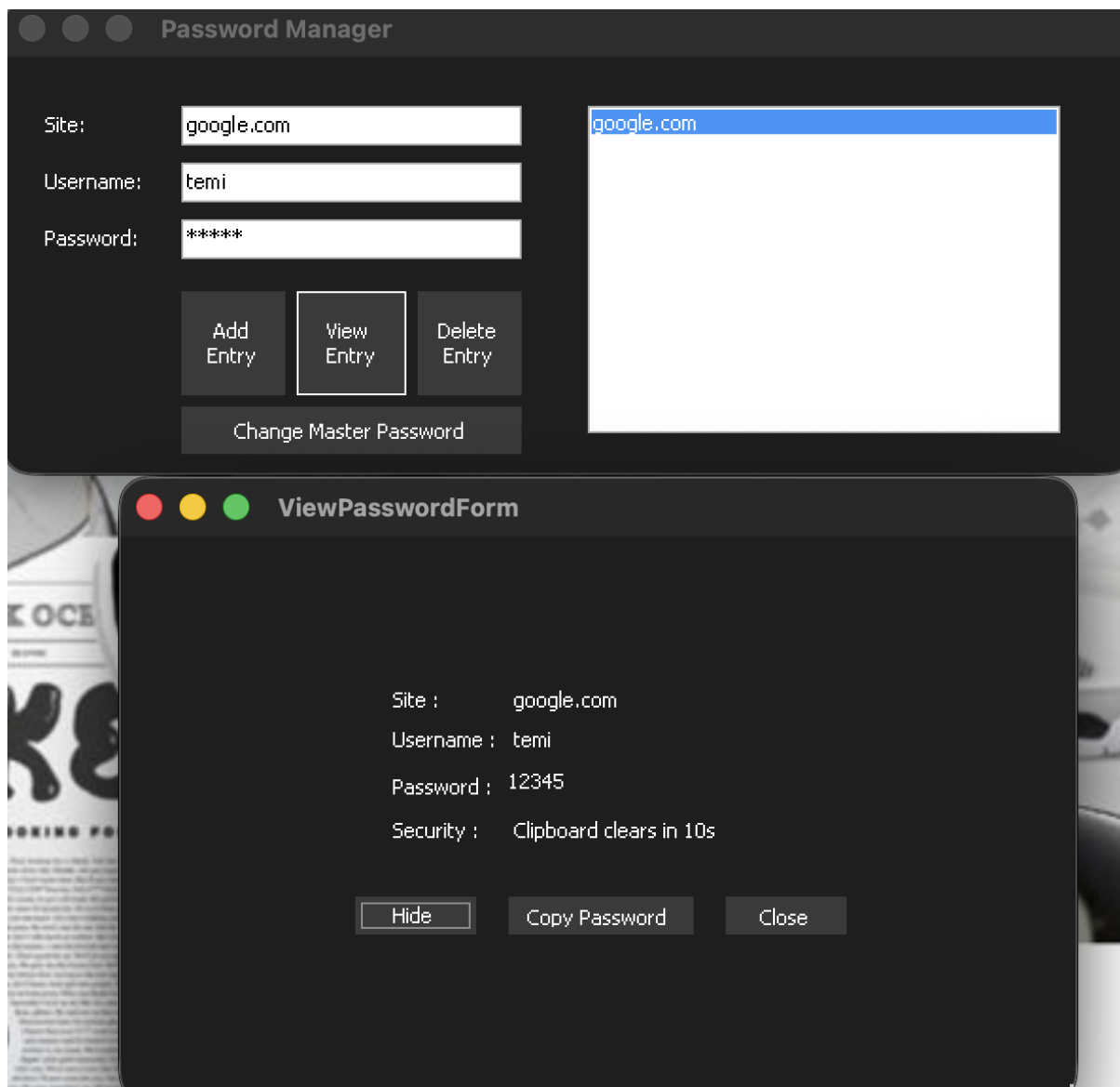
Exponential Lock out Check



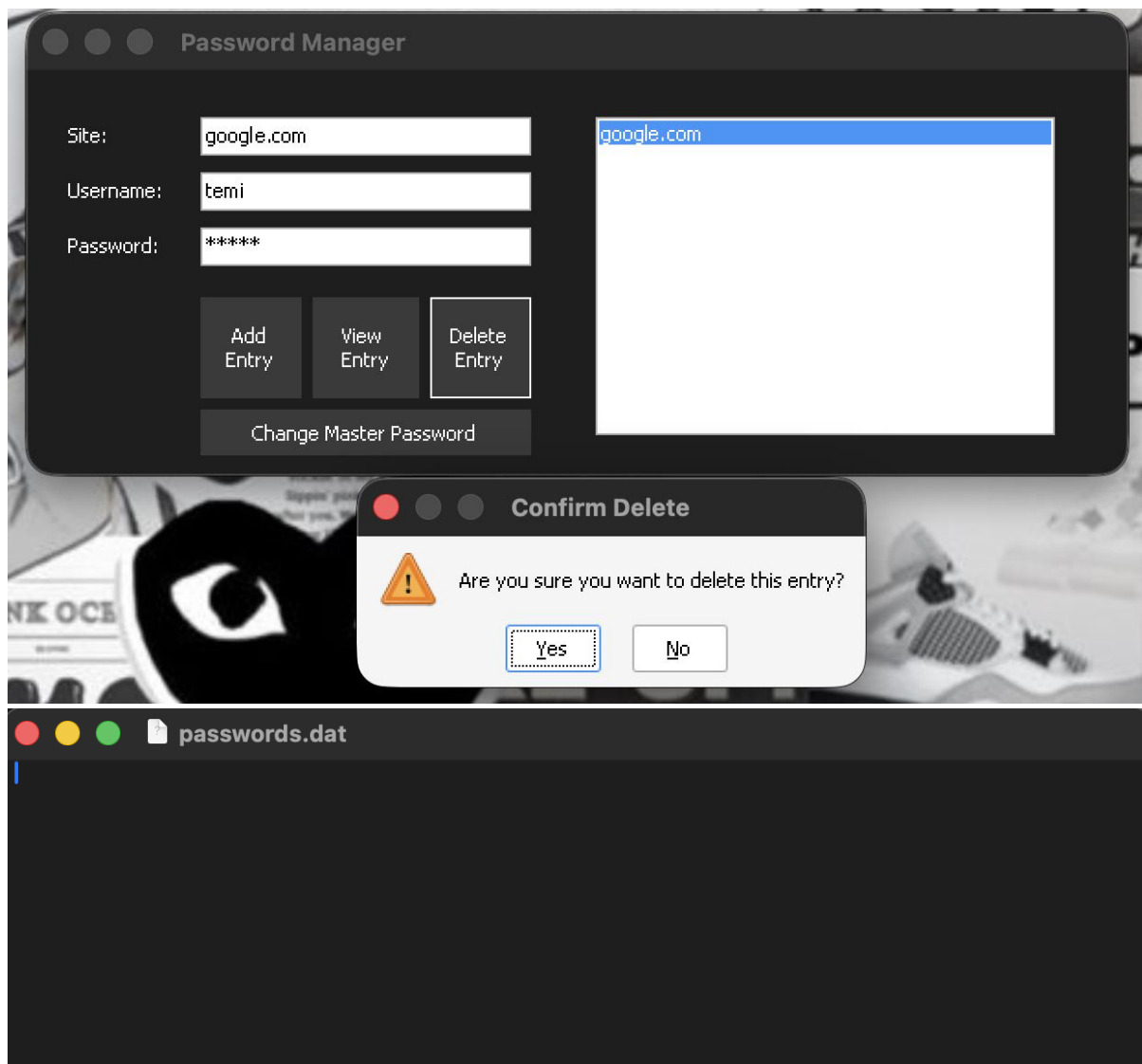
Adding Passwords



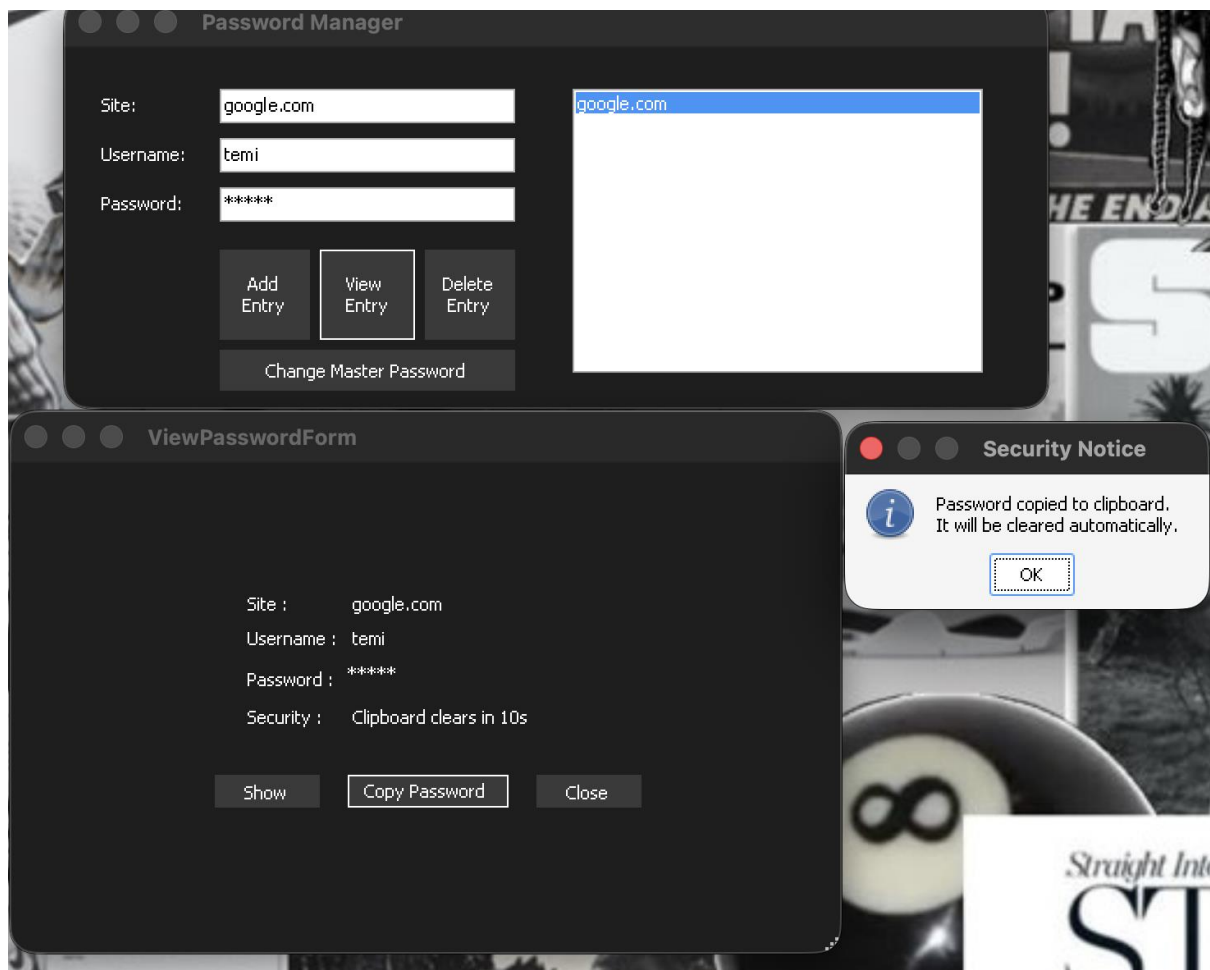
Viewing Password



Deleting Passwords

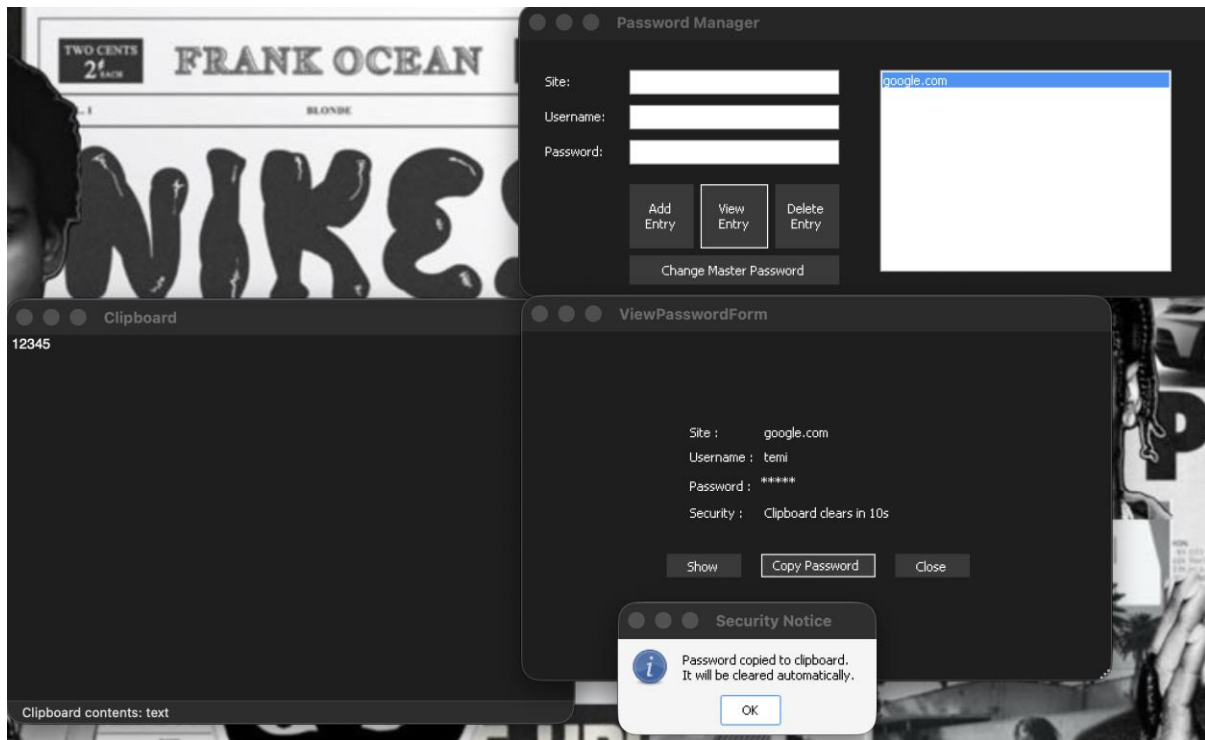


Copying Passwords

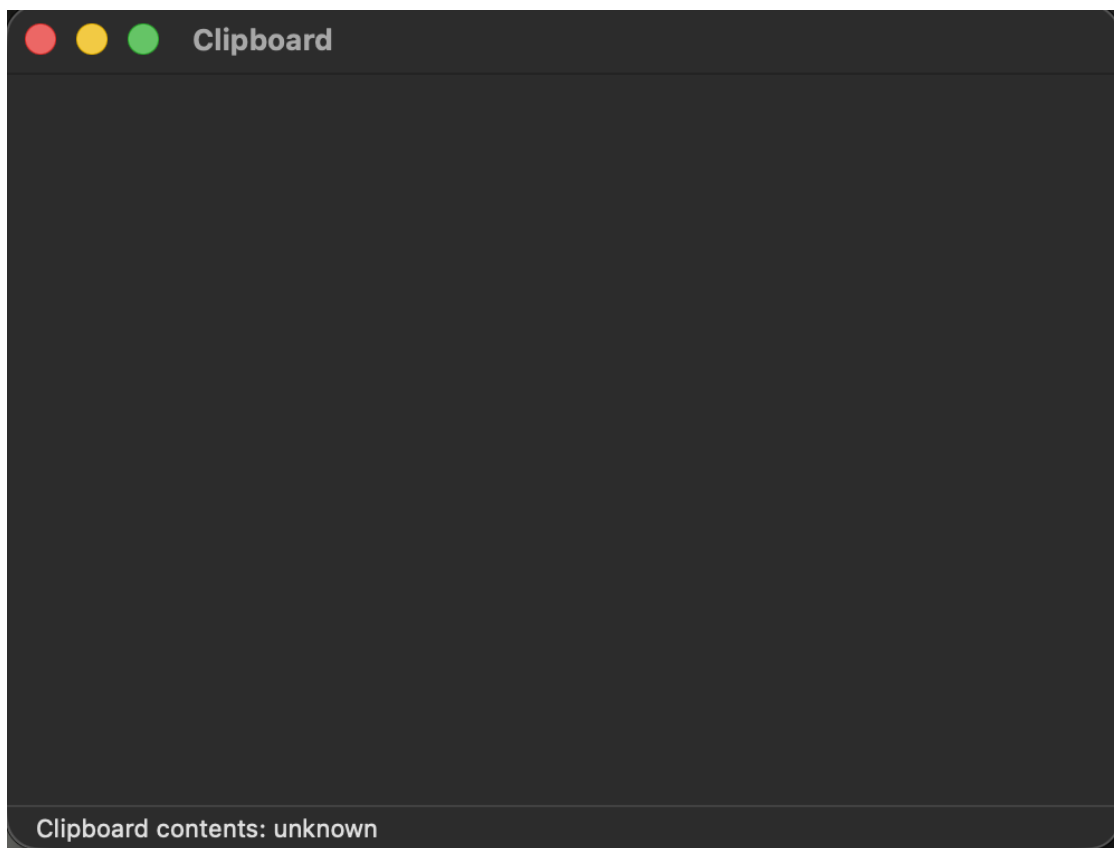


Clipboard Check and timed password reveal

Before



After

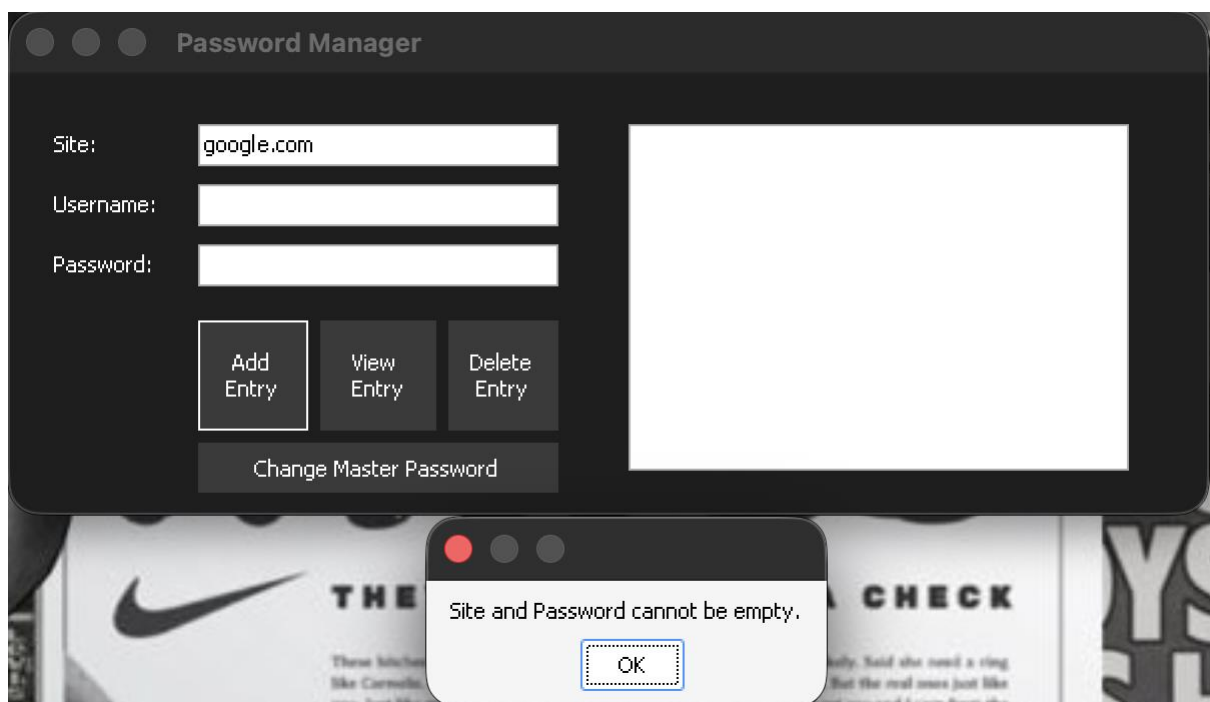


Post Development Test Data

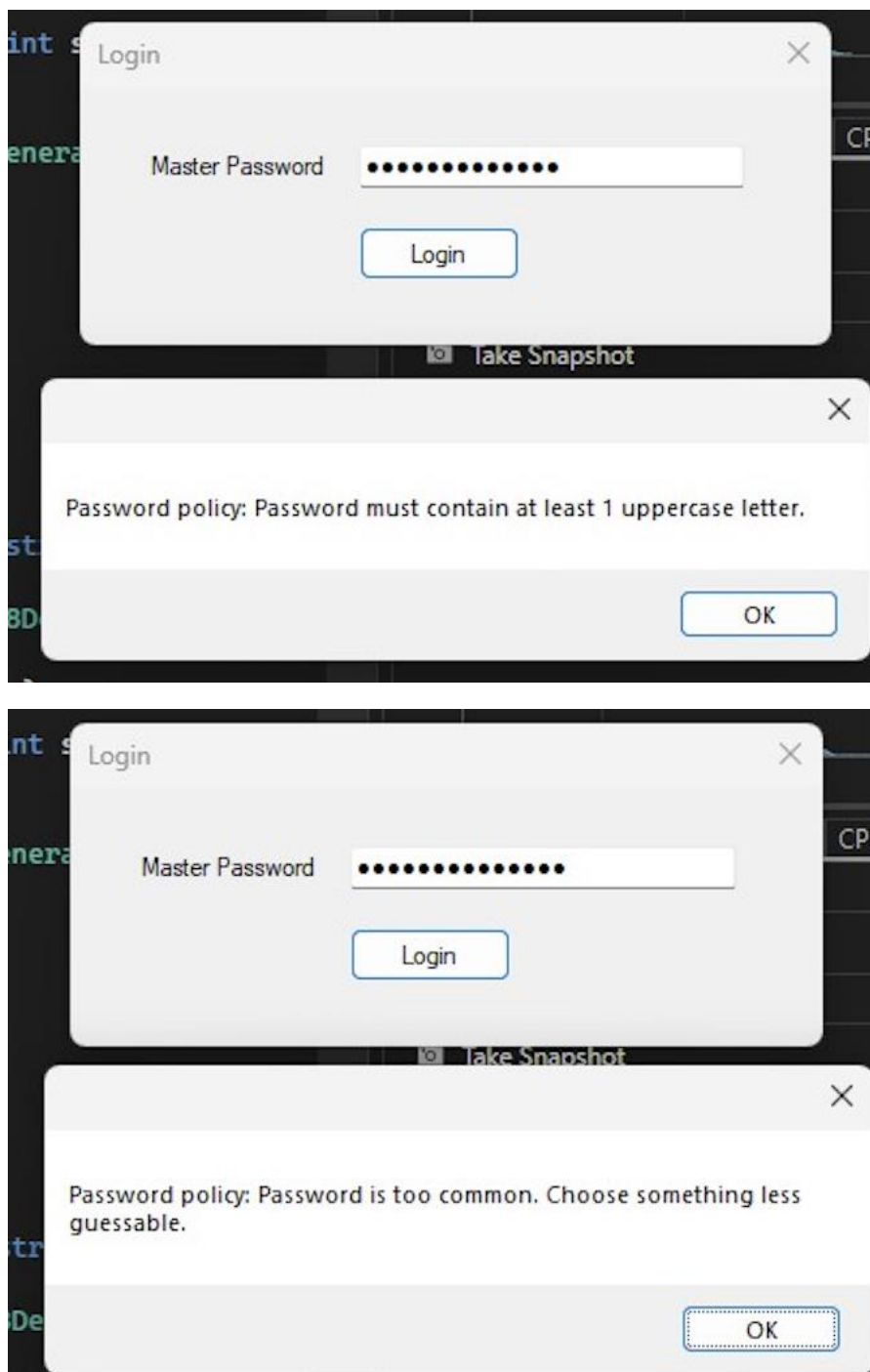
Test	Input	Expected Outcome
Empty field validation	Submit add entry form with one or more fields blank	Error message shown, no entry added
Common password rejection	Enter a commonly used password as master	Rejection with message stating password is too common
AES encryption verification	Add entry and inspect passwords.dat in a hex editor	Only ciphertext visible, no plaintext
Master password change	Change master password using ChangePasswordForm	New password accepted, old password rejected on next login, existing entries still accessible
Session persistence	Add entries, close and reopen application, log in	All previously added entries still accessible and correctly decrypted
Lockout persists after restart	Fail login attempts, close and reopen application	Lockout timer continues from where it left off

Testing Screenshots

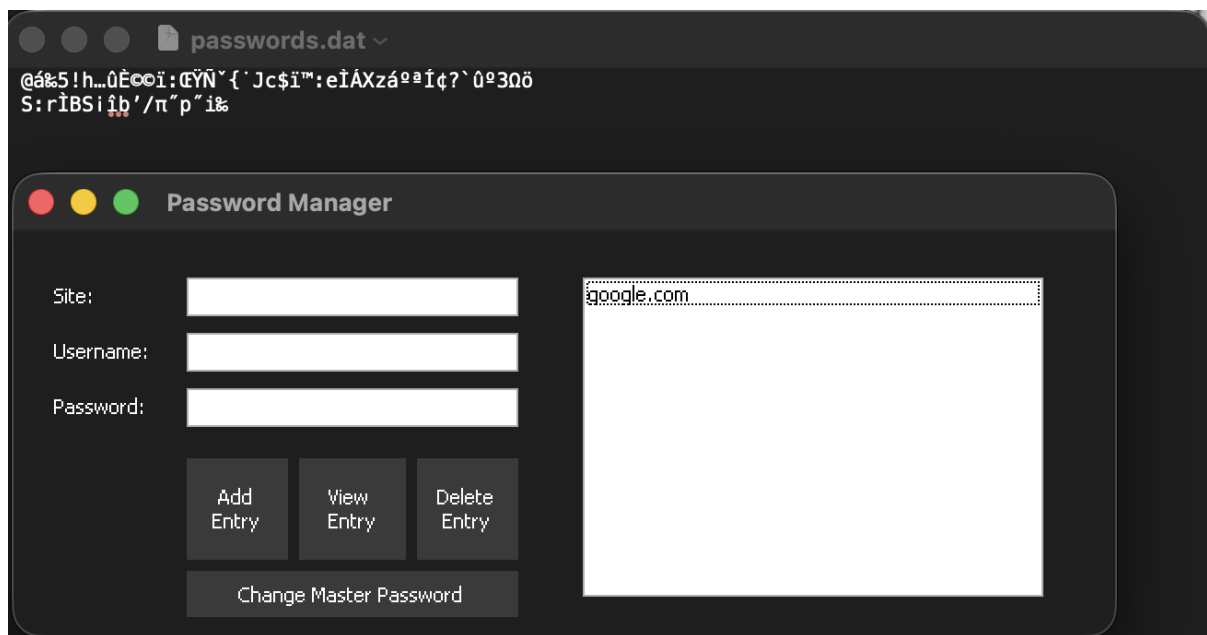
Empty Fields



Common Password Rejection

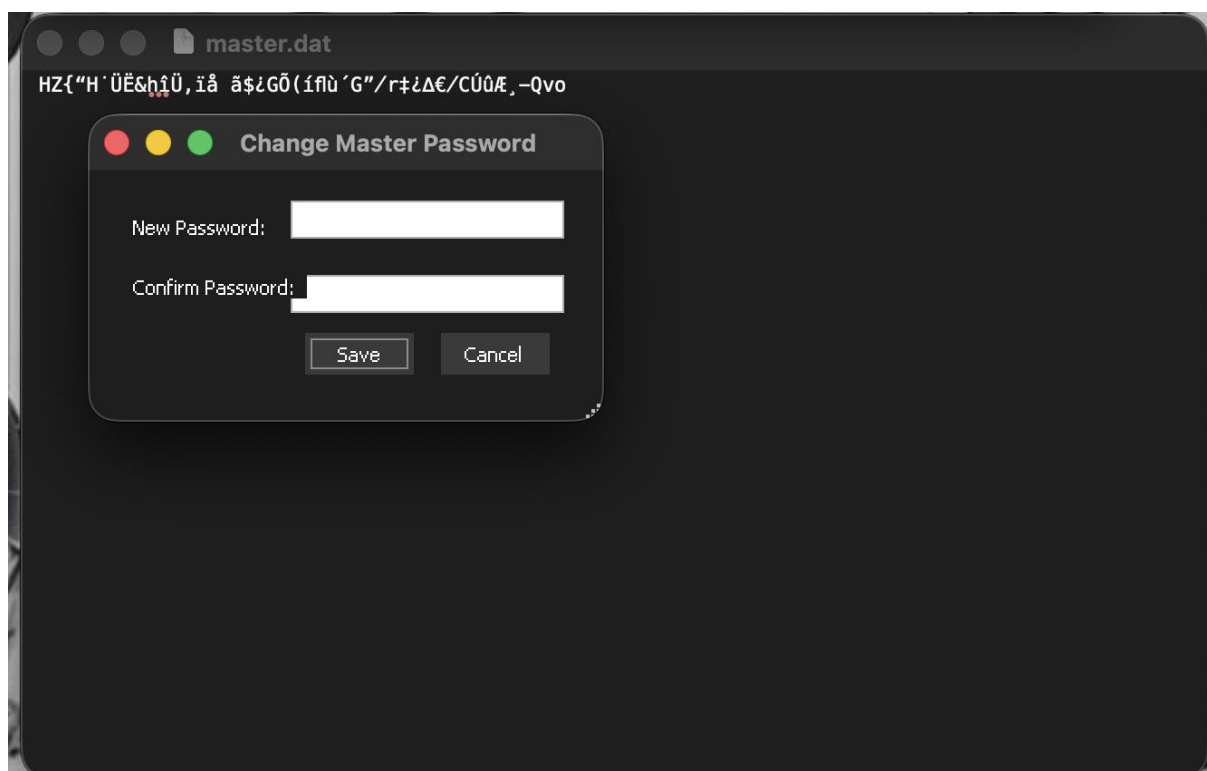


Encryption Verification

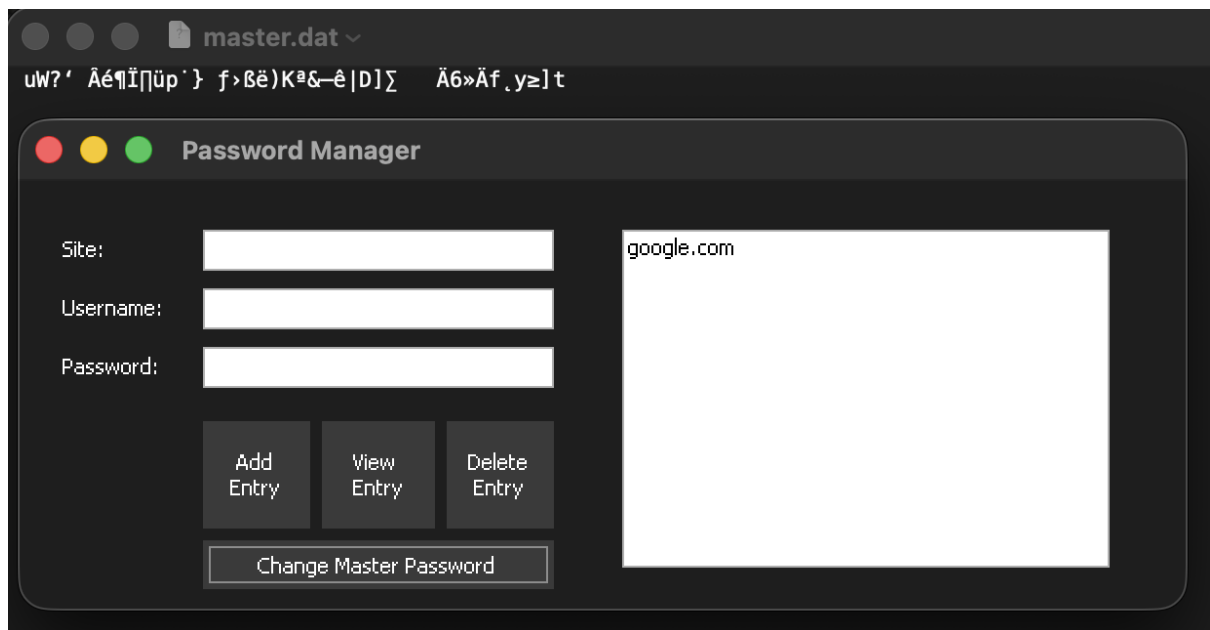


Master Password Change

Before



After



Lockout Persistence

Before



After



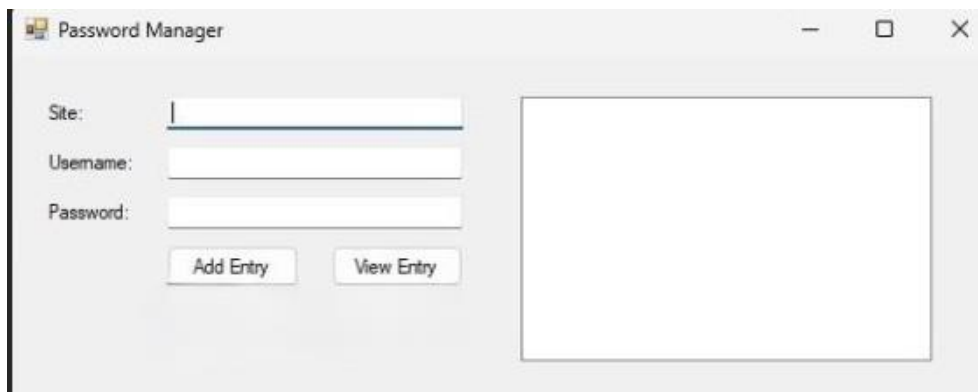
4. Development

4.1 Iterative Development Process

Screenshot evidence of most runtime and logic errors were not retained

The project was built in five clearly defined phases, each adding security and functionality on top of a working foundation.

The first phase established the Windows Forms application structure. The focus at this stage was purely on functional behaviour. Login forms, vault display and storing password entries. No cryptography was implemented yet. Passwords were stored in plaintext, which meant the system worked but was entirely insecure. This was a deliberate starting point to get the application structure and UI flow working correctly before introducing any complexity.

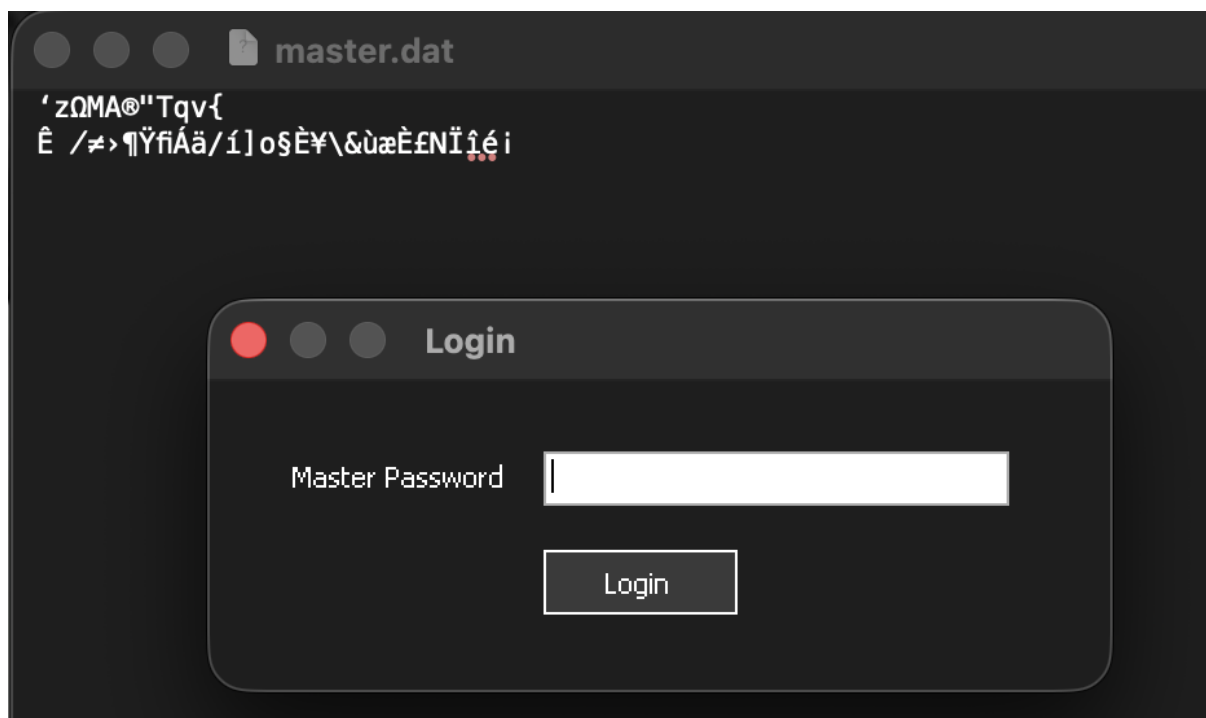


Screenshots of plaintext storage were not retained

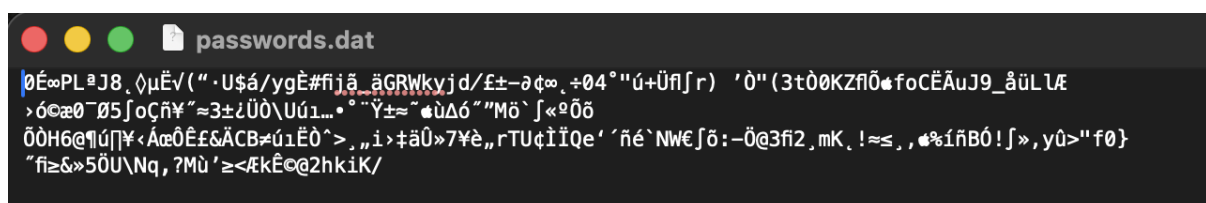
The second phase addressed compilation and framework errors that arose when the project was first being set up. Target type object creation syntax caused a CS error because the project was initially targeting an older C# version. The `Application.SetHighDPIMode` call was removed after it was found to be unavailable in the framework version I was using. `CryptographicOperations.FixedTimeEquals` was not available either, so I replaced with a manual byte-by-byte hash comparison.

An `EndOfStreamException` was thrown when reading `master.dat` because the write and read formats were not aligned. The fix standardised the binary file structure to write salt length, salt, hash length and hash in sequence, with the read operation expecting exactly that layout.

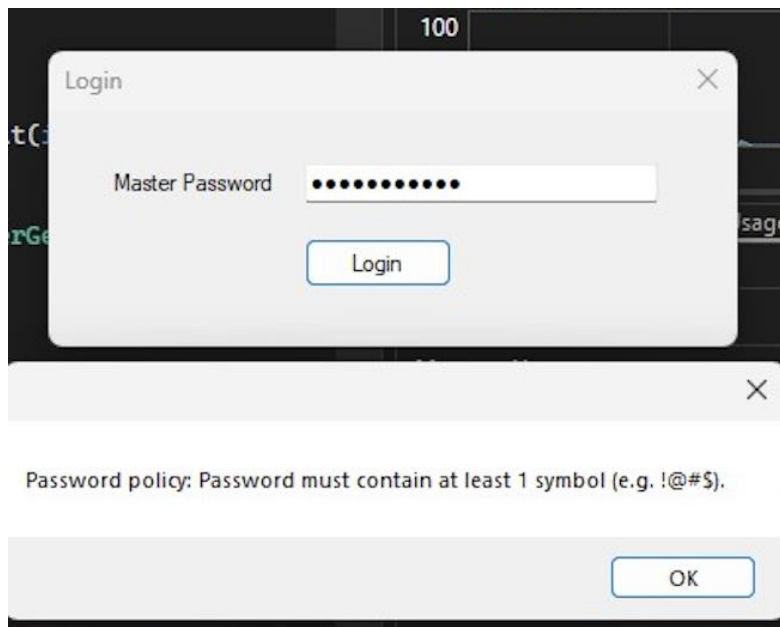
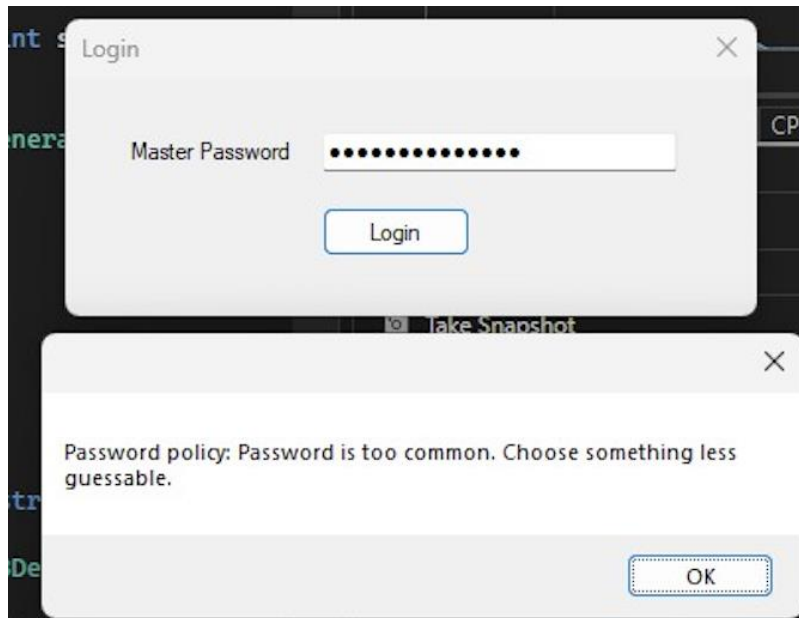
The third phase introduced the master password authentication system. Rather than storing the password directly, PBKDF2 hashing was implemented using **Rfc2898DeriveBytes** with a randomly generated salt. This prevents rainbow table attacks because every stored hash is unique even if two users choose the same password, and slows brute force attacks significantly because of the iteration cost.



The fourth phase added AES 256 encryption for vault entries. The encryption key is derived from the master password using PBKDF2, and each entry generates a random initialisation vector so that identical passwords produce different ciphertext every time. The file format was updated to store IV length, IV, ciphertext length and ciphertext per entry.



The fifth phase added the remaining security features. Password complexity rules were enforced through SecurityPolicy. The exponential backoff lockout was implemented in LockoutManager. A session-based encryption key was introduced, stored only in memory for the duration of the session. The delete entry and view entry features were added with authentication checks. Clipboard auto-clear was implemented to remove copied passwords after 15 seconds.



4.2 Evidence of Prototyping and Milestones

The development followed a clear prototype then extend pattern. Each phase produced a working, testable version before the next layer of complexity was added.

The first milestone was a functional but insecure application. Users could create a master password, add entries and view them, but nothing was encrypted and nothing was hashed. This validated the UI structure and application flow before any cryptographic complexity was introduced.

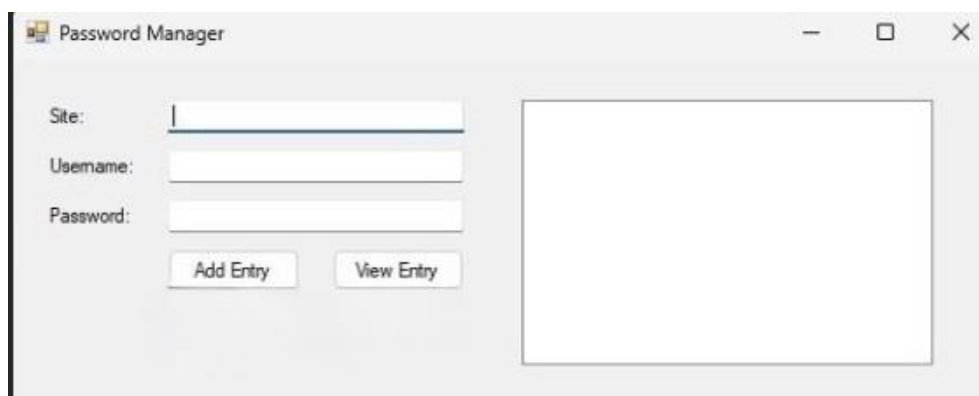
The second milestone was the introduction of PBKDF2 hashing for the master password. From this point forward, the stored master.dat file contained only a salt and a derived hash rather than any plaintext. This was the first meaningful security improvement and the foundation everything else was built on.

The third milestone was the addition of AES 256 encryption for vault entries. After this point, passwords.dat contained only ciphertext. Even if an attacker obtained the file directly, they would have no access to the stored credentials without the master password.

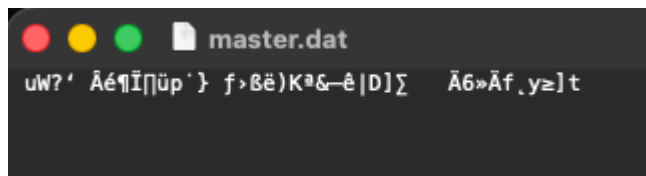
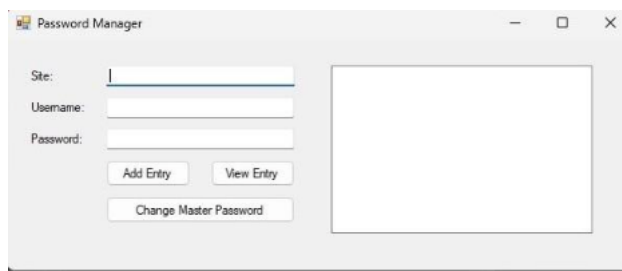
The fourth milestone was the complete security feature set including password policy, lockout, clipboard clearing and timed reveal. At this point the application met all the security requirements identified in the analysis section.

Main Version History

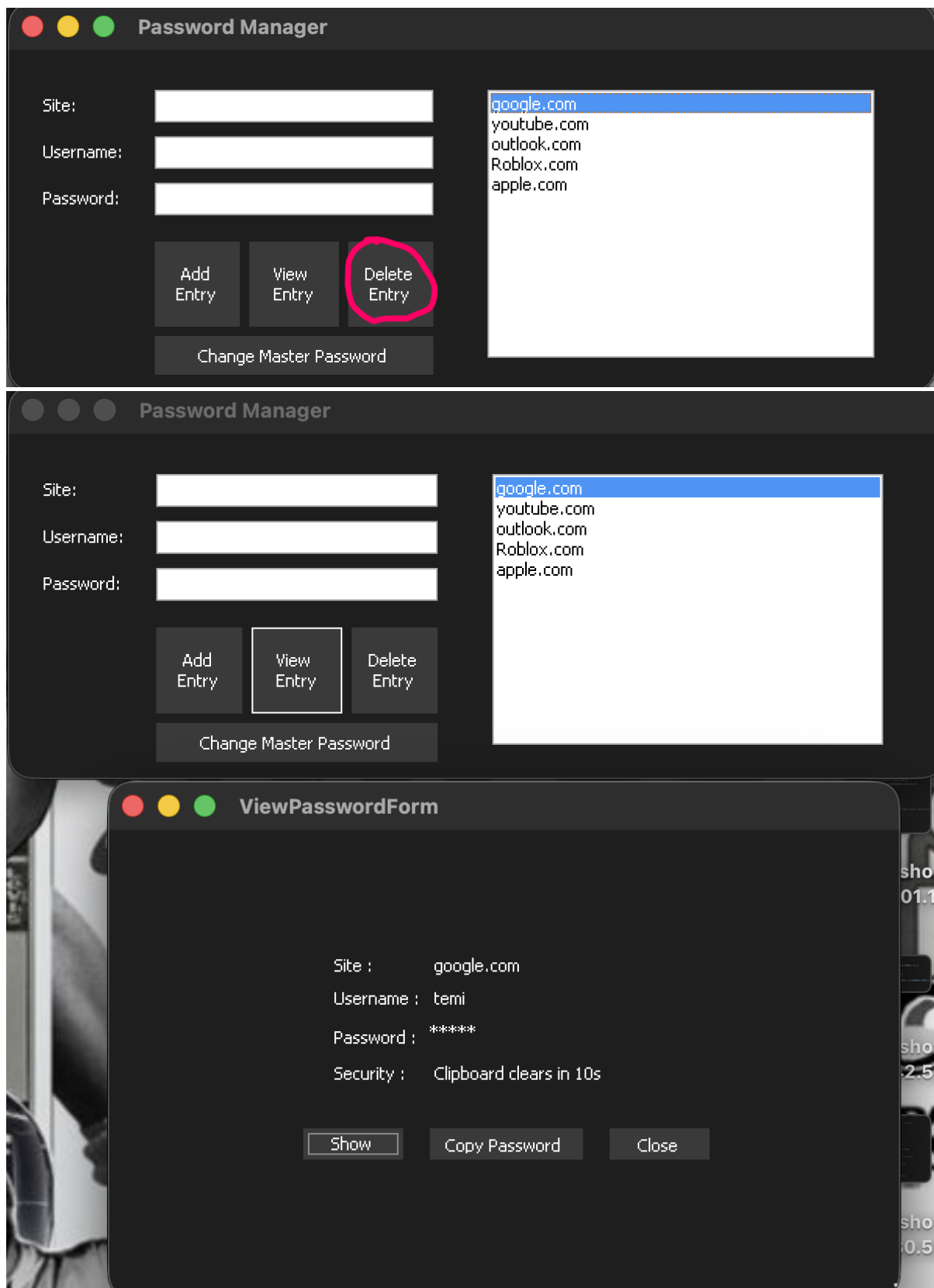
Version 1 (Beta)



Version 2 (Added master password change option + Encryption and Hashing)



Version 3 (Delete Entry Option + Expanded View Entry Option + Dark Theme)



4.3 Code Modules

The solution is structured around three core classes, each with a single clearly defined responsibility. This modular design means changes to one part of the system do not require changes to others, and each module can be tested independently without needing the full application to be running.

- (1) **PasswordUtils** handles all cryptographic operations. It is the only class in the project that has access to master.dat or passwords.dat directly. Any part of the application that needs to hash a password, encrypt an entry, decrypt an entry or read and write the vault files goes through PasswordUtils. Nothing else in the project handles file I/O for sensitive data. This means if the storage format or encryption algorithm ever needed to change, the only file that would need to be updated is PasswordUtils.cs.
- (2) **LockoutManager** handles brute force protection exclusively. It tracks failed login attempts and calculates the remaining lockout duration using the exponential backoff formula. It reads from and writes to lockout.dat independently of the rest of the application. LoginForm calls into it before processing any password input, but LockoutManager itself has no knowledge of the UI or any other class.
- (3) **SecurityPolicy** contains the password complexity validation logic. It checks minimum length, character type requirements and common password patterns. It is kept entirely separate from PasswordUtils so that the security policy can be tightened or changed without touching any cryptographic code. LoginForm and ChangePasswordForm both call at SecurityPolicy, but both are independent of the policy.

The **Forms** act purely as the presentation layer. LoginForm, MainForm, ViewPasswordForm, ChangePasswordForm and ConfirmPasswordForm handle what the user sees and interacts with. These forms call on PasswordUtils, LockoutManager and SecurityPolicy to get things done but none of those classes ever reference the forms back. Data flows in one direction only, from the UI into the logic layer and back out as results. No sensitive data is stored in the forms beyond what is immediately needed for the current operation, and SecureCleanup is called when forms close to clear any remaining data from memory.

This separation follows the single responsibility principle, where each class does one thing. It also means the security critical code in PasswordUtils and LockoutManager can be reviewed and audited without needing to understand the UI code at all, which is standard practice in security sensitive applications.

4.4 Code Comments

The code is commented throughout to explain the reasoning behind key decisions rather than just describing what the code does. The objective here was to comment on sections where the logic behind the code is not immediately obvious, particularly around the cryptographic operations and security mechanisms.

For example, in `PasswordUtils.cs` the salt generation method has a comment explaining why `RandomNumberGenerator` is used instead of the standard `Random` class. This is not obvious from the code alone since both produce random numbers, but the distinction matters because `Random` is seeded by time and is predictable, which would make the salts guessable. The PBKDF2 hashing method has a comment explaining that the 100,000 iteration count is a deliberate performance cost rather than an accident, which is important context for anyone reading the code later.

The `FixedTimeEquals` method has the most detailed comments in the project because its behaviour is counterintuitive. A developer reading it for the first time would likely question why the loop does not return early on a mismatch, so the comment explains the timing attack risk that this design prevents.

In `LockoutManager.cs` the `Save` method has a comment explaining why the lockout state is persisted to disk rather than held in memory. Without this comment the design choice looks unnecessary since most state is held in memory, but the reason is that an in-memory lockout would be trivially bypassed by closing and reopening the application.

In `ViewPasswordForm.cs` the clipboard timer tick handler has a comment explaining why it checks whether the clipboard still contains the copied value before clearing it. Without this context the check looks redundant, but it prevents the application from clearing something the user has copied from a different application after copying the password.

In `LoginForm.cs` the single handler that manages both first run setup and returning user login has comments marking each logical branch clearly. The first run path includes a comment explaining why the password is validated before being stored, since a weak master password would undermine all the cryptographic protections regardless of how strong the encryption is.

The annotated versions of all key files are included in the appendix for reference.

4.5 Variable and Structure Names

All variables, methods and classes throughout the project use names that clearly describe what they hold or do. This was a consistent practice across the whole program code rather than something applied selectively.

Class level: PasswordUtils, LockoutManager and SecurityPolicy all describe their purpose directly. A developer reading the project for the first time would understand what each class is responsible for before opening the file.

Method level: I assigned names like GenerateSalt, HashPassword, EncryptString, DecryptBytes, DeriveEncryptionKey, VerifyPassword, SaveMasterHash and LoadMasterHash to describe the exact operation being performed. There are no abbreviated or ambiguous method names anywhere in the cryptographic layer.

Variable level: I assigned names like masterKey, sessionKey, passwordEntries, clipboardValue and passwordVisible which communicate the purpose of each variable without the use of comments to explain them. The LockoutInfo class uses FailedCount and LastFailed which are immediately clear to anyone reading the lockout logic. Constants like BaseDelaySeconds and MaxDelaySeconds are named to make the lockout formula readable, so the line “BaseDelaySeconds * Math.Pow(2, info.FailedCount - 1)” communicates the intent without needing an explanation.

Where temporary variables are used inside methods, names like salt, hash, encryptor and plainText were chosen over single letter names to keep the code readable even in short scopes. The one area where single letter variable names appear is in loop counters and character type counts inside SecurityPolicy; where uppers, lowers, digits and symbols are short but still descriptive enough to be clear in context.

4.6 Validation

Validation is applied at every point where user input enters the system, ensuring that invalid or malicious data is caught before it reaches the cryptographic or storage logic.

The most critical validation point is the master password. On first run, before the password is hashed or stored, SecurityPolicy.ValidatePasswordComplexity is called to enforce the complexity rules. This check happens before any cryptographic operation so a weak master password can never be saved to disk in the first place. The validation returns a specific reason for each rule violation so the user knows exactly what needs to change.

The add entry form validates both the site field and the password field before processing. The site field is checked against a **regex** pattern to confirm it matches a valid URL format, rejecting blank input or text that does not resemble a real web address. The password field is checked with `string.IsNullOrEmpty` to prevent blank entries being encrypted and stored, which would produce entries that cannot be meaningfully displayed or used later. If either check fails the operation stops immediately and the user sees a clear error message.

The `ConfirmPasswordForm` validates that the password field is not blank before attempting verification, and shows a specific message if no master password has been set yet rather than throwing an unhandled exception.

The `LoadMasterHash` method in `PasswordUtils` performs defensive validation when reading `master.dat`. It checks the file length before reading, validates each length prefix before using it to read bytes, and confirms that the number of bytes read, matches the expected length. This prevents crashes or undefined behaviour if the file is corrupted or truncated.

The **LoadPasswords** method in `MainForm` wraps the entire file read in a try-catch block. If the `passwords.dat` file is corrupted for any reason the application catches the exception, notifies the user, clears the corrupted file and continues running cleanly rather than crashing.

4.7 Review

The development of the program was reviewed at each stage before moving on to the next feature. Several changes were made directly as a result of these reviews.

1. After implementing the initial master password system the code was reviewed against NIST SP 800-63B guidance. This review identified that the original implementation had no password policy, meaning a user could set a single character master password that would undermine all the cryptographic protections. `SecurityPolicy` was added specifically as a result of this review.
2. After implementing the binary file format for `master.dat`, I reviewed the program code after running the full read and write cycle. This review revealed the `EndOfStreamException` caused by the mismatched read and write structure. The length-prefixed format was introduced as a direct result.

3. After implementing the hash verification logic, I reviewed the code for security vulnerabilities. This review identified the timing attack risk in the original equality comparison, and this led me to change the solution from using an equality comparison method to a `FixedTimeEquals` method.
4. After implementing the clipboard copy feature the code was reviewed during usability testing. This review revealed that the 10 second timer was too short in practice, so I extended the time to 15 seconds.
5. The overall architecture was also reviewed mid-development when the codebase was becoming difficult to manage with all logic inside the form classes. This review led to the separation of `PasswordUtils`, `LockoutManager` and `SecurityPolicy` into independent static classes, which made the remaining development significantly more manageable and made each class independently testable.

In summary, the result of testing and implemented changes in each stage resulted in the codebase being more efficient. The testing helped to establish a user-friendly solution and interface whilst validating the output.

5. Testing: Developmental

5.1 Evidence of Testing During Iterative Development

Testing was carried out manually at each stage of development by running the application and verifying it behaved as expected. The following issues were identified and resolved during the development process.

EndOfStreamException Reading master.dat

When testing the master password system for the first time after implementing the binary file format, the application threw an `EndOfStreamException` when attempting to read back the stored hash. The cause was a mismatch between how the data was being written and how it was being read. The write operation was storing the salt and hash without any length prefixes, so the read operation had no way of knowing where the salt ended and the hash began. The fix was standardising the format to write salt length, salt bytes, hash length and hash bytes in sequence, with the read operation expecting exactly that layout. After this change the file read correctly every time.

CryptographicOperations.FixedTimeEquals Not Available

During the second phase of development a compiler error appeared when trying to use `CryptographicOperations.FixedTimeEquals` for timing-safe hash comparison. The method was not available in the framework version being used at the time. The fix was writing a manual byte-by-byte comparison that always iterates through the full array

regardless of where a mismatch occurs, preserving the timing-safe property without relying on the unavailable API.

Password Policy Not Enforced Initially

During testing it was discovered that the application would accept any string as a master password including single characters, with no rejection. This was identified when comparing the implementation against NIST SP 800-63B guidance, which makes clear that a weak master password undermines all cryptographic protections regardless of how strong the encryption is. SecurityPolicy was added to enforce minimum length, character type requirements and common password rejection as a direct result of this review.

Blank Entry Validation Missing

During testing, I observed that submitting the add entry form with blank fields would encrypt and store an empty entry, which then caused errors when trying to load and display it on the next run. A null and whitespace check was added to the add entry button handler to catch this before it reaches the encryption logic, displaying a clear error message instead.

Clipboard Timer Duration

During usability testing the clipboard auto-clear timer was set to 10 seconds initially. In practice this was not long enough to switch to another application and paste, particularly on slower machines. The timer was extended to 15 seconds based on this observation.

5.2 Failed Tests and Remedial Actions

During developmental testing several features did not work correctly on the first attempt and required fixes before moving on.

The binary file read for master.dat failed on the first test run throwing an EndOfStreamException. The root cause was identified as the write and read operations using different structures. The fix was adding length prefixes to both fields so the reader always knows how many bytes to expect. The file was then tested again and read back correctly.

The hash verification initially used a standard array equality check that returned as soon as a mismatch was found. This was identified during a code review as a potential timing vulnerability. The fix was replacing it with FixedTimeEquals which always iterates the full array. The change was verified by checking that the comparison still correctly rejected wrong passwords while no longer returning faster on early mismatches.

The add entry form initially accepted blank fields and stored empty encrypted entries. This was caught during testing when reloading the application after submitting a blank entry caused an error during the load. The fix was adding null and whitespace checks before the entry is processed. This was tested by submitting blank fields again and confirming the error message appeared and no entry was added.

The clipboard timer was set to 10 seconds initially. During testing this consistently did not give enough time to switch applications and paste before the clipboard was cleared. The interval was increased to 15 seconds and retested, which resolved the issue in all tested scenarios.

6. Testing: Evaluative

6.1 Post Development Testing

The following tests were carried out on the completed application to verify that each success criterion from section 2.7 was met.

Num	Description	Test Carried Out	Expected Outcome	Actual Outcome	Status
1	Master password set and verified using PBKDF2	Entered valid password on first run, re-entered on login, tested incorrect password, inspected master.dat	Correct password grants access, incorrect rejected, master.dat contains only binary data	Correct password granted access, incorrect rejected with error message, master.dat showed only binary hash data with no readable text	Pass
2	Password entries encrypted with AES-256	Added entry and opened passwords.dat in text editor, attempted login with wrong key	File shows ciphertext only, wrong key fails	passwords.dat contained no readable text, attempting to open with wrong key produced garbled output	Pass
3	Password complexity	Attempted passwords that were too short, missing symbols,	Each violation rejected with specific reason	All violations rejected with correct specific messages	Pass

	rules enforced	missing uppercase, common words			
4	Exponential lockout activates after failed logins	Entered wrong password multiple times and measured delays between attempts	Delays increase as 5s, 10s, 20s up to 300s, reset on successful login	Delays increased correctly, lockout persisted after application restart, reset on successful login	Pass
5	Clipboard auto-clears after 15 seconds	Copied password and waited 15 seconds, checked clipboard, also tested with app minimised	Password cleared from clipboard after timeout	Clipboard cleared after 15 seconds in all tested scenarios including with app minimised	Pass
6	Password reveal auto-hides after 10 seconds	Clicked Show and waited 10 seconds without interacting	Password visible briefly then hidden behind asterisks	Password auto-hid after 10 seconds correctly	Pass
7	Master password can be changed securely	Changed password via ChangePasswordForm, tested old and new password on login, checked entries still accessible	New password works, old rejected, all entries still accessible	All three outcomes confirmed	Pass
8	Application handles erroneous inputs	Submitted empty add form, manually corrupted passwords.dat and reloaded	Error messages shown, no crash	Empty form showed validation error, corrupted file was caught gracefully with error message and reset	Pass
9	UI consistently themed in dark mode	Opened all forms and checked for consistent colour scheme	All forms display consistent dark theme	Dark theme applied correctly across all forms with no unstyled elements	Pass
10	Secure cleanup on form close	Opened ViewPasswordForm, closed it, checked fields in debug session	No passwords remain visible or in memory after close	Fields cleared and timers stopped on close confirmed via debug inspection	Pass
11	Session key rotates every 30 seconds	Ran application with debug logging enabled and monitored console output	New session key fingerprint appears every 30 seconds, UI remains responsive	Rotation messages appeared at correct intervals with distinct fingerprints, no UI interruption	Pass

6.2 Usability Testing

The application was tested end to end by going through the full user journey from setting a master password through to adding, viewing, copying and deleting entries.

The single master password model worked well in practice. Setting up the application on first run took under a minute. The password policy feedback was specific enough that the correct password was set on the second attempt after the first was rejected for missing a symbol.

The clipboard auto-clear was the most noticeable security feature during usability testing. The security notice that appears immediately after copying made the behaviour transparent, and the 15 second window was sufficient in all tested scenarios to switch to another application and paste. The feature worked correctly even when the application was minimised during the countdown.

The lockout feedback messages were clear and informative. Displaying the exact number of seconds remaining rather than a generic blocked message meant it was obvious what was happening and how long to wait. Two consecutive failed attempts produced delays of 5 seconds and then 10 seconds, confirming the exponential increase was working correctly.

The timed password reveal worked as intended. The 10 second window was sufficient to read a password in all tested cases. The auto-hide on focus loss also worked correctly, hiding the password immediately when switching to another window without waiting for the timer.

One usability issue identified during testing was that there is no way to recover a forgotten master password. If the master password is lost all stored entries are permanently inaccessible. This is a known and documented limitation of the local-only design and is acceptable for the target user group of security conscious individuals, but it would be a significant problem for more casual users.

7. Evaluation

7.1 Evaluation Against Success Criteria

Num	Criteria	Evidence	Outcome
1	Master password set and verified using PBKDF2	Correct password granted access, incorrect rejected, master.dat contains only binary hash data	Fully met
2	AES 256 encryption working correctly	passwords.dat showed only ciphertext, decryption with wrong key failed	Fully met
3	Session key rotates every 30 seconds	Debug log showed rotation messages with distinct fingerprints at 30 second intervals	Fully met
4	Password complexity rules enforced	All weak password attempts rejected with specific messages	Fully met
5	Exponential lockout working correctly	Delays increased correctly, persisted after restart, reset on successful login	Fully met
6	Clipboard clears after 15 seconds	Clipboard confirmed empty after timeout in all test scenarios	Fully met

7	Password reveal auto-hides after 10 seconds	Field reverted to asterisks after 10 seconds in all tests	Fully met
8	Master password change works securely	Old password rejected, new password accepted, all entries still accessible	Fully met
9	Application handles erroneous inputs	Error messages shown for all invalid inputs, no crashes observed	Fully met
10	UI consistently themed in dark mode	Dark theme applied correctly across all forms with no unstyled elements	Fully met
11	Secure cleanup on form close	Fields cleared and timers stopped on close confirmed via debug inspection	Fully met

The entire success criteria listed in the table above were implemented in the final version of the application. The criteria that required the most work to satisfy were the password complexity enforcement, which was not part of the original design and was only added after reviewing the implementation against NIST guidance, and the input validation for the add entry form, which was only identified as necessary during testing rather than the design phase. Both of these being identified late is worth acknowledging as a limitation of the initial planning rather than something that was always intended.

7.2 Evaluation of Usability Features

The dark theme was consistently applied across all five forms with no unstyled elements found during testing. It makes the application feel immediately distinct from a standard Windows application which helps users recognise they are in a security sensitive context.

The clipboard auto-clear was the most practically useful security feature during testing. The 15 second window struck a good balance between usability and security. The security notice shown immediately after copying made the behaviour transparent rather than surprising, which is important because a user whose clipboard is silently cleared without warning might assume something has gone wrong.

The lockout feedback messages were specific and informative. Showing the exact remaining wait time rather than a generic blocked message meant legitimate users understood what was happening. During testing two consecutive wrong passwords produced delays of 5 and 10 seconds respectively, confirming the exponential increase was clearly communicated to the user.

The password policy feedback was direct and actionable. Telling the user exactly which rule was violated meant the correct password was set on the second attempt during testing rather than

requiring multiple tries. A generic rejection message would have made this process significantly more frustrating.

The timed password reveal worked well in practice. The 10 second window was sufficient for reading a stored password in all tested cases. The auto-hide on focus loss was a useful addition that was not in the original design but was added during development after it became clear that switching windows could leave a password visible indefinitely without it.

One usability issue identified during testing was that there is no way to recover a forgotten master password. For the target user group of security conscious individuals this is an acceptable trade-off since local only storage with no recovery mechanism is the entire point of the design. For more casual users it would be a significant problem and would need addressing before the application could be used by a wider audience.

7.3 Limitations and Maintenance Considerations

The most significant security limitation is that the common password list in SecurityPolicy only checks against three hardcoded patterns. A proper implementation would reference a file containing the most frequently used passwords, which would significantly reduce the risk of users setting a weak master password that happens to pass the current checks. This would be a straightforward improvement that only requires changing SecurityPolicy without touching any other class.

The master password change does not re-encrypt existing vault entries. This was a deliberate design decision to avoid the memory exposure of decrypting all entries simultaneously, but it does mean entries added before a password change remain encrypted with the old key. If an attacker obtained the old master key before the change they would still be able to decrypt those entries. A more complete implementation would re-encrypt entries one at a time on password change, limiting the plaintext exposure to a single entry at any moment.

There is no password generator built into the application. Users still have to create their own passwords manually before adding them to the vault, which often leads to weaker choices. This was omitted to keep the scope focused on secure storage but it is the most obvious missing feature from a usability perspective.

The codebase is well structured for future maintenance. The separation between PasswordUtils, LockoutManager, SecurityPolicy and the UI forms means changes to any one area do not require changes to the others. The cryptographic parameters like iteration count and key size are defined as constants rather than magic numbers, so they can be updated in one place without searching through the codebase. Adding new features like a password generator or a search function would only require adding new code rather than modifying existing logic.

7.4 Potential Improvements

The first improvement would be expanding the common password list in `SecurityPolicy` to reference an external file rather than a hardcoded list. This would significantly improve the quality of the policy without changing the underlying validation logic and could be implemented by replacing the three `Contains` checks with a file lookup against a comprehensive list of the most frequently breached passwords.

The second improvement would be adding a built-in password generator. This is the most commonly requested feature in password managers and its absence means users still have to generate passwords manually. The implementation would use the same `RandomNumberGenerator` already present in `PasswordUtils` with a configurable character set and length, keeping it consistent with the existing cryptographic approach.

The third improvement would be adding a search or filter function to the main vault list. As the number of stored entries grows, scrolling through a flat list becomes impractical. A simple text filter on the site name would significantly improve usability for users with large vaults without requiring any changes to the underlying storage format.

References and Resources

Bavly, G. (2023). *The Challenges of Seamless Encryption*. IBM Community. Available at: <https://community.ibm.com/community/user/blogs/guy-bavly/2023/02/09/the-challenges-of-seamless-encryption>

Bitwarden. (2024). *Bitwarden Open Source Password Manager*. Available at: <https://bitwarden.com>

Cowan, N. (2010). *The Magical Mystery Four: How is Working Memory Capacity Limited, and Why? Current Directions in Psychological Science*. Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC2864034/>

Intel. (2010). *Intel Advanced Encryption Standard New Instructions (AES-NI)*. Intel Developer Zone. Available at:

<https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-encryption-standard-instructions-aes-ni.html>

KeePass. (2024). *KeePass Password Safe*. Available at: <https://keepass.info>

LastPass. (2022). *Notice of Recent Security Incident*. LastPass Blog. Available at: <https://blog.lastpass.com/2022/12/notice-of-recent-security-incident/>

Microsoft. (2024). *.NET Cryptography Model*. Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/dotnet/standard/security/cryptography-model>

NIST. (2002). *Federal Information Processing Standard 140-2: Security Requirements for Cryptographic Modules*. National Institute of Standards and Technology. Available at: <https://csrc.nist.gov/pubs/fips/140-2/upd2/final>

NIST. (2017). *NIST Special Publication 800-63B: Digital Identity Guidelines*. National Institute of Standards and Technology. Available at: <https://pages.nist.gov/800-63-3/sp800-63b.html>

NordPass. (2023). *How Many Passwords Does the Average Person Have?* Available at: <https://nordpass.com/blog/how-many-passwords-does-average-person-have/>

OWASP. (2021). *Authentication Cheat Sheet*. OWASP Foundation. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

OWASP. (2021). *Password Storage Cheat Sheet*. OWASP Foundation. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Viezelyte, K. (2023). *Juggling Security: How Many Passwords Does the Average Person Have?* NordPass. Available at: <https://nordpass.com/blog/how-many-passwords-does-average-person-have/>

Craig'n'Dave, youtube study materials. <https://www.youtube.com/@craigndave>

Appendix

Program.cs

```
using System;

using System.Windows.Forms;

namespace NEA_Project_test
{
    static class Program
```



```

{
    // STAThread tells Windows this application uses a single UI thread
    // This is required for all Windows Forms applications to work correctly
    [STAThread]
    static void Main()
    {
        // Makes buttons, textboxes and other controls look like
        // a modern Windows application rather than the old Windows XP style
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);

        // Show the login screen first before anything else
        // The using keyword ensures the form is removed from memory
        // properly once it is no longer needed
        using (var loginForm = new LoginForm())
        {
            // ShowDialog opens the form and pauses here until it closes
            // DialogResult.OK means the user logged in successfully
            if (loginForm.ShowDialog() == DialogResult.OK)
            {
                // Login was successful so open the main password vault
                // We pass the encryption key that was derived from the
                // master password during login so the vault can decrypt entries
                Application.Run(new MainForm(loginForm.DerivedKey));
            }
            else
            {
                // User closed or cancelled the login screen
                // so close the application entirely
                Application.Exit();
            }
        }
    }
}

```

```
    }  
    }  
    }  
}
```

LoginForm.cs

// This is the login screen. It handles two situations:

// 1. First time the app is opened - creates a new master password

// 2. Every time after that - verifies the entered password against the stored hash

```
using System;
```

```
using System.Windows.Forms;
```

```
namespace NEA_Project_test
```

```
{
```

```
    public partial class LoginForm : Form
```

```
    {
```

```
        // This property holds the encryption key after a successful login
```

```
        // It is passed to MainForm so the vault can encrypt and decrypt entries
```

```
        // It is stored in memory only and never written to disk
```

```
        public byte[]? DerivedKey { get; private set; }
```

```
        public LoginForm()
```

```
        {
```

```
            // Set up all the buttons, textboxes and labels defined in the designer file
```

```
            InitializeComponent();
```

```
            // Apply the dark colour scheme to this form
```

```
        ApplyTheme();
    }

    private void btnLogin_Click(object sender, EventArgs e)
    {
        // Get whatever the user typed in the password box

        // Trim removes any accidental spaces at the start or end

        // The ?? operator means if the result is null use an empty string instead
        string password = txtPassword.Text?.Trim() ?? string.Empty;

        // Do not proceed if the password box is empty
        if (string.IsNullOrEmpty(password))
        {
            MessageBox.Show("Please enter your master password.");
            return;
        }

        // Check if the user is currently locked out before doing anything else
        // There is no point processing the password if they have to wait anyway
        int wait = LockoutManager.GetRemainingLockoutSeconds();
        if (wait > 0)
        {
            MessageBox.Show($"Too many failed attempts. Try again in {wait} seconds.");
            return;
        }

        // These will hold the salt and hash loaded from master.dat
        byte[] salt;
```

```
byte[] storedHash;

// Try to load the stored master password hash from disk
// If the file does not exist this is the first time the app has been run
if (!PasswordUtils.LoadMasterHash(out salt, out storedHash))
{
    // First run - validate the password meets complexity rules
    // before storing it, because a weak master password would
    // undermine all the encryption even if the rest is secure
    if (!SecurityPolicy.ValidatePasswordComplexity(password, out string reason))
    {
        // Tell the user exactly which rule their password broke
        MessageBox.Show("Password policy: " + reason);
        return;
    }

    // Generate a brand new random salt for this master password
    // The salt ensures two users with the same password get different hashes
    salt = PasswordUtils.GenerateSalt();

    // Hash the password with the salt using PBKDF2
    // The hash is what gets stored, never the actual password
    storedHash = PasswordUtils.HashPassword(password, salt);

    // Save the salt and hash to master.dat on disk
    PasswordUtils.SaveMasterHash(salt, storedHash);

    // Ask the user to log in again now that the password is set
```

```
        MessageBox.Show("Master password created. Please log in again.");

        txtPassword.Clear();

        return;
    }

    // Returning user - check if the entered password matches the stored hash
    // VerifyPassword uses a timing-safe comparison to prevent timing attacks
    if (!PasswordUtils.VerifyPassword(password, salt, storedHash))
    {
        // Wrong password - record this failed attempt in the lockout file
        LockoutManager.RegisterFailedAttempt();

        // Check if this failed attempt has triggered a lockout
        int nextWait = LockoutManager.GetRemainingLockoutSeconds();
        if (nextWait > 0)
        {
            // Tell the user how long they need to wait
            MessageBox.Show($"Incorrect password. Try again in {nextWait} seconds.");
        }
        else
        {
            MessageBox.Show("Incorrect password.");
        }

        // Clear the password box so they can try again
        txtPassword.Clear();

        return;
    }
}
```

```

// Login was successful

// Reset the lockout counter since the correct password was entered
LockoutManager.Reset();

// Derive the encryption key from the password and salt
// This key is used to encrypt and decrypt vault entries
// It is kept in memory only for the duration of this session
DerivedKey = PasswordUtils.DeriveEncryptionKey(password, salt);

// Set the result to OK so Program.cs knows login succeeded
DialogResult = DialogResult.OK;

Close();
}

private void ApplyTheme()
{
    // Set the background of the whole form to a dark grey colour
    // The numbers represent Red, Green, Blue values (0-255)
    this.BackColor = Color.FromArgb(30, 30, 30);

    // Loop through every control on the form and style it
    foreach (Control c in Controls)
    {
        // Make all labels white so they are visible on the dark background
        if (c is Label lbl)
            lbl.ForeColor = Color.White;
    }
}

```

```

        // Style buttons to match the dark theme
        if (c is Button btn)
        {
            btn.BackColor = Color.FromArgb(58, 58, 58);

            btn.ForeColor = Color.White;

            btn.FlatStyle = FlatStyle.Flat;

            btn.FlatAppearance.BorderSize = 0;
        }

        // Style read-only textboxes to blend into the dark background
        if (c is TextBox tb && tb.ReadOnly)
        {
            tb.BackColor = this.BackColor;

            tb.ForeColor = Color.White;

            tb.BorderStyle = BorderStyle.None;
        }
    }
}
}
}
}
}
}
}

```

LoginForm.Designer.cs

```

// This file is automatically generated by Visual Studio when you design the form visually
// It contains all the code that creates and positions the buttons, labels and textboxes
// You should not edit this file manually as Visual Studio will overwrite your changes

```

```

namespace NEA_Project_test
{
    partial class LoginForm

```

```
{  
    // This variable keeps track of all the components on the form  
    // so they can be properly cleaned up when the form closes  
    private System.ComponentModel.IContainer components = null;  
  
    // This method runs automatically when the form is closed  
    // It frees up any memory or resources the form was using  
    protected override void Dispose(bool disposing)  
    {  
        if (disposing && (components != null))  
        {  
            components.Dispose();  
        }  
        base.Dispose(disposing);  
    }  
  
    // This method sets up all the visual elements on the form  
    // such as where each control is positioned and how big it is  
    private void InitializeComponent()  
    {  
        // Create each control that appears on the form  
        this.lblPassword = new System.Windows.Forms.Label();  
        this.txtPassword = new System.Windows.Forms.TextBox();  
        this.btnLogin = new System.Windows.Forms.Button();  
  
        // Temporarily pause the layout engine while we set everything up  
        // This prevents the form from trying to redraw itself mid-setup  
        this.SuspendLayout();  
    }  
}
```



```
// ---- Password Label ----
```

```
// This is the text that says "Master Password" next to the input box
```

```
this.lblPassword.AutoSize = true;
```

```
this.lblPassword.Location = new System.Drawing.Point(30, 30);
```

```
this.lblPassword.Name = "lblPassword";
```

```
this.lblPassword.Size = new System.Drawing.Size(87, 13);
```

```
this.lblPassword.TabIndex = 0;
```

```
this.lblPassword.Text = "Master Password";
```

```
// ---- Password TextBox ----
```

```
// This is the box where the user types their master password
```

```
// UseSystemPasswordChar = true means it shows dots instead of letters
```

```
this.txtPassword.Location = new System.Drawing.Point(130, 27);
```

```
this.txtPassword.Name = "txtPassword";
```

```
this.txtPassword.Size = new System.Drawing.Size(180, 20);
```

```
this.txtPassword.TabIndex = 1;
```

```
this.txtPassword.UseSystemPasswordChar = true;
```

```
// ---- Login Button ----
```

```
// This is the button the user clicks to attempt login
```

```
this.btnLogin.Location = new System.Drawing.Point(130, 65);
```

```
this.btnLogin.Name = "btnLogin";
```

```
this.btnLogin.Size = new System.Drawing.Size(75, 25);
```

```
this.btnLogin.TabIndex = 2;
```

```
this.btnLogin.Text = "Login";
```

```
this.btnLogin.UseVisualStyleBackColor = true;
```

```
// Wire up the click event so pressing the button calls btnLogin_Click
// in LoginForm.cs
this.btnLogin.Click += new System.EventHandler(this.btnLogin_Click);

// ---- Form Settings ----

// AcceptButton means pressing Enter on the keyboard triggers the login button
this.AcceptButton = this.btnLogin;

this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;

// Set the overall size of the login window
this.ClientSize = new System.Drawing.Size(350, 120);

// Add all the controls to the form so they actually appear
this.Controls.Add(this.btnLogin);
this.Controls.Add(this.txtPassword);
this.Controls.Add(this.lblPassword);

// FixedDialog means the window cannot be resized or maximised
this.FormBorderStyle = System.Windows.Forms.FormBorderStyle.FixedDialog;
this.MaximizeBox = false;
this.MinimizeBox = false;
this.Name = "LoginForm";

// CenterScreen means the window appears in the middle of the screen
this.StartPosition = System.Windows.Forms.FormStartPosition.CenterScreen;
this.Text = "Login";
```

```

        // Resume the layout engine now that everything is set up
        this.ResumeLayout(false);
        this.PerformLayout();
    }

    // Declare the controls as private fields so they can be accessed
    // from anywhere in the LoginForm class
    private System.Windows.Forms.Label lblPassword;
    private System.Windows.Forms.TextBox txtPassword;
    private System.Windows.Forms.Button btnLogin;
}
}

```

MainForm.cs

```

// This is the main password vault screen that appears after a successful login // It
allows the user to add, view and delete password entries // It also handles session key
rotation, clipboard clearing and the dark theme

using System; using System.Collections.Generic; using System.IO; using System.Linq;
using System.Text.RegularExpressions; using System.Windows.Forms;

// We specify the full name here because there are two Timer classes available // and
we want the Windows Forms one, not the System.Threading one using Timer =
System.Windows.Forms.Timer;

namespace NEA_Project_test { public partial class MainForm : Form { // The master key
is the encryption key derived from the master password // It is used to encrypt and
decrypt all vault entries // It never changes while the app is running and is never written
to disk private readonly byte[] masterKey;

    // The session key is a temporary key that rotates every 30 seconds
    // It is used for short-term in-memory operations
    // Rotating it limits how long a stolen key remains useful
    private byte[] sessionKey;

    // The name of the file where encrypted passwords are stored on disk

```

```

private readonly string passwordFile = "passwords.dat";

// This list holds all the decrypted password entries in memory
// while the app is running
private readonly List<string> passwordEntries = new();

// This timer fires every 30 seconds to rotate the session key
private readonly Timer rotationTimer;

// This timer fires 15 seconds after a password is copied
// to clear it from the clipboard automatically
private readonly Timer clipboardClearTimer;

// Stores the value that was most recently copied to the clipboard
// so we can check it is still there before clearing
private string? lastClipboardSecret;

public MainForm(byte[] key)
{
    // Set up all the buttons, labels and listbox from the designer file
    InitializeComponent();

    // Store the master key passed in from the login form
    masterKey = key;

    // Start with the session key equal to the master key
    sessionKey = key;

    // Set the title bar text of the window
    Text = "Password Manager";

    // Load any previously saved password entries from disk
    LoadPasswords();

    // Set up the session key rotation timer
    // Interval is in milliseconds so 30000 = 30 seconds
    rotationTimer = new Timer { Interval = 30_000 };
    rotationTimer.Tick += RotateSessionKey;
    rotationTimer.Start();

    // Set up the clipboard clear timer

```

```

// It does not start automatically, only when a password is copied
clipboardClearTimer = new Timer();
clipboardClearTimer.Interval = 15_000; // 15 seconds
clipboardClearTimer.Tick += ClipboardClearTimer_Tick;

// Apply the dark colour scheme
ApplyTheme();
}

// This method runs when the clipboard clear timer fires
private void ClipboardClearTimer_Tick(object? sender, EventArgs e)
{
    try
    {
        // Only clear the clipboard if it still contains what we copied
        // If the user has copied something else in the meantime
        // we do not want to clear that
        if (!string.IsNullOrEmpty(lastClipboardSecret) &&
            Clipboard.ContainsText() &&
            Clipboard.GetText() == lastClipboardSecret)
        {
            Clipboard.Clear();
        }
    }
    catch
    {
        // The clipboard can sometimes be locked by Windows or another app
        // If that happens we just move on rather than crashing
    }
    finally
    {
        // Always clear our stored reference and stop the timer
        // regardless of whether the clipboard clear succeeded
        lastClipboardSecret = null;
        clipboardClearTimer.Stop();
    }
}

// This method runs every 30 seconds to rotate the session key
private void RotateSessionKey(object? sender, EventArgs e)
{

```

```

// Derive a new session key using the master key and the current time
// Using the current time as salt means each rotation produces a different key
sessionKey = PasswordUtils.DeriveEncryptionKey(
    Convert.ToHexString(masterKey),
    BitConverter.GetBytes(DateTime.UtcNow.Ticks));

// Print the first 8 characters of the new key's fingerprint to the console
// This is only visible in debug mode and helps confirm rotation is working
Console.WriteLine($"Session key rotated -> {Fingerprint(sessionKey)}");
}

// Creates a short readable fingerprint of a key for debug logging
// Takes the first 8 characters of the key's SHA256 hash in hex format
private static string Fingerprint(byte[] key)
{
    return Convert.ToHexString(
        System.Security.Cryptography.SHA256.HashData(key)
    ).Substring(0, 8);
}

// Checks whether a site string looks like a valid URL
// Returns true if it matches the URL pattern, false otherwise
private Boolean IsSiteValid(string site)
{
    if (string.IsNullOrEmpty(site)) return false;

    // This pattern matches most common URL formats
    // including with or without http, www, and various domain endings
    const string pattern = @"(?:http[s]?://.)*(?:www\.)?[-a-zA-Z0-9@%.*+~#=#]{2,256}\.[a-z]{2,6}\b(?:[-a-zA-Z0-9@:%*+.~#?&//=]*)";
    return Regex.IsMatch(site, pattern, RegexOptions.IgnoreCase);
}

// This method runs when the user clicks the Add Entry button
private void btnAdd_Click(object sender, EventArgs e)
{
    string site = txtSite.Text.Trim();

    // Check the site field contains a valid URL before proceeding
    if (!IsSiteValid(site))
    {

```

```

        MessageBox.Show("Please enter a valid URL for the site.");
        return;
    }

    string username = txtUsername.Text.Trim();
    string password = txtPassword.Text.Trim();

    // Check the site and password fields are not empty
    // Username is optional so we do not check it here
    if (string.IsNullOrEmpty(site) || string.IsNullOrEmpty(password))
    {
        MessageBox.Show("Site and Password cannot be empty.");
        return;
    }

    // Combine the three fields into a single string separated by pipes
    // This is the format used to store each entry
    string entry = $"{site} | {username} | {password}";

    // Add to the in-memory list
    passwordEntries.Add(entry);

    // Save the updated list to disk in encrypted form
    SavePasswords();

    // Refresh the listbox on screen to show the new entry
    RefreshPasswordList();

    // Clear the input fields ready for the next entry
    txtSite.Clear();
    txtUsername.Clear();
    txtPassword.Clear();
}

// This method runs when the user clicks the Delete Entry button
private void btnDelete_Click(object sender, EventArgs e)
{
    int index = listBoxPasswords.SelectedIndex;

    // Check something is actually selected before trying to delete
    if (index < 0)

```

```

    {
        MessageBox.Show("Please select an entry to delete.");
        return;
    }

    // Ask the user to confirm before permanently deleting
    var confirm = MessageBox.Show(
        "Are you sure you want to delete this entry?",
        "Confirm Delete",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Warning);

    // If the user clicked No, do nothing
    if (confirm != DialogResult.Yes)
        return;

    // Remove the entry from the in-memory list
    passwordEntries.RemoveAt(index);

    // Save the updated list back to disk
    SavePasswords();

    // Refresh the listbox to reflect the deletion
    RefreshPasswordList();
}

// Encrypts all entries and saves them to passwords.dat
private void SavePasswords()
{
    // Encrypt each entry individually using the master key
    // Select creates a new list where each entry has been encrypted
    List<byte[]> encryptedList = passwordEntries
        .Select(entry => PasswordUtils.EncryptString(entry, masterKey))
        .ToList();

    // Open the file for writing, creating it if it does not exist
    using var fs = new FileStream(passwordFile, FileMode.Create);
    using var bw = new BinaryWriter(fs);

    // Write how many entries there are first
    // so when we read back we know how many to expect

```



```

        bw.Write(encryptedList.Count);

        // Write each encrypted entry with its length prefix
        foreach (var enc in encryptedList)
        {
            bw.Write(enc.Length);
            bw.Write(enc);
        }
    }

    // Reads and decrypts all entries from passwords.dat
    private void LoadPasswords()
    {
        // Clear the in-memory list before loading
        passwordEntries.Clear();

        // If the file does not exist yet there is nothing to load
        if (!File.Exists(passwordFile))
            return;

        try
        {
            using var fs = new FileStream(passwordFile, FileMode.Open);
            using var br = new BinaryReader(fs);

            // Read how many entries were saved
            int count = br.ReadInt32();

            for (int i = 0; i < count; i++)
            {
                // Read the length of this entry first
                int length = br.ReadInt32();

                // Read exactly that many bytes
                byte[] enc = br.ReadBytes(length);

                // If we got fewer bytes than expected the file is corrupted
                if (enc.Length != length)
                    throw new EndOfStreamException();

                // Decrypt the entry using the master key

```

```

        string decrypted = PasswordUtils.DecryptBytes(enc, masterKey);

        // Add the decrypted entry to the in-memory list
        passwordEntries.Add(decrypted);
    }
}
catch
{
    // If anything goes wrong reading the file, warn the user
    // and reset rather than crashing the application
    MessageBox.Show("Password file is corrupted and will be reset.");
    passwordEntries.Clear();
    File.Delete(passwordFile);
}

// Update the listbox to show whatever was loaded
RefreshPasswordList();
}

// Updates the listbox on screen to show the current list of sites
private void RefreshPasswordList()
{
    listBoxPasswords.Items.Clear();

    foreach (var entry in passwordEntries)
    {
        // Each entry is stored as "site | username | password"
        // We only show the site name in the list for security
        string site = entry.Split('|')[0].Trim();
        listBoxPasswords.Items.Add(site);
    }
}

// This method runs when the user clicks the View Entry button
private void btnView_Click(object sender, EventArgs e)
{
    // Check something is selected in the list
    if (listBoxPasswords.SelectedIndex == -1)
    {
        MessageBox.Show("Please select a password entry first.");
        return;
    }
}

```

```

    }

    // Ask the user to re-enter their master password before showing
    // the entry, as an extra layer of security
    using var confirmForm = new ConfirmPasswordForm();
    if (confirmForm.ShowDialog() != DialogResult.OK || !confirmForm.IsConfirmed)
        return;

    // Get the full entry for the selected item
    string entry = passwordEntries[listBoxPasswords.SelectedIndex];

    // Split it back into its three parts
    string[] parts = entry.Split('|');

    string site = parts[0].Trim();
    string username = parts.Length > 1 ? parts[1].Trim() : "";
    string password = parts.Length > 2 ? parts[2].Trim() : "";

    // Open the view password form with the decrypted details
    using (var viewForm = new ViewPasswordForm(site, username, password))
    {
        viewForm.ShowDialog();
    }
}

// This method runs when the user clicks the Change Master Password button
private void btnChangePassword_Click(object sender, EventArgs e)
{
    using var changeForm = new ChangePasswordForm();
    if (changeForm.ShowDialog() == DialogResult.OK)
    {
        MessageBox.Show("Master password changed successfully.");
    }
}

// Applies the dark colour scheme to all controls on this form
private void ApplyTheme()
{
    this.BackColor = Color.FromArgb(30, 30, 30);

    foreach (Control c in Controls)

```

```

{
    if (c is Label lbl)
        lbl.ForeColor = Color.White;

    if (c is Button btn)
    {
        btn.BackColor = Color.FromArgb(58, 58, 58);
        btn.ForeColor = Color.White;
        btn.FlatStyle = FlatStyle.Flat;
        btn.FlatAppearance.BorderSize = 0;
    }

    if (c is TextBox tb && tb.ReadOnly)
    {
        tb.BackColor = this.BackColor;
        tb.ForeColor = Color.White;
        tb.BorderStyle = BorderStyle.None;
    }
}
}
}

}

```

MainForm.Designer.cs

// This file is automatically generated by Visual Studio when you design the form visually

// It creates and positions all the buttons, labels, textboxes and the password list

// Do not edit this file manually as Visual Studio will overwrite your changes

```

namespace NEA_Project_test
{
    partial class MainForm
    {
        // Keeps track of all components so they can be cleaned up when the form closes
        private System.ComponentModel.IContainer components = null;
    }
}

```

```
// Runs automatically when the form closes to free up memory
protected override void Dispose(bool disposing)
{
    if (disposing && (components != null))
    {
        components.Dispose();
    }
    base.Dispose(disposing);
}

private void InitializeComponent()
{
    // Create all the controls that appear on the main vault screen
    txtSite = new TextBox();
    txtUsername = new TextBox();
    txtPassword = new TextBox();
    btnAdd = new Button();
    btnView = new Button();
    btnChangePassword = new Button();
    listBoxPasswords = new ListBox();
    lblSite = new Label();
    lblUsername = new Label();
    lblPassword = new Label();
    btnDelete = new Button();

    // Pause layout while we set everything up
    SuspendLayout();
}
```

```
// ---- Site TextBox ----
```

```
// Where the user types the website address for a new entry
```

```
txtSite.Location = new Point(95, 25);
```

```
txtSite.Name = "txtSite";
```

```
txtSite.Size = new Size(180, 23);
```

```
txtSite.TabIndex = 0;
```

```
// ---- Username TextBox ----
```

```
// Where the user types the username for a new entry
```

```
txtUsername.Location = new Point(95, 55);
```

```
txtUsername.Name = "txtUsername";
```

```
txtUsername.Size = new Size(180, 23);
```

```
txtUsername.TabIndex = 1;
```

```
// ---- Password TextBox ----
```

```
// Where the user types the password for a new entry
```

```
// UseSystemPasswordChar = true means it shows dots instead of letters
```

```
txtPassword.Location = new Point(95, 85);
```

```
txtPassword.Name = "txtPassword";
```

```
txtPassword.Size = new Size(180, 23);
```

```
txtPassword.TabIndex = 2;
```

```
txtPassword.UseSystemPasswordChar = true;
```

```
// ---- Add Entry Button ----
```

```
// Clicking this saves the new entry to the vault
```

```
btnAdd.Location = new Point(95, 123);
```

```
btnAdd.Name = "btnAdd";
```

```
btnAdd.Size = new Size(55, 55);

btnAdd.TabIndex = 3;

btnAdd.Text = "Add Entry";

btnAdd.UseVisualStyleBackColor = true;

btnAdd.Click += btnAdd_Click;


// ---- View Entry Button ----

// Clicking this shows the selected entry after re-authentication

btnView.Location = new Point(156, 123);

btnView.Name = "btnView";

btnView.Size = new Size(58, 55);

btnView.TabIndex = 4;

btnView.Text = "View Entry";

btnView.UseVisualStyleBackColor = true;

btnView.Click += btnView_Click;


// ---- Change Master Password Button ----

// Clicking this opens the change password form

btnChangePassword.Location = new Point(95, 184);

btnChangePassword.Name = "btnChangePassword";

btnChangePassword.Size = new Size(180, 25);

btnChangePassword.TabIndex = 5;

btnChangePassword.Text = "Change Master Password";

btnChangePassword.UseVisualStyleBackColor = true;

btnChangePassword.Click += btnChangePassword_Click;


// ---- Password List Box ----

// This shows the list of saved site names on the right side of the screen
```

```
// Only site names are shown here, never usernames or passwords
```

```
listBoxPasswords.FormattingEnabled = true;
```

```
listBoxPasswords.ItemHeight = 15;
```

```
listBoxPasswords.Location = new Point(310, 25);
```

```
listBoxPasswords.Name = "listBoxPasswords";
```

```
listBoxPasswords.Size = new Size(250, 184);
```

```
listBoxPasswords.TabIndex = 6;
```

```
// ---- Site Label ----
```

```
// The text that says "Site:" next to the site input box
```

```
lblSite.AutoSize = true;
```

```
lblSite.Location = new Point(20, 28);
```

```
lblSite.Name = "lblSite";
```

```
lblSite.Size = new Size(29, 15);
```

```
lblSite.TabIndex = 7;
```

```
lblSite.Text = "Site:";
```

```
// ---- Username Label ----
```

```
// The text that says "Username:" next to the username input box
```

```
lblUsername.AutoSize = true;
```

```
lblUsername.Location = new Point(20, 58);
```

```
lblUsername.Name = "lblUsername";
```

```
lblUsername.Size = new Size(63, 15);
```

```
lblUsername.TabIndex = 8;
```

```
lblUsername.Text = "Username:";
```

```
// ---- Password Label ----
```

```
// The text that says "Password:" next to the password input box
```



```
lblPassword.AutoSize = true;

lblPassword.Location = new Point(20, 88);

lblPassword.Name = "lblPassword";

lblPassword.Size = new Size(60, 15);

lblPassword.TabIndex = 9;

lblPassword.Text = "Password:";


// ---- Delete Entry Button ----

// Clicking this permanently removes the selected entry from the vault

btnDelete.Location = new Point(220, 123);

btnDelete.Name = "btnDelete";

btnDelete.Size = new Size(55, 55);

btnDelete.TabIndex = 10;

btnDelete.Text = "Delete Entry";

btnDelete.UseVisualStyleBackColor = true;

btnDelete.Click += btnDelete_Click;


// ---- Form Settings ----

// Set the default background colour before the theme is applied

BackColor = Color.DarkGray;


// Set the overall size of the main vault window

ClientSize = new Size(600, 220);


// Add all controls to the form so they actually appear on screen

Controls.Add(btnDelete);

Controls.Add(lblPassword);

Controls.Add(lblUsername);
```

```
Controls.Add(lblSite);

Controls.Add(listBoxPasswords);

Controls.Add(btnChangePassword);

Controls.Add(btnView);

Controls.Add(btnAdd);

Controls.Add(txtPassword);

Controls.Add(txtUsername);

Controls.Add(txtSite);

Name = "MainForm";

Text = "Password Manager";


// Resume the layout engine now everything is configured

ResumeLayout(false);

PerformLayout();

}


// Declare all controls as private fields so they can be
// accessed from anywhere in the MainForm class

private System.Windows.Forms.TextBox txtSite;

private System.Windows.Forms.TextBox txtUsername;

private System.Windows.Forms.TextBox txtPassword;

private System.Windows.Forms.Button btnAdd;

private System.Windows.Forms.Button btnView;

private System.Windows.Forms.Button btnChangePassword;

private System.Windows.Forms.ListBox listBoxPasswords;

private System.Windows.Forms.Label lblSite;

private System.Windows.Forms.Label lblUsername;

private System.Windows.Forms.Label lblPassword;
```

```
        private Button btnDelete;

    }

}
```

ViewPasswordForm.cs

```
// This form shows the details of a single selected password entry
// It includes the site name, username and the password itself
// The password is hidden by default and can only be revealed temporarily
// It also handles copying the password to the clipboard and clearing it automatically
```

```
using System;

using System.Windows.Forms;

using Timer = System.Windows.Forms.Timer;

namespace NEA_Project_test
{
    public partial class ViewPasswordForm : Form
    {
        // The actual password stored here in memory only
        // It is never written to disk from this form
        private readonly string password;
```

```
// Timer that automatically clears the clipboard 15 seconds
// after the password is copied
private Timer clipboardTimer;

// Stores what we copied to the clipboard so we can check
// it is still there before clearing it
private string? clipboardValue;

// Tracks whether the password is currently visible or hidden
private bool passwordVisible = false;

// Timer that automatically hides the password 10 seconds
// after the user clicks Show
private Timer revealTimer;

public ViewPasswordForm(string site, string username, string password)
{
    // Set up all the labels and buttons from the designer file
    InitializeComponent();

    // Fill in the site and username labels with the actual values
    lblSite.Text = site;
    lblUsername.Text = username;

    // Put the password in the textbox but hide it behind dots
    // ReadOnly means the user cannot type in or modify this box
    // ShortcutsEnabled = false prevents copying via keyboard shortcuts
```

```
// TabStop = false means pressing Tab skips over this box
txtPassword.Text = password;
txtPassword.ReadOnly = true;
txtPassword.UseSystemPasswordChar = true;
txtPassword.ShortcutsEnabled = false;
txtPassword.TabStop = false;

// Store the password so we can use it for copying later
this.password = password;

// Show a message in the security label at the bottom
lblSecurityStatus.Text = "Clipboard clears in 10s";

// Set up the clipboard clear timer
// It does not start until the user copies a password
clipboardTimer = new Timer();
clipboardTimer.Interval = 15_000; // 15 seconds
clipboardTimer.Tick += ClipboardTimer_Tick;

// Set up the reveal timer
// It does not start until the user clicks Show
revealTimer = new Timer();
revealTimer.Interval = 10_000; // 10 seconds
revealTimer.Tick += RevealTimer_Tick;

// Apply the dark colour scheme
ApplyTheme();
}
```

```
// This method runs when the user clicks the Show or Hide button
```

```
private void btnToggleVisibility_Click(object sender, EventArgs e)
```

```
{
```

```
    // Flip the visibility state
```

```
    passwordVisible = !passwordVisible;
```

```
    // If passwordVisible is true show the actual characters
```

```
    // If false show dots instead
```

```
    txtPassword.UseSystemPasswordChar = !passwordVisible;
```

```
    // Update the button text to reflect the current state
```

```
    btnToggleVisibility.Text = passwordVisible ? "Hide" : "Show";
```

```
    if (passwordVisible)
```

```
    {
```

```
        // Restart the reveal timer so the user always gets
```

```
        // a full 10 seconds from the last time they clicked Show
```

```
        revealTimer.Stop();
```

```
        revealTimer.Start();
```

```
    }
```

```
}
```

```
// This method runs when the reveal timer fires after 10 seconds
```

```
private void RevealTimer_Tick(object? sender, EventArgs e)
```

```
{
```

```
    // Hide the password again automatically
```

```
    txtPassword.UseSystemPasswordChar = true;
```

```
        btnToggleVisibility.Text = "Show";
        passwordVisible = false;
        revealTimer.Stop();
    }

    // This method runs when the user clicks the Copy Password button
    private void btnCopyPassword_Click(object sender, EventArgs e)
    {
        // Copy the password to the Windows clipboard
        Clipboard.SetText(password);

        // Store what we copied so we can verify it is still there
        // before clearing it 15 seconds later
        clipboardValue = password;

        // Reset and start the clipboard clear timer
        clipboardTimer.Stop();
        clipboardTimer.Start();

        // Tell the user the password has been copied and will be cleared
        MessageBox.Show(
            "Password copied to clipboard.\nIt will be cleared automatically.",
            "Security Notice",
            MessageBoxButtons.OK,
            MessageBoxIcon.Information);
    }

    // This method runs when the clipboard timer fires after 15 seconds
```

```
private void ClipboardTimer_Tick(object? sender, EventArgs e)
{
    try
    {
        // Only clear the clipboard if it still contains what we copied
        // The user might have copied something else since then
        // and we do not want to clear that
        if (!string.IsNullOrEmpty(clipboardValue) &&
            Clipboard.ContainsText() &&
            Clipboard.GetText() == clipboardValue)
        {
            Clipboard.Clear();
        }
    }
    catch
    {
        // The clipboard can be locked by Windows or another application
        // If that happens we just move on rather than crashing
    }
    finally
    {
        // Always clear our stored reference and stop the timer
        // regardless of whether the clipboard clear succeeded
        clipboardValue = null;
        clipboardTimer.Stop();
    }
}
```



```
// This method runs when the user clicks the Close button
```

```
private void btnClose_Click(object sender, EventArgs e)
```

```
{
```

```
    // Run the security cleanup before closing
```

```
    SecureCleanup();
```

```
    Close();
```

```
}
```

```
// Clears all sensitive data from the form before it closes
```

```
private void SecureCleanup()
```

```
{
```

```
    // Clear the password textbox so nothing remains in the UI
```

```
    txtPassword.Text = string.Empty;
```

```
    passwordVisible = false;
```

```
    // Stop both timers so no callbacks fire after the form is gone
```

```
    revealTimer.Stop();
```

```
    clipboardTimer.Stop();
```

```
}
```

```
// This runs automatically if the user clicks on another window
```

```
// while this form is open
```

```
protected override void OnDeactivate(EventArgs e)
```

```
{
```

```
    base.OnDeactivate(e);
```

```
    // Hide the password immediately when the user switches away
```

```
    // This prevents the password being left visible on an unattended screen
```

```
txtPassword.UseSystemPasswordChar = true;

btnToggleVisibility.Text = "Show";

passwordVisible = false;

revealTimer.Stop();
}

// Applies the dark colour scheme to all controls on this form
private void ApplyTheme()
{
    this.BackColor = Color.FromArgb(30, 30, 30);

    foreach (Control c in Controls)
    {
        if (c is Label lbl)
            lbl.ForeColor = Color.White;

        if (c is Button btn)
        {
            btn.BackColor = Color.FromArgb(58, 58, 58);
            btn.ForeColor = Color.White;
            btn.FlatStyle = FlatStyle.Flat;
            btn.FlatAppearance.BorderSize = 0;
        }

        if (c is TextBox tb && tb.ReadOnly)
        {
            tb.BackColor = this.BackColor;
            tb.ForeColor = Color.White;
        }
    }
}
```

```
        tb.BorderStyle = BorderStyle.None;
    }
}
}
}
```

ViewPasswordForm.Designer.cs

// This file is automatically generated by Visual Studio when you design the form visually

// It creates and positions all the labels, textbox and buttons on the view password screen

// Do not edit this file manually as Visual Studio will overwrite your changes

```
namespace NEA_Project_test
```

```
{
```

```
    partial class ViewPasswordForm
```

```
    {
```

```
        // Keeps track of all components so they can be cleaned up when the form closes
```

```
        private System.ComponentModel.IContainer components = null;
```

```
        // Runs automatically when the form closes to free up memory
```

```
        protected override void Dispose(bool disposing)
```

```
        {
```

```
            if (disposing && (components != null))
```

```
{
    components.Dispose();
}
base.Dispose(disposing);
}

private void InitializeComponent()
{
    // Create all the controls that appear on the view password screen
    lblSiteTitle = new Label();
    lblUsernameTitle = new Label();
    lblPasswordTitle = new Label();
    lblSecurityTitle = new Label();
    lblSite = new Label();
    lblUsername = new Label();
    txtPassword = new TextBox();
    btnToggleVisibility = new Button();
    btnCopyPassword = new Button();
    btnClose = new Button();
    lblSecurityStatus = new Label();

    // Pause layout while we set everything up
    SuspendLayout();

    // ---- Site Title Label ----

    // The static text that says "Site :" on the left side
    lblSiteTitle.AutoSize = true;
    lblSiteTitle.Location = new Point(166, 91);
```

```
lblSiteTitle.Name = "lblSiteTitle";

lblSiteTitle.Size = new Size(32, 15);

lblSiteTitle.TabIndex = 0;

lblSiteTitle.Text = "Site :";


// ---- Username Title Label ----

// The static text that says "Username :" on the left side

lblUsernameTitle.AutoSize = true;

lblUsernameTitle.Location = new Point(166, 115);

lblUsernameTitle.Name = "lblUsernameTitle";

lblUsernameTitle.Size = new Size(66, 15);

lblUsernameTitle.TabIndex = 1;

lblUsernameTitle.Text = "Username :";


// ---- Password Title Label ----

// The static text that says "Password :" on the left side

lblPasswordTitle.AutoSize = true;

lblPasswordTitle.Location = new Point(166, 144);

lblPasswordTitle.Name = "lblPasswordTitle";

lblPasswordTitle.Size = new Size(63, 15);

lblPasswordTitle.TabIndex = 2;

lblPasswordTitle.Text = "Password :";


// ---- Security Title Label ----

// The static text that says "Security :" on the left side

lblSecurityTitle.AutoSize = true;

lblSecurityTitle.Location = new Point(166, 171);

lblSecurityTitle.Name = "lblSecurityTitle";
```

```
lblSecurityTitle.Size = new Size(55, 15);  
lblSecurityTitle.TabIndex = 3;  
lblSecurityTitle.Text = "Security :";  
  
// ---- Site Value Label ----  
  
// This shows the actual site name, populated from the entry data  
lblSite.AutoSize = true;  
lblSite.Location = new Point(240, 91);  
lblSite.Name = "lblSite";  
lblSite.Size = new Size(0, 15);  
lblSite.TabIndex = 4;  
  
// ---- Username Value Label ----  
  
// This shows the actual username, populated from the entry data  
lblUsername.AutoSize = true;  
lblUsername.Location = new Point(240, 115);  
lblUsername.Name = "lblUsername";  
lblUsername.Size = new Size(0, 15);  
lblUsername.TabIndex = 5;  
  
// ---- Password TextBox ----  
  
// This shows the password, hidden behind dots by default  
// ReadOnly means the user cannot type in or edit it  
// ShortcutsEnabled = false prevents copying via keyboard shortcuts  
txtPassword.Location = new Point(240, 141);  
txtPassword.Name = "txtPassword";  
txtPassword.ReadOnly = true;  
txtPassword.ShortcutsEnabled = false;
```

```
txtPassword.Size = new Size(114, 23);

txtPassword.TabIndex = 6;

txtPassword.TabStop = false;

txtPassword.UseSystemPasswordChar = true;


// ---- Toggle Visibility Button ----

// Clicking this shows or hides the password for 10 seconds

btnToggleVisibility.Location = new Point(146, 219);

btnToggleVisibility.Name = "btnToggleVisibility";

btnToggleVisibility.Size = new Size(75, 23);

btnToggleVisibility.TabIndex = 8;

btnToggleVisibility.Text = "Show ";

btnToggleVisibility.UseVisualStyleBackColor = true;

btnToggleVisibility.Click += btnToggleVisibility_Click;


// ---- Copy Password Button ----

// Clicking this copies the password to the clipboard

// The clipboard will be cleared automatically after 15 seconds

btnCopyPassword.Location = new Point(240, 219);

btnCopyPassword.Name = "btnCopyPassword";

btnCopyPassword.Size = new Size(114, 23);

btnCopyPassword.TabIndex = 9;

btnCopyPassword.Text = "Copy Password ";

btnCopyPassword.UseVisualStyleBackColor = true;

btnCopyPassword.Click += btnCopyPassword_Click;


// ---- Close Button ----

// Clicking this runs SecureCleanup and closes the form
```

```
btnClose.Location = new Point(375, 219);
btnClose.Name = "btnClose";
btnClose.Size = new Size(75, 23);
btnClose.TabIndex = 10;
btnClose.Text = "Close";
btnClose.UseVisualStyleBackColor = true;
btnClose.Click += btnClose_Click;

// ---- Security Status Label ----
// This shows the clipboard countdown message after copying
lblSecurityStatus.AutoSize = true;
lblSecurityStatus.Location = new Point(240, 171);
lblSecurityStatus.Name = "lblSecurityStatus";
lblSecurityStatus.Size = new Size(0, 15);
lblSecurityStatus.TabIndex = 11;

// ---- Form Settings ----
AutoScaleDimensions = new.SizeF(7F, 15F);
AutoScaleMode = AutoScaleMode.Font;

// Set the overall size of the view password window
ClientSize = new Size(592, 345);

// Add all controls to the form so they appear on screen
Controls.Add(lblSecurityStatus);
Controls.Add(btnClose);
Controls.Add(btnCopyPassword);
Controls.Add(btnToggleVisibility);
```



```
Controls.Add(txtPassword);
Controls.Add(lblUsername);
Controls.Add(lblSite);
Controls.Add(lblSecurityTitle);
Controls.Add(lblPasswordTitle);
Controls.Add(lblUsernameTitle);
Controls.Add(lblSiteTitle);
Name = "ViewPasswordForm";
Text = "ViewPasswordForm";

// Resume the layout engine now everything is configured
ResumeLayout(false);
PerformLayout();
}

// Declare all controls as private fields so they can be
// accessed from anywhere in the ViewPasswordForm class
private Label lblSiteTitle;
private Label lblUsernameTitle;
private Label lblPasswordTitle;
private Label lblSecurityTitle;
private Label lblSite;
private Label lblUsername;
private TextBox txtPassword;
private Button btnToggleVisibility;
private Button btnCopyPassword;
private Button btnClose;
private Label lblSecurityStatus;
```

```
}  
}
```

ChangePasswordForm.cs

```
using System;  
  
using System.Windows.Forms;  
  
namespace NEA_Project_test  
{  
    public partial class ChangePasswordForm : Form  
    {  
        public ChangePasswordForm()  
        {  
            // Set up all the buttons and textboxes from the designer file  
            InitializeComponent();  
  
            // Apply the dark colour scheme  
            ApplyTheme();  
        }  
    }  
}
```

```
private void btnOK_Click(object sender, EventArgs e)
{
    // Get whatever the user typed in each field
    string oldPass = txtOldPassword.Text;

    // Trim removes any accidental spaces at the start or end
    string newPass = txtNewPassword.Text?.Trim();
    string confirmPass = txtConfirmPassword.Text;

    // Make sure none of the fields are left empty
    if (string.IsNullOrEmpty(oldPass) ||
        string.IsNullOrEmpty(newPass) ||
        string.IsNullOrEmpty(confirmPass))
    {
        MessageBox.Show("All fields are required.");
        return;
    }

    // Check the new password meets the complexity rules
    // before doing anything else
    string reason;
    if (!SecurityPolicy.ValidatePasswordComplexity(newPass, out reason))
    {
        MessageBox.Show("New password does not meet complexity rules: " + reason);
        return;
    }

    // Make sure the new password and confirmation match
```

```
if (newPass != confirmPass)
{
    MessageBox.Show("New passwords do not match.");
    return;
}

// Try to load the currently stored master password hash from disk
if (!PasswordUtils.LoadMasterHash(out byte[] salt, out byte[] storedHash))
{
    MessageBox.Show("No existing master password found.");
    return;
}

// Verify the old password is correct before allowing the change
// This prevents someone changing the password on an unlocked device
if (!PasswordUtils.VerifyPassword(oldPass, salt, storedHash))
{
    MessageBox.Show("Old password is incorrect.");
    return;
}

// Generate a brand new random salt for the new password
// Never reuse the old salt even if it would be valid
byte[] newSalt = PasswordUtils.GenerateSalt();

// Hash the new password with the new salt
byte[] newHash = PasswordUtils.HashPassword(newPass, newSalt);
```

```
// Save the new salt and hash to disk, overwriting the old ones
PasswordUtils.SaveMasterHash(newSalt, newHash);

MessageBox.Show("Master password successfully changed.");

// Set the result to OK so the calling form knows it succeeded
DialogResult = DialogResult.OK;
Close();
}
```

```
private void btnCancel_Click(object sender, EventArgs e)
{
    // User cancelled so set result to Cancel and close
    DialogResult = DialogResult.Cancel;
    Close();
}
```

```
// Applies the dark colour scheme to all controls on this form
private void ApplyTheme()
{
    this.BackColor = Color.FromArgb(30, 30, 30);

    foreach (Control c in Controls)
    {
        if (c is Label lbl)
            lbl.ForeColor = Color.White;

        if (c is Button btn)
```

```

        {
            btn.BackColor = Color.FromArgb(58, 58, 58);
            btn.ForeColor = Color.White;
            btn.FlatStyle = FlatStyle.Flat;
            btn.FlatAppearance.BorderSize = 0;
        }

        if (c is TextBox tb && tb.ReadOnly)
        {
            tb.BackColor = this.BackColor;
            tb.ForeColor = Color.White;
            tb.BorderStyle = BorderStyle.None;
        }
    }
}

```

ChangePasswordForm.Designer.cs

```
using System;
```

```
namespace NEA_Project_test
```

```

{
    partial class ChangePasswordForm
    {
        // Keeps track of all components so they can be cleaned up when the form closes
        private System.ComponentModel.IContainer components = null;

        // Runs automatically when the form closes to free up memory
    }
}

```

```
protected override void Dispose(bool disposing)
```

```
{  
    if (disposing && (components != null))  
    {  
        components.Dispose();  
    }  
    base.Dispose(disposing);  
}
```

```
private void InitializeComponent()
```

```
{  
    // Create all the controls that appear on the change password screen  
    this.lblNewPassword = new System.Windows.Forms.Label();  
    this.txtNewPassword = new System.Windows.Forms.TextBox();  
    this.lblConfirmPassword = new System.Windows.Forms.Label();  
    this.txtConfirmPassword = new System.Windows.Forms.TextBox();  
    this.btnSave = new System.Windows.Forms.Button();  
    this.btnCancel = new System.Windows.Forms.Button();  
  
    // Pause layout while we set everything up  
    this.SuspendLayout();  
  
    // ---- New Password Label ----  
    // The static text that says "New Password:" on the left side  
    this.lblNewPassword.AutoSize = true;  
    this.lblNewPassword.Location = new System.Drawing.Point(30, 30);  
    this.lblNewPassword.Name = "lblNewPassword";  
    this.lblNewPassword.Size = new System.Drawing.Size(98, 16);
```

```
this.lblNewPassword.TabIndex = 0;

this.lblNewPassword.Text = "New Password:";


// ---- Confirm Password Label ----

// The static text that says "Confirm Password:" on the left side
this.lblConfirmPassword.AutoSize = true;

this.lblConfirmPassword.Location = new System.Drawing.Point(30, 70);

this.lblConfirmPassword.Name = "lblConfirmPassword";

this.lblConfirmPassword.Size = new System.Drawing.Size(115, 16);

this.lblConfirmPassword.TabIndex = 2;

this.lblConfirmPassword.Text = "Confirm Password:";


// ---- Old Password TextBox ----

// Where the user types their current master password to verify identity
this.txtOldPassword = new System.Windows.Forms.TextBox();

this.txtOldPassword.Location = new System.Drawing.Point(150, 30);

this.txtOldPassword.Name = "txtOldPassword";

this.txtOldPassword.Size = new System.Drawing.Size(200, 20);

this.txtOldPassword.UseSystemPasswordChar = true;


// ---- New Password TextBox ----

// Where the user types their desired new master password
this.txtNewPassword = new System.Windows.Forms.TextBox();

this.txtNewPassword.Location = new System.Drawing.Point(150, 70);

this.txtNewPassword.Name = "txtNewPassword";

this.txtNewPassword.Size = new System.Drawing.Size(200, 20);

this.txtNewPassword.UseSystemPasswordChar = true;
```



```
// ---- Confirm Password TextBox ----

// Where the user types their new password a second time to confirm
// Both new password fields must match before the change is accepted
this.txtConfirmPassword = new System.Windows.Forms.TextBox();
this.txtConfirmPassword.Location = new System.Drawing.Point(150, 20);
this.txtConfirmPassword.Name = "txtConfirmPassword";
this.txtConfirmPassword.Size = new System.Drawing.Size(200, 20);
this.txtConfirmPassword.UseSystemPasswordChar = true;


// ---- Save Button ----

// Clicking this validates all fields and saves the new master password
this.btnSave.Location = new System.Drawing.Point(160, 110);
this.btnSave.Name = "btnSave";
this.btnSave.Size = new System.Drawing.Size(80, 28);
this.btnSave.TabIndex = 4;
this.btnSave.Text = "Save";
this.btnSave.UseVisualStyleBackColor = true;


// Wire up the click event to the btnOK_Click handler in the main file
this.btnSave.Click += new System.EventHandler(this.btnOK_Click);


// ---- Cancel Button ----

// Clicking this closes the form without making any changes
this.btnCancel.Location = new System.Drawing.Point(260, 110);
this.btnCancel.Name = "btnCancel";
this.btnCancel.Size = new System.Drawing.Size(80, 28);
this.btnCancel.TabIndex = 5;
this.btnCancel.Text = "Cancel";
```

```
this.btnCancel.UseVisualStyleBackColor = true;

// Close the form directly when cancel is clicked
this.btnCancel.Click += (s, e) => this.Close();

// ---- Form Settings ----

// AcceptButton means pressing Enter triggers the Save button
this.AcceptButton = this.btnSave;
this.CancelButton = this.btnCancel;
this.AutoScaleDimensions = new System.Drawing.SizeF(8F, 16F);
this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;

// Set the overall size of the change password window
this.ClientSize = new System.Drawing.Size(380, 170);

// Add all controls to the form so they appear on screen
this.Controls.Add(this.btnCancel);
this.Controls.Add(this.btnSave);
this.Controls.Add(this.txtConfirmPassword);
this.Controls.Add(this.lblConfirmPassword);
this.Controls.Add(this.txtNewPassword);
this.Controls.Add(this.lblNewPassword);
this.Name = "ChangePasswordForm";
this.Text = "Change Master Password";

// Resume the layout engine now everything is configured
this.ResumeLayout(false);
this.PerformLayout();
```

```

    }

    // Declare all controls as private fields so they can be
    // accessed from anywhere in the ChangePasswordForm class
    private System.Windows.Forms.Label lblNewPassword;
    private System.Windows.Forms.TextBox txtNewPassword;
    private System.Windows.Forms.Label lblConfirmPassword;
    private System.Windows.Forms.TextBox txtConfirmPassword;
    private System.Windows.Forms.Button btnSave;
    private System.Windows.Forms.Button btnOK;
    private System.Windows.Forms.Button btnCancel;
    private System.Windows.Forms.TextBox txtOldPassword;
}
}

```

ConfirmPasswordForm.cs

```

using System;
using System.IO;
using System.Windows.Forms;

namespace NEA_Project_test
{
    public partial class ConfirmPasswordForm : Form
    {
        // This property tells the calling form whether the user
        // successfully confirmed their master password
        // It starts as false and is only set to true if verification succeeds
        public bool IsConfirmed { get; private set; } = false;
    }
}

```

```
public ConfirmPasswordForm()
{
    // Set up all the buttons and textboxes from the designer file
    InitializeComponent();

    // Apply the dark colour scheme
    ApplyTheme();
}

private void btnOK_Click(object sender, EventArgs e)
{
    // Get whatever the user typed in the password box
    string enteredPassword = txtPassword.Text;

    // Do not proceed if the password box is empty
    if (string.IsNullOrEmpty(enteredPassword))
    {
        MessageBox.Show("Password cannot be blank.");
        return;
    }

    // Try to load the stored master password hash from disk
    if (PasswordUtils.LoadMasterHash(out byte[] salt, out byte[] storedHash))
    {
        // Check if the entered password matches the stored hash
        if (PasswordUtils.VerifyPassword(enteredPassword, salt, storedHash))
        {
            // Password is correct so mark as confirmed
        }
    }
}
```

```
        IsConfirmed = true;

        // Set result to OK so the calling form knows verification passed
        DialogResult = DialogResult.OK;
        Close();
    }
    else
    {
        // Wrong password so clear the box and let them try again
        MessageBox.Show("Incorrect master password.");
        txtPassword.Clear();
    }
}
else
{
    // master.dat does not exist which should not normally happen
    // since the user had to log in to get here
    MessageBox.Show("No master password set yet.");
}
}

private void btnCancel_Click(object sender, EventArgs e)
{
    // User cancelled so mark as not confirmed
    IsConfirmed = false;

    // Set result to Cancel so the calling form knows to abort
    DialogResult = DialogResult.Cancel;
```

```
        Close();
    }

    // Applies the dark colour scheme to all controls on this form
    private void ApplyTheme()
    {
        this.BackColor = Color.FromArgb(30, 30, 30);

        foreach (Control c in Controls)
        {
            if (c is Label lbl)
            {
                lbl.ForeColor = Color.White;
            }

            if (c is Button btn)
            {
                btn.BackColor = Color.FromArgb(58, 58, 58);
                btn.ForeColor = Color.White;
                btn.FlatStyle = FlatStyle.Flat;
                btn.FlatAppearance.BorderSize = 0;
            }

            if (c is TextBox tb && tb.ReadOnly)
            {
                tb.BackColor = this.BackColor;
                tb.ForeColor = Color.White;
                tb.BorderStyle = BorderStyle.None;
            }
        }
    }
}
```

```
    }  
  }  
}
```

ConfirmPasswordForm.Designer.cs

```
using System;
```

```
namespace NEA_Project_test
```

```
{
```

```
    partial class ConfirmPasswordForm
```

```
    {
```

```
        // Keeps track of all components so they can be cleaned up when the form closes
```

```
        private System.ComponentModel.IContainer components = null;
```

```
        // Runs automatically when the form closes to free up memory
```

```
        protected override void Dispose(bool disposing)
```

```
        {
```

```
            if (disposing && (components != null))
```

```
            {
```

```
                components.Dispose();
```

```
            }
```

```
        base.Dispose(disposing);
    }

    private void InitializeComponent()
    {
        // Create all the controls that appear on the confirm password screen

        label1 = new Label();
        txtPassword = new TextBox();
        btnCancel = new Button();
        btnOK = new Button();

        // Pause layout while we set everything up
        SuspendLayout();

        // ---- Instruction Label ----
        // The text that tells the user what to do
        label1.AutoSize = true;
        label1.Location = new Point(108, 45);
        label1.Margin = new Padding(4, 0, 4, 0);
        label1.Name = "label1";
        label1.Size = new Size(126, 15);
        label1.TabIndex = 0;
        label1.Text = "Enter Master Password";

        // ---- Password TextBox ----
        // Where the user types their master password to confirm their identity
        // UseSystemPasswordChar = true means it shows dots instead of letters
        txtPassword.Location = new Point(90, 80);
```



```
txtPassword.Margin = new Padding(4, 3, 4, 3);

txtPassword.Name = "txtPassword";

txtPassword.Size = new Size(171, 23);

txtPassword.TabIndex = 1;

txtPassword.UseSystemPasswordChar = true;


// ---- Cancel Button ----

// Clicking this closes the form without confirming
// and sets IsConfirmed to false in the main file

btnCancel.Location = new Point(193, 121);

btnCancel.Margin = new Padding(4, 3, 4, 3);

btnCancel.Name = "btnCancel";

btnCancel.Size = new Size(68, 27);

btnCancel.TabIndex = 4;

btnCancel.Text = "Cancel";

btnCancel.UseVisualStyleBackColor = true;

btnCancel.Click += btnCancel_Click;


// ---- OK Button ----

// Clicking this verifies the entered password against the stored hash
// If correct it sets IsConfirmed to true and closes the form

btnOK.Location = new Point(90, 121);

btnOK.Margin = new Padding(4, 3, 4, 3);

btnOK.Name = "btnOK";

btnOK.Size = new Size(56, 27);

btnOK.TabIndex = 5;

btnOK.Text = "OK";

btnOK.UseVisualStyleBackColor = true;
```

```
btnOK.Click += btnOK_Click;

// ---- Form Settings ----

AutoScaleDimensions = new.SizeF(7F, 15F);

AutoScaleMode = AutoScaleMode.Font;

// Set the overall size of the confirm password window
ClientSize = new Size(360, 201);

// Add all controls to the form so they appear on screen
Controls.Add(btnOK);
Controls.Add(btnCancel);
Controls.Add(txtPassword);
Controls.Add(label1);
Margin = new Padding(4, 3, 4, 3);
Name = "ConfirmPasswordForm";
Text = "ConfirmPasswordForm";

// Resume the layout engine now everything is configured
ResumeLayout(false);
PerformLayout();
}

// Declare all controls as private fields so they can be
// accessed from anywhere in the ConfirmPasswordForm class
private System.Windows.Forms.Label label1;
private System.Windows.Forms.TextBox txtPassword;
private System.Windows.Forms.Button btnCancel;
```

```
        private System.Windows.Forms.Button btnOK;
    }
}
```

PasswordUtils.cs

```
using System;
using System.IO;
using System.Security.Cryptography;
using System.Text;

namespace NEA_Project_test
{
    public static class PasswordUtils
    {
        // The name of the file where the master password hash is stored
        // Stored as a constant so it only needs to be changed in one place
        private const string MasterFile = "master.dat";

        // =====
        // SALT GENERATION
        // =====

        // Generates a random salt of the specified size in bytes
        // A salt is a random value added to a password before hashing
        // so that two users with the same password get completely different hashes
```

```

public static byte[] GenerateSalt(int size = 16)
{
    // Create an empty byte array to hold the salt
    var salt = new byte[size];

    // RandomNumberGenerator is cryptographically secure
    // unlike System.Random which is predictable and seeded by time
    // Predictable random numbers would make the salt useless
    using (var rng = RandomNumberGenerator.Create())
    {
        // Fill the array with unpredictable random bytes
        rng.GetBytes(salt);
    }
    return salt;
}

// =====
// HASHING
// =====

// Hashes a password using PBKDF2 with SHA256
// PBKDF2 is a key derivation function designed specifically
// for password hashing because it is deliberately slow
// making brute force attacks much harder
public static byte[] HashPassword(string password, byte[] salt,
    int iterations = 100000, int size = 32)
{
    // Rfc2898DeriveBytes is the .NET implementation of PBKDF2

```

```

// 100,000 iterations means an attacker must run 100,000
// SHA256 operations for every single password guess
using (var pbkdf2 = new Rfc2898DeriveBytes(
    password, salt, iterations, HashAlgorithmName.SHA256))
{
    // Returns a 32 byte derived key

    // The actual password is never stored, only this derived value
    return pbkdf2.GetBytes(size);
}
}

// =====

// SAVE MASTER HASH

// =====

// Saves the salt and hash to master.dat on disk
// The format is: salt length, salt bytes, hash length, hash bytes
// Writing the length before each field means the reader always knows
// exactly how many bytes to read for each part
public static void SaveMasterHash(byte[] salt, byte[] hash)
{
    if (salt == null || hash == null)
        throw new ArgumentNullException("Salt or hash cannot be null.");

    // FileShare.None prevents any other process from reading or
    // writing the file while we are writing to it
    using (var fs = new FileStream(MasterFile, FileMode.Create,
        FileAccess.Write, FileShare.None))

```

```

using (var bw = new BinaryWriter(fs))
{
    // Write the length of the salt first so the reader
    // knows how many bytes to read for the salt
    bw.Write(salt.Length);
    bw.Write(salt);

    // Write the length of the hash first so the reader
    // knows how many bytes to read for the hash
    bw.Write(hash.Length);
    bw.Write(hash);

    // Flush forces all buffered data to be written to disk immediately
    bw.Flush();
}
}

// =====
// LOAD MASTER HASH
// =====

// Reads the salt and hash back from master.dat
// Returns true if successful, false if the file does not exist
// or if anything goes wrong reading it
// The out parameters are set to the loaded values on success
public static bool LoadMasterHash(out byte[] salt, out byte[] hash)
{
    salt = null;

```

```
hash = null;

// If master.dat does not exist this is the first run
// Return false to signal that no master password has been set yet
if (!File.Exists(MasterFile))
    return false;

try
{
    using (var fs = new FileStream(MasterFile, FileMode.Open,
        FileAccess.Read, FileShare.Read))
    using (var br = new BinaryReader(fs))
    {
        // Check there are at least 8 bytes in the file
        // 4 bytes for salt length + 4 bytes for hash length minimum
        // If not the file is empty or corrupted
        if (fs.Length < 8)
            return false;

        // Read the salt length prefix
        int saltLen = br.ReadInt32();

        // Validate the salt length is sensible before reading
        if (saltLen < 0 || saltLen > fs.Length - fs.Position)
            return false;

        // Read exactly as many bytes as the salt length says
        salt = br.ReadBytes(saltLen);
```

```
// If we got fewer bytes than expected the file is truncated
if (salt.Length != saltLen)
    return false;

// Check there are still at least 4 bytes left for the hash length
if (fs.Position + 4 > fs.Length)
    return false;

// Read the hash length prefix
int hashLen = br.ReadInt32();

// Validate the hash length before reading
if (hashLen < 0 || hashLen > fs.Length - fs.Position)
    return false;

// Read exactly as many bytes as the hash length says
hash = br.ReadBytes(hashLen);

// If we got fewer bytes than expected the file is truncated
if (hash.Length != hashLen)
    return false;
}

return true;
}

catch (EndOfStreamException)
{
```



```
// File ended before we finished reading

salt = null;

hash = null;

return false;
}

catch (IOException)
{
    // Something went wrong accessing the file

    salt = null;

    hash = null;

    return false;
}

catch (Exception)
{
    // Catch any other unexpected errors

    salt = null;

    hash = null;

    return false;
}
}


// =====

// VERIFY PASSWORD

// =====


// Compares two byte arrays in a way that always takes the same amount of time
// regardless of where the first difference is found
// A normal comparison stops as soon as it finds a mismatch which leaks
```

```
// information about how close a guess is through the response time
```

```
// This method prevents that timing attack
```

```
private static bool FixedTimeEquals(byte[] a, byte[] b)
```

```
{
```

```
    if (a == null || b == null || a.Length != b.Length) return false;
```

```
    int diff = 0;
```

```
    // XOR each pair of bytes together and OR the result into diff
```

```
    // If any bytes differ then diff will end up non-zero
```

```
    // Crucially the loop always runs all the way through
```

```
    // so it always takes the same amount of time
```

```
    for (int i = 0; i < a.Length; i++) diff |= a[i] ^ b[i];
```

```
    // If diff is still zero every byte matched
```

```
    return diff == 0;
```

```
}
```

```
// Checks whether an entered password matches the stored hash
```

```
// Hashes the entered password with the stored salt and compares
```

```
// using the timing safe FixedTimeEquals method
```

```
public static bool VerifyPassword(string password, byte[] salt,
```

```
    byte[] storedHash)
```

```
{
```

```
    byte[] hash = HashPassword(password, salt);
```

```
    return FixedTimeEquals(hash, storedHash);
```

```
}
```

```

// =====

// DERIVE ENCRYPTION KEY

// =====

// Derives a separate AES encryption key from the master password
// This key is used to encrypt and decrypt vault entries
// It is stored in memory only for the duration of the session
// and is never written to disk
public static byte[] DeriveEncryptionKey(string password,
    byte[] salt, int size = 32)
{
    using (var pbkdf2 = new Rfc2898DeriveBytes(
        password, salt, 100000, HashAlgorithmName.SHA256))
    {
        return pbkdf2.GetBytes(size);
    }
}

// =====

// ENCRYPT STRING

// =====

// Encrypts a plain text string using AES 256 and returns the result
// as a byte array containing the IV followed by the ciphertext
// AES 256 is the industry standard for symmetric encryption
public static byte[] EncryptString(string plainText, byte[] key)
{
    if (plainText == null)

```

```
        throw new ArgumentNullException(nameof(plainText));
    if (key == null)
        throw new ArgumentNullException(nameof(key));

    using (var aes = Aes.Create())
    {
        aes.Key = key;

        // Generate a new random IV for every single encryption operation
        // IV stands for Initialisation Vector, it is a random value that
        // ensures the same password encrypted twice looks completely different
        // This prevents an attacker spotting patterns in the vault file
        aes.GenerateIV();

        using (var ms = new MemoryStream())
        {
            // Write the IV to the front of the output
            // The IV does not need to be secret, only unique
            // Writing it here means it is always available when decrypting
            ms.Write(aes.IV, 0, aes.IV.Length);

            using (var encryptor = aes.CreateEncryptor())

                // CryptoStream applies the encryption as data is written through it
                using (var cs = new CryptoStream(ms, encryptor,
                    CryptoStreamMode.Write))

                    // StreamWriter converts the string to bytes before writing
```

```

        using (var sw = new StreamWriter(cs, Encoding.UTF8))
        {
            sw.Write(plainText);

            // When StreamWriter and CryptoStream are disposed here
            // they flush and finalise the encrypted output
        }

        // Return the IV and ciphertext together as one byte array
        return ms.ToArray();
    }
}

// =====
// DECRYPT BYTES
// =====

// Decrypts a byte array that was previously encrypted by EncryptString
// Reads the IV from the front of the data then decrypts the rest
public static string DecryptBytes(byte[] cipherData, byte[] key)
{
    if (cipherData == null)
        throw new ArgumentNullException(nameof(cipherData));

    if (key == null)
        throw new ArgumentNullException(nameof(key));

    using (var aes = Aes.Create())
    {

```

```
aes.Key = key;

// The IV size in bytes is the block size in bits divided by 8
// For AES this is always 16 bytes
int ivSize = aes.BlockSize / 8;

// If the data is shorter than the IV it cannot be valid
if (cipherData.Length < ivSize)
    throw new ArgumentException("Cipher data too short or invalid.");

// Extract the IV from the front of the byte array
// EncryptString wrote it there so we know exactly where it is
byte[] iv = new byte[ivSize];
Array.Copy(cipherData, iv, iv.Length);
aes.IV = iv;

// Read the ciphertext starting after the IV
using (var ms = new MemoryStream(cipherData, iv.Length,
    cipherData.Length - iv.Length))
using (var decryptor = aes.CreateDecryptor())

// CryptoStream applies decryption as data is read through it
using (var cs = new CryptoStream(ms, decryptor,
    CryptoStreamMode.Read))

// StreamReader converts the decrypted bytes back to a string
using (var sr = new StreamReader(cs, Encoding.UTF8))
{
```

```

        return sr.ReadToEnd();
    }
}
}
}
}
}

```

LockoutManager.cs

```

using System;

using System.IO;

namespace NEA_Project_test
{
    // This class holds the lockout state information
    // It is a simple data container with no logic of its own
    public class LockoutInfo
    {
        // How many times the user has entered the wrong password in a row
        public int FailedCount { get; set; }

        // The exact date and time of the most recent failed attempt
        // Used to calculate how much of the lockout delay has already passed
        public DateTime LastFailed { get; set; }
    }

    public static class LockoutManager
    {
        // The name of the file where lockout state is saved on disk
        // Saving to disk means the lockout survives the app being closed and reopened
    }
}

```

```
// Without this an attacker could simply restart the app to bypass the lockout
```

```
private const string LockoutFile = "lockout.dat";
```

```
// How many seconds to wait after the first failed attempt
```

```
private const int BaseDelaySeconds = 5;
```

```
// The maximum number of seconds to ever wait
```

```
// This prevents the lockout becoming a permanent denial of service
```

```
// where a legitimate user can never get back in
```

```
private const int MaxDelaySeconds = 300; // 5 minutes
```

```
// Reads the current lockout state from disk
```

```
// If the file does not exist yet returns a default state with no failures
```

```
public static LockoutInfo Load()
```

```
{
```

```
    if (!File.Exists(LockoutFile))
```

```
        return new LockoutInfo
```

```
        {
```

```
            FailedCount = 0,
```

```
            LastFailed = DateTime.MinValue
```

```
        };
```

```
    try
```

```
    {
```

```
        using (var fs = new FileStream(LockoutFile,
```

```
            FileMode.Open, FileAccess.Read))
```

```
        using (var br = new BinaryReader(fs))
```

```
        {
```



```

        // Read the failure count stored as a 32 bit integer
        int failedCount = br.ReadInt32();

        // Read the timestamp stored as ticks
        // Ticks are the number of 100 nanosecond intervals
        // since January 1st 0001, used to represent any date and time
        long ticks = br.ReadInt64();

        return new LockoutInfo
        {
            FailedCount = failedCount,
            LastFailed = new DateTime(ticks)
        };
    }
}
catch
{
    // If the file is corrupted or unreadable for any reason
    // reset to a clean state rather than crashing
    return new LockoutInfo
    {
        FailedCount = 0,
        LastFailed = DateTime.MinValue
    };
}

// Saves the current lockout state to disk

```

```
// Called after every failed attempt and after a successful login
public static void Save(LockoutInfo info)
{
    using (var fs = new FileStream(LockoutFile,
        FileMode.Create, FileAccess.Write))
    using (var bw = new BinaryWriter(fs))
    {
        // Write the failure count as a 32 bit integer
        bw.Write(info.FailedCount);

        // Write the timestamp as ticks so it can be read back exactly
        bw.Write(info.LastFailed.Ticks);
    }
}

// Records a failed login attempt by incrementing the count
// and saving the current time to disk
public static void RegisterFailedAttempt()
{
    var info = Load();

    // Increment the failure count
    info.FailedCount = info.FailedCount++;

    // Record exactly when this failure happened
    // This is used later to calculate how much time has passed
    info.LastFailed = DateTime.UtcNow;
}
```

```

        // Save to disk immediately so the lockout persists
        // even if the app is closed right now
        Save(info);
    }

    // Resets the lockout state back to zero after a successful login
    public static void Reset()
    {
        var info = new LockoutInfo
        {
            FailedCount = 0,
            LastFailed = DateTime.MinValue
        };
        Save(info);
    }

    // Returns how many seconds the user must still wait before trying again
    // Returns 0 if they are allowed to attempt login right now
    public static int GetRemainingLockoutSeconds()
    {
        var info = Load();

        // No failed attempts on record so no wait is needed
        if (info.FailedCount <= 0) return 0;

        // Calculate the delay for this number of failures
        // The formula is BaseDelay multiplied by 2 to the power of failures minus 1
        // So: 1 failure = 5s, 2 failures = 10s, 3 = 20s, 4 = 40s and so on
    }

```

```

        double delay = BaseDelaySeconds * Math.Pow(2, info.FailedCount - 1);

        // Never exceed the maximum delay
        if (delay > MaxDelaySeconds) delay = MaxDelaySeconds;

        // Work out how many seconds have already passed since the last failure
        DateTime windowStart = info.LastFailed;
        double elapsed = (DateTime.UtcNow - windowStart).TotalSeconds;

        // Subtract the elapsed time from the total delay
        // to get how many seconds are still remaining
        int remaining = (int)Math.Ceiling(delay - elapsed);

        // Return the remaining seconds, or 0 if the wait is already over
        return remaining > 0 ? remaining : 0;
    }
}

```

SecurityPolicy.cs

```

using System;

using System.Text.RegularExpressions;

namespace NEA_Project_test
{
    public static class SecurityPolicy
    {
        // These constants define the minimum requirements for a valid master password
        // They are defined here as constants so the policy can be tightened

```

```
// in one place without having to hunt through the validation logic below

public const int MinLength = 10;    // Must be at least 10 characters long
public const int MinUpper = 1;      // Must have at least 1 uppercase letter
public const int MinLower = 1;      // Must have at least 1 lowercase letter
public const int MinDigits = 1;     // Must have at least 1 number
public const int MinSymbols = 1;    // Must have at least 1 symbol like ! @ # $

// Checks whether a password meets all the complexity requirements
// Returns true if the password is acceptable
// Returns false and sets the reason string if it is not
// The reason string tells the user exactly which rule they broke
public static bool ValidatePasswordComplexity(string password,
    out string reason)
{
    // Set reason to null to start with
    // It will only be set to a message if a rule is broken
    reason = null;

    // Reject completely empty passwords before checking anything else
    if (string.IsNullOrEmpty(password))
    {
        reason = "Password cannot be empty.";
        return false;
    }

    // Check the minimum length first since this is the most common failure
    if (password.Length < MinLength)
    {
```

```

        reason = $"Password must be at least {MinLength} characters.";
        return false;
    }

    // Count how many of each character type the password contains
    // We do this in a single pass through the password for efficiency
    int uppers = 0, lowers = 0, digits = 0, symbols = 0;
    foreach (char c in password)
    {
        if (char.IsUpper(c)) uppers++;    // A B C D etc
        else if (char.IsLower(c)) lowers++; // a b c d etc
        else if (char.IsDigit(c)) digits++; // 0 1 2 3 etc
        else symbols++;                  // ! @ # $ % etc
    }

    // Check each character type requirement in turn
    // Each one returns a specific message so the user knows
    // exactly what they need to add to their password
    if (uppers < MinUpper)
    {
        reason = $"Password must contain at least {MinUpper} uppercase letter.";
        return false;
    }

    if (lowers < MinLower)
    {
        reason = $"Password must contain at least {MinLower} lowercase letter.";
        return false;
    }

```

```
}
```

```
if (digits < MinDigits)
```

```
{
```

```
    reason = $"Password must contain at least {MinDigits} digit.";
```

```
    return false;
```

```
}
```

```
if (symbols < MinSymbols)
```

```
{
```

```
    reason = $"Password must contain at least {MinSymbols} symbol (e.g. !@#$).";
```

```
    return false;
```

```
}
```

```
// Convert the password to lowercase once so we can check
```

```
// for common patterns without worrying about capitalisation
```

```
string lower = password.ToLowerInvariant();
```

```
// Reject passwords that contain very well known weak patterns
```

```
// Even if a password like "Password1!" passes all the rules above
```

```
// it is still far too easy to guess because attackers specifically
```

```
// try common words and patterns first
```

```
if (lower.Contains("password") ||
```

```
    lower.Contains("12345") ||
```

```
    lower.Contains("qwerty"))
```

```
{
```

```
    reason = "Password is too common. Choose something less guessable.";
```

```
    return false;
```

```
}
```

```
// If we get here the password passed all the checks
```

```
return true;
```

```
}
```

```
}
```

```
}
```